

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 11:28:14
Файлов исходного кода: 82
Суммарный объём кода: 1.17 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
```

Универсальное пособие • полный исходный код

```
-----
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

- ```
        if [ -f "$policy" ]; then
            sudo sed -i 's/rights="none" pattern="\{P
        fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages"
 echo "| \public/export/full_code_all_pages"
 echo "| PNG-страниц | \$(ls tmp/export-pages
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          node-version: "22"
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....

## `index.html`
```

Универсальное пособие • полный исходный код

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
```

```
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
```

```
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```

```
import { chapters } from '../data/content';
```

```
import { Navbar } from '../components/Navbar';
```

```
import { Sidebar } from '../components/Sidebar';
```

```
import { MobileToc } from '../components/MobileToc';
```

```
import { ChapterView } from '../components/ChapterView';
```

```
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-900"
 style={{ width: `${progress}%` }}
 />

```

### Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

const [activeChapterId, setActiveChapterId] = useState<string>();

const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

-) : (

+) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

</>

</main>

+) : (

+ <PythonCompiler />

)}

<SimulatorModal vizId={modalVizId} onClose={() => setModalVizId('')} />

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-900">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-auto" >
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <button
      id="tab-compiler-btn"
      onClick={() => setActiveTab('compiler')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'compiler'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Code className="w-4 h-4" /> Компилятор
    </button>
  </div>
</div>
```

```
-----
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
    >
        {busy ? <Loader2 className="w-4 h-4 animate
    </button>
    </div>
  </div>
</header>
);
};
},

```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
 >
 <Cpu className="w-4 h-4" /> Тренажёры

```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
 module as PyodideModule,
 }).then((module) => module.loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 }
 }, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
 <section className="rounded-2xl border border-slate-200 p-4">
 <div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center gap-2">


```

```

 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 border"
 <label className="block px-4 pt-3 text-slate-500"
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border"
 style={{border: '1px solid #333'}}
 />
 </div>
</section>

<section className="rounded-2xl border border-slate-800 border"
 <div className="flex items-center justify-between"
 <div className="flex items-center gap-2 text-slate-500"
 <Terminal className="w-4 h-4 text-emerald-500"
 />
 <span className={`inline-flex items-center gap-2`}
 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"
 {runtimeReady ? "готов" : "не загружен"}
 />

 </div>
 <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4"
 x] leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 border"
 </div>
</section>
</div>

```

```

+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+ const [output, setOutput] = useState("Нажмите «Запус
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">
+ <button
+ type="button"
+ onClick={reset}

```

```

+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border-slate-200 p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex items-center justify-between">
+ <div className="flex items-center gap-2">
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4">
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+ <div className="flex gap-2 rounded-xl border border-slate-200 p-4">
+ <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+ Ctrl или Cmd + Enter запускает код. L
+ </div>
+ <div className="flex gap-2 rounded-xl border border-slate-200 p-4">
+ <Info className="w-4 h-4 shrink-0 text-amber-500" />

```

```
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
 },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`.
* **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты/карты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация |

...

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика — параграфы `

`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|--------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; « |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – **Стек и DFS**
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **Очередь
- * `heap-beam-search` → `HeapViz.tsx` – **Куча и лучевой
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – **Дер
- * `dynamic-programming` → `DPVisualizer.tsx` – **Динами

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | | |
|----|--------------------------------------|---|---|---|--|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - | |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - | |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - | |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - | |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - | |
| 22 | 22. Z-функция | 4 | - | - | |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - | |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - | |
| - | Обзор: Кратчайшие пути и Графы | - | - | - | |
| - | Алгоритмы на карте | - | - | - | |
| - | Эйлер: Путь ≠ Цикл | - | - | - | |
| - | Мосты в графах: код + визуализация | 1 | - | - | |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-50">
```

```
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
```

```
      {activeTab === "guide" ? (
```

```
        <>
```

```
          <MobileToc
```

```
            selectedChapterId={activeChapterId}
```

```
            setSelectedChapterId={goTo}
```

```
            currentTitle={activeChapter.title}
```

```
            index={index}
```

```
            total={chapters.length}
```

```
          />
```

```
        <div className="flex flex-col lg:flex-row max-w-7xl">
```

```
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
```

```
          <div className="hidden lg:block lg:w-80 shrink-0">
```

```
            <Sidebar selectedChapterId={activeChapterId}>
```

```
          </div>
```

```
          <main id="main-content" className="flex-1 min-w-0">
```

```
            {/* Шапка главы с быстрыми переходами */}
```

```
            <div className="flex items-center justify-between">
```

```
              <div className="min-w-0">
```

```
                <div className="text-[10px] uppercase">
```

```
                  {activeChapter.category || "Универсальное пособие"}>
```

```
                </div>
```

```
                <h2 className="text-base sm:text-xl font-medium">
```

```
                  {activeChapter.title}>
```

```
                </h2>
```

```
              </div>
```

```
              <ChapterNav prev={prev} next={next} index={index}>
```

```
            </div>
```

```
          <div className="h-1 w-full bg-slate-800 rounded">
```

```
            <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}]_)(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("span"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("span"));
  textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    }
  };
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```
<p class="text-xl font-mono text-white">
  двоичная куча. Пара – точка на декартовой
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
  платит  $\approx n \cdot \log n$  сравнений, counting по ма
  точка вставляется спуском от корня:  $x \leq y$  уз
  единственно. Весь процесс – в симуляторе под
  Merge / Erase</b> по шагам, разборы граблей
  едини точки сам, а судья-инварианты объясни
```

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
  <p class="my-1 font-bold leading-none">
    <span class="text-2xl text-slate-300">И</span>
  </p>
  <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
  rder border-emerald-500 shadow-md">
    Аналогия 1: Split – таможня
  </div>
  <p class="text-slate-300 text-sm">S
  х»». Спуск от корня: узел, который вместе
  скрытия</b> – досматривается только одна ве
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
  border-rose-500 shadow-md">
    Аналогия 2: Merge – стыковка
  </div>
  <p class="text-slate-300 text-sm">M
  аг – одно сравнение: чей у у корня больше,
```

```

    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}

```

```

        </div>
    </details>

```

```

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">

```

Код: вставка, merge, erase (Скрыт)

```

    </summary>

```

```

    <div class="p-5 text-sm text-slate-300"
      <pre class="bg-slate-950 p-4 rounded

```

```

def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

```

```

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)

```

```

    <pre class="bg-slate-950 p-4 rounded

```

```

def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

```

```

def erase(root, key): # удалить произвольн
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)

```

```

        <p class="text-slate-400 text-xs mt-4">
      </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wider">
      <p class="text-emerald-300 font-bold">
      <p class="text-slate-400 text-xs mt-4">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wider">
      <p class="text-white font-bold">O(n
      <p class="text-slate-400 text-xs mt-4">
    </div>
  </div>
</div>

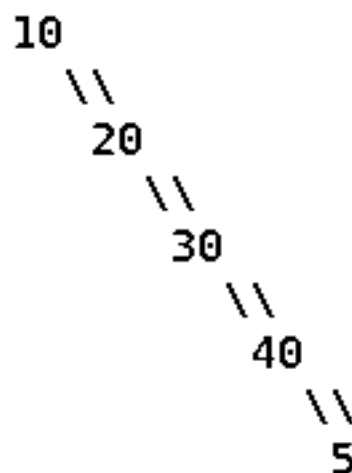
```

<!-- 1. Зачем -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border">
  <h3 class="text-lg font-bold text-white mb-4">
  <p class="text-slate-300 text-sm mb-4">В об
  ючи по возрастанию – и дерево вырождается в
  <pre class="bg-slate-950 p-4 rounded-lg over
  ading-relaxed">вставили 10, 20, 30, 40, 50

```



высота = 5, поиск 50 = 5 сравнений
это уже не дерево, а односвязный

```

    50</pre>
    <p class="text-slate-300 text-sm mt-4">Есть
    <ul class="list-disc list-inside text-slate">
      <li><span class="text-white font-bold">
        каждой вставки чинят дерево, чтобы оно <em>
      </li>
      <li><span class="text-white font-bold">

```

```

-----
      А      В      поворот 40      В      С

обход слева направо ДО:      А 20 В 40 С
обход слева направо ПОСЛЕ:  А 20 В 40 С      ← тот же сам

      <div class="grid grid-cols-1 md:grid-cols-2"
        <div class="bg-slate-900 rounded-lg p-4"
          <p class="text-emerald-400 font-bol
          <p class="text-slate-300 text-xs">П
        сла поворотов – сломать его поворотами нево
      </div>
      <div class="bg-slate-900 rounded-lg p-4"
        <p class="text-amber-400 font-bold
        <p class="text-slate-300 text-xs">П
      одно и то же место, поэтому В просто перев
    </div>
  </div>
</div>

<!-- 4. Три случая -->
<div class="bg-slate-800/60 p-6 rounded-xl border
  <h3 class="text-lg font-bold text-white mb-
  <p class="text-slate-300 text-sm mb-4">Обозн
    узел, который поднимаем, <span class="font
  >g</span> – дед (родитель родителя). Смотри

  <div class="overflow-x-auto -mx-2 px-2">
    <table class="w-full text-xs sm:text-sm"
      <thead>
        <tr class="text-left text-slate
          <th class="py-2 pr-3 font-b
          <th class="py-2 pr-3 font-b
          <th class="py-2 pr-3 font-b
          <th class="py-2 font-bold">
        </tr>
      </thead>
      <tbody class="text-slate-300">
        <tr class="border-b border-slate
          <td class="py-3 pr-3 font-b

```

```

      40      →      20      →      60
      /          \          /
     20          40      40
  
```

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

```

      <p class="text-slate-400 text-xs mt-4">
        мортизации не получится.</p>
  
```

```

      </div>
  
```

```

      <div class="bg-slate-950 rounded-lg p-4">
  
```

```

        <p class="text-emerald-400 font-bold">
  
```

```

        <pre class="overflow-x-auto text-[12px]>
  
```

```

              20
             / \
            40  60
           /   \
          20    A
         /
        40
       /
      20
     /
    40
   /
  20
 
```

было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

```

      <p class="text-slate-400 text-xs mt-4">
        ем.</p>
  
```

```

      </div>
  
```

```

</div>
  
```

```

<p class="text-slate-300 text-sm mt-4"><span>
  
```

```

  з, змейка (zig-zag) – крути СНИЗУ вверх.</p>
  
```

```

</div>
  
```

```

<!-- 6. Полный пример -->
  
```

```

<div class="bg-slate-800/60 p-6 rounded-xl border">
  
```

```

  <h3 class="text-lg font-bold text-white mb-4">
  
```

```

  <p class="text-slate-300 text-sm mb-4">Дерево
  
```

```

    3 шага, а потом делаем Splay(10).</p>
  
```

```

  <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto">
  
```

```

    0 leading-relaxed">старт                после Zi
  
```

```

      глубина 3 → 1
  
```

```

      10 в корне
  
```

```
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-3">Операции
  <p class="text-slate-300 text-sm mb-4">Красная ссылка удаляется в корне.</p>
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3">
      <span class="text-slate-400">— снимаем последний узел на пути (соседний по значению)</span>
      <div class="bg-slate-900 rounded-lg p-3">
        <span class="text-slate-400">— Splay-операция: просто отрезаем ссылки.</span></div>
        <div class="bg-slate-900 rounded-lg p-3">
          <span class="text-slate-400">— splay-операция: k)</span> <span class="text-slate-400">— splay-операция:
        </div>
      <div class="bg-slate-900 rounded-lg p-3">
        <span class="text-slate-400">— Splay-операция: него не будет правого сына — туда и вешаем left child.</span>
      </div>
    </div>
  </div>
```

```
<!-- 9. Код -->
```

```
<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800"
  <summary class="p-4 font-bold text-slate-400">
    open:border-b border-slate-700/50">
      Код: rotate + splay + find (нажми, чтобы увидеть)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-2">
      <pre class="bg-slate-950 p-4 rounded overflow-x-auto">// узел: { key, left, right, parent }
```

```
// один поворот: x встаёт на место своего родителя
```

```
function rotate(x) {
  const p = x.parent, g = p.parent;
```

```
  if (p.left === x) { // правый поворот
```

```

    cur = key < cur.key ? cur.left : cur.right;
  }
  return last ? splay(last) : null;      // splay делает
}/pre>

```

Вся работа против монотонности своей оценки.

10. Плюсы/минусы

Плюсы:

- Простой код: нет высот, цветов,
- Меньше памяти, чем у AVL и красных черных.
- Сам подстраивается под «горячие» элементы.
- split / merge получаются почти бесплатно.

Минусы:

- Нет гарантии на отдельную операцию.
- Не годится для реального времени.
- Дерево меняется даже при чтении.
- Постоянные записи в память при split / merge.

11. Вопросы

Вопросы, которые задают на экзамене:

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
        , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          × Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb">
          з него маленький целевой прямоугольник в цел
          ый верхний угол отрезается дважды! Поэтому
        </div>
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-white">

```



```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-blue-300 italic">
  <p class="text-xl font-mono text-white">
перед.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Учеба в вузе
    </div>
    <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Ограничения: Циклы
    </div>
    <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выяв
  </div>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
и fup[v] > tin[u].</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Единственный мост
        </div>
        <p class="text-slate-300 text-sm mb
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя
    category: "Графы. Продвинутое",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-rose-400"
      <div class="bg-slate-900 p-4 rounded-lg mb-

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Рисование не отрывая
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Если сходится нечетное число линий, значит, из
    а везде должно быть четное число (зашел-вышел)
  </div>
</div>
</div>
</div>
</section>`,
},
{
  id: "pathfinding-accel",
  title: "17. Графы. Ускорения поиска",
  type: "html",
  description: "",
  category: "Графы. Пути",
  content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-white">17. Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Аналогия 1: Рисование не отрывая
        <p class="text-xl font-mono text-white">
          d-длина.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```

```

        с самых дешевых дорог в стране". Он строит
        единую сеть.</p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                    которое торчит из посещенных в непосещенных
            </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
                    rder border-emerald-500 shadow-md">
                    ✦ Аналогия: Заражение вирусом
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    ли вирус). Мы стартуем из одного города, и
                    Сеть растет только из одного связного куска
                </div>

```

```

</section>`,
  },
  {
    id: "string-kmp",
    title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
    type: "html",
    description: "",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                i], который одновременно является её суффиксом.
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <!-- Аналогия 1 -->
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия 1: Бумеранг (Префикс и суффикс)
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  идишь "АБРА". Если ты ошибешься при поиске
                  концовка "АБРА" совпадает с началом "АБРА"
                  моя перепроверка!</p>
                </div>
              </div>
            <details class="bg-slate-900/40 rounded-lg">
              <summary class="p-4 font-bold text-slate-300">
                -b border-slate-700/50">

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
    Аналогия 1: Копипаста
  </div>
  <p class="text-slate-300 text-sm mb-4" style="color: #e0e0e0; font-size: 0.9em; margin-bottom: 10px;">
    в тексте кричит: "Эй! Начиная с этой буквы найдите в строке "окно" [L, R] самой дальней найденной копии этой буквы".
  </p>
</div>
<details class="bg-slate-900/40 rounded-lg" style="background-color: #1a202c40; border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <summary class="p-4 font-bold text-slate-300" style="padding: 5px 10px; color: #e0e0e0; font-weight: bold;">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300" style="padding: 10px; color: #e0e0e0; font-size: 0.9em;">
    <pre class="bg-slate-950 p-4 rounded" style="background-color: #0d1b2a; padding: 10px; border-radius: 10px;">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
</pre>
    </div>
  </details>
</div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12" style="margin-bottom: 20px;">
  <div class="flex items-center mb-6 flex-wrap gap-3" style="margin-bottom: 10px;">

```

ФАЙЛ 18/82: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

```
export interface Chapter {
  id: string;
  title: string;
  type: "html";
  content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">Не бы
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-blue-300 italic
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
          <p><strong class="text-white">П
```

```

        <br><span class="text-xs text-rose-500">
        лноту).</span>
      </p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700"
    <summary class="p-4 font-bold text-slate-400">
    Душный режим: Формальное доказательство
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p class="text-blue-400">Теорема Эйлера</p>
    <p>
      Связный граф содержит эйлеров цикл ⇔
      <br>Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины
      <br>с нечётной степенью.
    </p>
    <p class="text-rose-400">Почему Гамильтон</p>
    <p>
      Задача коммивояжёра (TSP) — это взрыв мозга.
      Доказано, что любой полиномиальный алгоритм
      В общем, не парься. Просто запомни это.
    </p>
  </div>
</details>
</div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
    type: "html",
    content: `<section id="bridges-code" class="mb-20 scroll">
  <div class="flex items-center mb-6">
    <span class="bg-emerald-600 text-white px-4 py-2">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>

```



```
<div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

Аналогия: Кровеносный сосуд

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Граф – это кровеносная система. Ребра (артериалы и вены) – это не мост, живём.

Если же перерезать единственную артерию, то всё плохо.

Алгоритм DFS ищет такие «критические» ребра.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border">
```

```
<summary class="p-4 font-bold text-slate-400">
  Полный код на C++ (Скрыто)
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p>Классическая реализация – <code class="text-slate-400">
  оходов.</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<pre class="text-xs font-mono text-slate-400">
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежности
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
    visited[v] = true;
```

```
    tin[v] = low[v] = timer++; // устанавливаем время
```

```
id: "planarity-euler-formula",
title: "Планарность: K5, K3,3 и формула Эйлера",
type: "html",
content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 r
  <h2 class="text-3xl font-bold text-white">Планар
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400 m

    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-blue-300 italic m
      <p class="text-3xl font-mono text-white"
      <div class="text-sm text-slate-400 spac
        <p><span class="text-blue-400 font-l
        <p><span class="text-emerald-400 fo
        <p><span class="text-rose-400 font-l
      </div>
    </div>
  </div>

  <p class="text-slate-300 text-sm">
    <strong>Следствие:</strong> Для планарн
ode>.
    Если ребер больше – граф гарантированно
  </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
  <div class="bg-slate-800 p-6 rounded-lg bord
    <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
      K5: Полный граф на 5 вершинах
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У K5: <code class="text-white">V=5,
```

```

        </p>
      </div>
    </details>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    content: `<section id="graph-articulation" class="mb-6">
<div class="flex items-center mb-6">
  <span class="bg-rose-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
          <div class="p-3 bg-slate-950 rounded">
            <span class="text-rose-400 font">
              <p class="text-xl font-bold tex">
                <p class="text-xs text-slate-400">
          </div>
        </div>
      </div>
    </div>

    <p class="text-slate-300 text-sm">
      <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
    </p>
  `
  }
}
```

```
</p>
<div class="bg-slate-950 p-4 rounded-lg">
  <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
</pre>
```

```
</div>
</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">
  <h4 class="text-lg font-bold text-indigo-400">
  <p class="text-slate-300 text-sm mb-4">
    Это глубокий дуальный мост между ориентиром
  </p>
  <ul class="list-disc list-inside text-sm text-slate-300">
    <li><strong class="text-rose-400">Точки
    onг> и показывают физическую уязвимость свя
    <li><strong class="text-emerald-400">SC
    ависимые "конгломераты".</li>
    <li><strong>Как точка сочленения рушит
```

```

merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20)
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler",
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
  "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
  "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
  "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

ФАЙЛ 20/82: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМ

```
import type { DemoKind } from "../components/hints/Mini
```

```

    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину"],
    title: "DFS – обход в глубину",
    short:
        "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
        "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
},
{
    id: "binsect",
    labels: ["binsect", "бинсект", "бинарный поиск", "д
    title: "Бинарный поиск (и его самозванец binsect)",
    short:
        "Бинарный поиск НАХОДИТ готовый элемент в уже отс
        о не сортирует. «binsect» из конспекта – шутлив
        quicksort.",
    formal:
        "Поиск одного элемента в отсортированном массиве -
        O(n log n). Бинарный поиск сортировку не выполн
    complexity: "поиск: O(log n); сортировкой не являет
},
{
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "
    title: "Куча (priority queue)",
    short:
        "Не отсортированный список, а «мешок, из которого
    formal:
        "Полное бинарное дерево в массиве: дети узла i ле
    complexity: "вставка/извлечение O(log n), «посмотре
    demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",

```

```

    },
    {
      id: "bfs-on-trie",
      labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
      ],
      title: "Обход бора в ширину (тот самый «какой-то обход»)",
      short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную очередь  

        о есть строго по уровням.",
      formal:
        "Кладём в очередь детей корня (их ссылка = корень).  

        Кладём с в очередь.",
      complexity: "O(суммарной длины словаря × алфавит)",
      demo: "trie-bfs",
      simulator: "aho-corasick",
    },
    {
      id: "suffix-link",
      labels: ["суффиксная ссылка", "суффиксные ссылки"],
      title: "Суффиксная ссылка",
      short:
        "«Куда откатиться, если следующая буква не подошла»",
      formal: "link(v) = go(link(parent(v)), ch). Считает",
      demo: "suffix-link",
      simulator: "aho-corasick",
    },
    {
      id: "toposort",

```



```

    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (суть в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полнота",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов",
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — потенциал,
        к путям сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты",
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждой",
    formal: "Косарайю: DFS с сохранением порядка выхода",
    complexity: " $O(V + E)$ ",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функция",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксы",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:

```

```

    id: "shortest-path",
    labels: ["кратчайший путь", "кратчайшие пути", "кра
    title: "Кратчайший путь",
    short:
        "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
    formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Фор
        ы,  $O(V^3)$ .",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "negative-cycle",
    labels: ["отрицательный цикл", "отрицательного цикл
    title: "Отрицательный цикл",
    short:
        "Круг, пройдя по которому вы становитесь «богаче»
    formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "greedy",
    labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
    formal:
        "Корректность обычно доказывается «леммой о разре
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",

```

```

    short:
        "Матрица смежности — таблица «есть ли ребро», быс
        й, экономно для разреженных графов.",
    formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Спис
},
{
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: " $O(E \log V)$  с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
    title: "Алгоритм Форда-Беллмана",
    short:
        "Тупой, но честный:  $V-1$  раз проходим по ВСЕМ рёбра
        ",
    formal:
        "После  $k$ -й итерации гарантированно верны все крат
        ось — есть отрицательный цикл.",
    complexity: " $O(V \cdot E)$ ",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
        "промежуточная вершина", "промежуточную вершину",
    ],
}
```

```

    labels: ["Краскал", "Краскала", "Краскалу", "Kruska
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка  $O(E \log E)$  +  $E$  операций union/find. С
    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        " $G = (V, E)$ . Ребро бывает ориентированным (улица
    demo: "graph",

```

```

},
{
  id: "inclusion-exclusion",
  labels: [
    "включений-исключений", "включения-исключения", "вырезание прямоугольников",
  ],
  title: "Включения-исключения на префиксных суммах",
  short:
    "Чтобы получить сумму прямоугольника, берём больший отрезали дважды.",
  formal:
    "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
  complexity: "запрос O(1) после O(nm) предподсчёта",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
},
{
  id: "reduction",
  labels: ["сведение", "сведения", "сведении", "сведё"],
  title: "Сведение задач",
  short:
    "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
  formal:
    "A ≤p B: есть полиномиальная функция, переводящая решение A в решение B.",
  demo: "np",
},
{
  id: "range",
  labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
  title: "Запрос на отрезке",
  short:
    "Спрашиваем не про один элемент, а про кусок массива. Нужны предподсчёты.",
  formal:
    "Суммы – префиксные суммы O(1). Минимум без изменений.",
  demo: "prefix-sum",
}

```

```

    },
    {
      id: "zig-zig",
      labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
      title: "Zig-Zig – x и родитель с одной стороны",
      short:
        "Дерево вытянулось в «бамбук»: g → p → x идут в од...
        м (p, x). Именно это вдвое сокращает глубину вс...
      formal:
        "Если крутить наивно (два раза поднимать x), бамб...
      demo: "zig-zig",
      simulator: "splay-rotations",
    },
    {
      id: "zig-zag",
      labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
      title: "Zig-Zag – x и родитель с разных сторон",
      short:
        "Узлы идут «змейкой»: p – левый сын g, а x – прав...
        p и g становятся двумя детьми x.",
      formal: "Эквивалентно двойному повороту AVL-дерева",
      demo: "zig-zag",
      simulator: "splay-rotations",
    },
  ],
};

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч...
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);

```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2"
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
        <p class="text-slate-300 text-sm mb-4">
        (нельзя поехать и заработать). Он всегда в пути.
        <div id="slot-chapter-image-dijkstra"
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Ограничения
        </div>
        <p class="text-slate-300 text-sm mb-4">
        охождение улицы), Дейкстра сломается, так как
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border border-slate-700/50"
      <summary class="p-4 font-bold text-slate-300">
        Строгое доказательство (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
```

```

        берет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman"
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg"
            <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
                Отрицательный цикл
            </div>
            <p class="text-slate-300 text-sm mb-2">
г станет ЕЩЕ короче – значит мы попали во в
            </div>
        </div>

        <div id="slot-bellman-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "floyd",
        title: "Билет 16. Флойд-Уоршелл",
        type: "html",
        category: "Графы",
        content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
        <h3 class="text-xl font-bold text-indigo-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">

```



```
<h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l"
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text"
      <p class="text-sm text-blue-300 italic mb-2">0c
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
  </div>
```

```
<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo"
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы движемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2"
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
    Аналогия 2: Матёшка (Терминальные)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    нашли и "he", потому что оно спрятано внутри
    терминальные ссылки</b> (term_link) – прямые мос
    p>
  </div>
```

```

        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].link;
        } else {
            nodes[child].term_link = nodes[nodes[child].link];
        }

        q.push(child);
    }
}

</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">
        <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут содержать вершины с большими дробными числами (долгота и широта), мы можем использовать массивы чисел, хранящих значения от 0 до N, и уже по ним строим графы.</p>
    </div>
    </div>
</div>
</section>`
}
];

```

```

    {
      question: "Куда ведёт эйлеров путь, если нечётных
options: [
  "Всё ещё можно пройти по всем рёбрам ровно раз",
  "Такого графа не существует",
  "Эйлерова пути нет – нужно удалять лишние рёбра",
],
correctIndex: 2,
explanation: "Если нечётных вершин больше 2, эйлеров
  ровно 2 нечётные или 0."
}
],
"bridges-code": [
  {
    question: "После DFS у нас low[u] = 4, а tin[v] =
options: [
  "Да, так как low[u] (4) > tin[v] (2)",
  "Нет, low[u] должно быть меньше или равно",
  "Только если граф связный и неориентированный"
],
correctIndex: 0,
explanation: "Условие моста: low[u] > tin[v]. В н
  },
  {
    question: "Для чего нужен массив low[v]?",
options: [
  "Для того чтобы было больше переменных в коде",
  "Чтобы знать минимальный tin предка, до которого
  "Чтобы хранить глубину рекурсии"
],
correctIndex: 1,
explanation: "low[v] хранит минимальное время входа
  поиска мостов и точек сочленения."
  },
  {
    question: "Какова временная сложность алгоритма по
options: [
  "O(V^2) – ищем мосты в квадрате",

```

$$\left. \begin{array}{l} \{; \\] \end{array} \right\}$$

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 23/82: src/data/tickets/ticket1 5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ:

```
import { Chapter } from '../..types';
```

```
export const tickets1to5: Chapter[] = [
  {
```

```
title: "1. Дерево отрезков с операциями на отрезках"
type: "html",
description: "Дерево отрезков – структура данных дл"
category: "Оптимизации и Продвинутое структуры",
content: `
```

```
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
```

```
<div class="space-y-8">
  <div id="intro" class="bg-slate-700/50 p-6 rounded-2xl">
    <h3 class="text-xl font-bold text-blue-400">
      Welcome to the future of web development
    </h3>
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
        This is a preview of the new features.
      </p>
      <p class="text-xl font-mono text-white">
        Promise (lazy).</p>
    </div>
  </div>
</div>
```

```

        lazy[node] = 0;
    }
}
}
</div>
</details>
</div>
</div>
</section>
`
},
{
  id: "sparse-table",
  title: "2. Двумерная разреженная матрица (Sparse Table)",
  type: "html",
  description: "Разреженная таблица для идемпотентных",
  category: "Продвинутые структуры",
  content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные за

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
    <h3 class="text-xl font-bold text-blue-400">Аналогия 1: Удар катаной (Split)
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">Представьте, что вы — катана. Вы видите врага, который стоит перед вами.
      <p class="text-xl font-mono text-white">Вы решаете, как его разрезать. Вы используете метод split.
    метода: Split и Merge.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
          Аналогия 1: Удар катаной (Split)
        </div>
        <p class="text-slate-300 text-sm mb-4">Вы решаете, как его разрезать. Вы используете метод split.
        по значению X. Все, кто меньше X улетают вправо.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
            Аналогия 2: Слияние капель (Merge)
          </div>
          <p class="text-slate-300 text-sm mb-4">Вы решаете, как его разрезать. Вы используете метод merge.
          вого). Мы просто смотрим на корни: у кого Y больше, тот становится родителем.
          как потомок.</p>
        </div>
      </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-slate-700/50">
      <summary class="p-4 font-bold text-slate-300">Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
      <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val <= x) {

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
        </div>
        <p class="text-slate-300 text-sm mb-2">
            вится корнем дерева (VIP-статус). Если ты член,
            ка почти не требуется времени!</p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
                Аналогия 2: Балансировка через бамбук
            </div>
            <p class="text-slate-300 text-sm mb-2">
                и становиться кривым как бамбук. Но как толк
            </p>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300">
                -b border-slate-700/50">
                    Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}</pre>
                </div>
            </details>

```

```

        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
              ≈ Аналогия 1: Вырезание прямоугольника
            </div>
            <p class="text-slate-300 text-sm mb-4">
              з него маленький целевой прямоугольник в центре.
              ый верхний угол отрезается дважды! Поэтому
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>`
    }
  ];

```



```

        дный и не умеет пересматривать уже "зафиксир
        </div>
    </div>
    <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
  id: "bellman-ford",
  title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
      <h3 class="text-xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">Аналогия 1: Параноик
        <p class="text-xl font-mono text-white">
          Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ♂ Аналогия 1: Параноик
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">

```

```

        шу себе проходить через вершину k?»</i>
      </p>
      <ul class="text-sm text-slate-300 space-y-4">
        <li><b>Шаг k=0:</b> Ты знаешь только вершину k.</li>
        <li><b>Шаг k=1:</b> Разрешаем пересадки в k.
        e> через путь <code class="text-indigo-300">graph[k]</code>
        <li><b>Шаг k=2:</b> Разрешаем пересадки в k, но не в k.
        нутри этих путей уже могут быть пересадки ч
        <li>...</li>
        <li><b>Шаг k=N:</b> Ты проверил все вершины.
      </li>
    </ul>
  </div>
</div>
</section>`
  },
  {
    id: "johnson-algo",
    title: "17. Графы. Алгоритм Джонсона (Фокус с переворотом)",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded-md">17</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Пути</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-amber-400">Алгоритм Джонсона</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-xl font-mono text-white">graph[k] = graph[k] + graph[k]
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">

```

```

        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        (проверяем через DSU) - берем в ответ.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
                Аналогия: Прокладка интернет
            </div>
            <p class="text-slate-300 text-sm mb-6">
                с самых дешевых дорог в стране". Он строит
                единую сеть.</p>
        </div>
    </div>
</div>
</section>`
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-6">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                которое торчит из посещенных в непосещенных
            </div>

```

Аналогия: Племена

```
        </div>
        <p class="text-slate-300 text-sm mb-1">
изкому племени для объединения. К вечеру пле-
ства шлют гонцов к ближайшему чужому царству.
        </div>
    </div>
</div>
</section>`
    }
};
```

ФАЙЛ 25/82: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.2 КБ

```
import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">Строки</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">21. Префикс-функция (π) — Взгляд назад (КМП)</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Префикс-функция (π)</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Определение: Для строки s длины n префикс-функция p — это массив из n элементов, где p[i] — это длина самого длинного собственного префикса строки s[0..i], который совпадает с суффиксом s[0..i].</p>
      </div>
    </div>
  </div>
</section>`
  }
];
```

```

</summary>
<div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px;">
  <p>Тот же пример: <b>abcabcd</b></p>
  <ul class="list-disc pl-5 space-y-2">
    <li><b>a</b>: Префиксов нет. <code>0</code></li>
    <li><b>ab</b>: Из начала ("a") не получилось. <code>0</code></li>
    <li><b>abc</b>: Опять мимо. <code>0</code></li>
    <li><b>abca</b>: 0! Начало ("a") не совпало с "abca" vs "abcd". <code>0</code></li>
    <li><b>abcab</b>: Начало ("ab") не совпало с "abcab" vs "abcd". <code>0</code></li>
    <li><b>abcabc</b>: Начало ("abc") не совпало с "abcabc" vs "abcd". <code>0</code></li>
    <li><b>abcabcd</b>: "d" всё испробовано. <code>1</code></li>
  </ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-300">
    Код C++ (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
  int j = pi[i-1];
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
  if (s[i] == s[j]) j++;
  pi[i] = j;
}</pre>
  </div>
</details>
</div>
</div>
</section>`

```

```
<div class="absolute -top-3 left-4 border-rose-500 shadow-md">
```

Аналогия 2: LLM Самокопиаст

```
</div>
```

```
<p class="text-slate-300 text-sm mb">
```

В LLM Z-функция может применять `[i]` (где `<i>i</i>` указывает назад на сама себя.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
<summary class="p-4 font-bold text-slate-300 border-rose-500 border-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<p>Тот же пример: abcabcd</p>
```

```
<ul class="list-disc pl-5 space-y-2">
```

```
<li>Z[0] – не счита
```

```
<li>Z[1] (bcabc
```

```
[1] = 0</li>
```

```
<li>Z[2] (cabcd
```

```
</li>
```

```
<li>Z[3] (abcd
```

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

```
<li>Z[4], Z[5], Z[6]</li>
```

```
</ul>
```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
<summary class="p-4 font-bold text-slate-300 border-rose-500 border-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```

<div class="bg-slate-800 p-6 rounded-lg border border-rose-500" style="border: 2px solid #f472b4; border-radius: 12px; padding: 10px;">
  <div class="absolute -top-3 left-4 bg-slate-700 emerald-500 shadow-md">
    Аналогия 1: Паутина (Бор)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    ет — мы падаем по <b>суффиксной ссылке</b> ("
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-500" style="border: 2px solid #f472b4; border-radius: 12px; padding: 10px;">
    <div class="absolute -top-3 left-4 bg-rose-900 -500 shadow-md">
      Аналогия 2: Матёшка (Терминальные)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      нашли и "he", потому что оно спрятано внутри
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border border-rose-500" style="border: 2px solid #f472b4; border-radius: 12px; padding: 10px;">
    <summary class="p-4 font-bold text-slate-400 outline-rose-500/50">
      Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
      <pre class="bg-slate-950 p-4 rounded overflow-x-auto">
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
  }
  return t[v].go[c];
}

```

```

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
            Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb-2">
            Это отличный переводчик на английский (Сведения о переводе:
            аче В, то В – это <b>NP-Полная</b> (сосредоточена на переводе)
        </div>
    </div>
</div>
</section>`
    }
};

```

ФАЙЛ 26/82: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
    {
        id: "graph-dfs-bfs",
        title: "6. Графы. Представление, DFS, BFS",
        type: "html",
        description: "Представление графов и базовые обходы",
        category: "Графы. База",
        content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">6. Графы. Представление, DFS, BFS
    </div>

```



```

    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска графов
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
        <p class="text-xl font-mono text-white">
          4 - это минимальное количество цветов, которым
          ый граф можно раскрасить 4 цветами.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            order border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description: "Нахождение кусков графа",
  }

```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
```

перед.</p>

```
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4"
  <div class="border border-emerald-500 shadow-md">
```

Аналогия 1: Учеба в вузе

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  еская сортировка – это расписание, которое
  >
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4"
  <div class="border border-rose-500 shadow-md">
```

Ограничения: Циклы

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  получить опыт, нужна работа". Алгоритм выяв
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
  <div class="border border-slate-700/50">
```

Душный мод: Код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">Компонент
и <i>ориентированного</i> графа, в котором </p>
        </div>
      </div>
    </div>
```

```
<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Нет!</strong>
</p>
<ul class="list-disc list-inside text-sm text-slate-300">
  <li>Если у тебя есть цикл <code>А → Б → А</code>
  <li>Если ты добавишь вершину <strong>Г</strong>
  <li>Правило: <b>Если два цикла имеют хотя бы
  </ul>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
      <div class="text-white">
        <h3>Аналогия: Интриги в офисе
      </div>
      <p class="text-slate-300 text-sm mb-4">
        <strong>Сильная связность – это "клуб сильных"
        </strong>
        <p>они в одной SCC. Добавить курьера Гошу, который
        </p>
        <p>йти наружу (к директору), но обратно в клуб
        </p>
      </div>
    </div>
  </div>
```

```

    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых расхреначивает",
    category: "Графы. Продвинутые",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя

```

```

        <p class="text-slate-300 text-sm mb-4">
            Представь город, где два района сое
            её перекроют (удалят вершину) – районы буд
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-
        <div class="absolute -top-3 left-4 bg-emera
        er border-emerald-500 shadow-md">
            Телеком: Центральный свитч
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У тебя несколько компьютеров в одно
            абелем (мостом). Сами свитчи – это точки со
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-
    <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
        Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-
        <p class="text-blue-400 font-bold">Особ
        <p>
            Для стартовой вершины обхода правил
            то выше неё прыгать некуда). Поэтому для не
            у него <strong>больше 1 независимого ребенка
        </p>
        <div class="bg-slate-950 p-4 rounded-lg
            <pre class="text-xs font-mono text-
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

```

```

    },
    {
      id: "graph-euler",
      title: "13. Графы. Эйлеров цикл",
      type: "html",
      description: "Пройти по всем ребрам ровно 1 раз.",
      category: "Графы",
      content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Задача
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Дана графа с  $n$  вершинами и  $m$  ребрами.
        <p class="text-xl font-mono text-white">Найти путь, который проходит по каждому ребру ровно один раз.
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">Если графа является эйлеровой, то она имеет четное число вершин, и если графа не эйлерова, то она имеет нечетное число вершин. Если графа имеет нечетное число вершин, то она не эйлерова. Если графа имеет четное число вершин, то она может быть эйлеровой. Если графа имеет четное число вершин, то она может быть эйлеровой. Если графа имеет четное число вершин, то она может быть эйлеровой.
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
  ];

```

```

    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
  ];

```

```

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);
edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

```

```

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

```

```

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

```

```

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

```



```

        });

        if (currentLow[to] >= currentTin[v] && p !==
            ap.add(v);
            recordStep(
                `Найдена точка сочленения: вершина ${v}
                v,
            );
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это
        v,
    );
} else if (p === -1) {
    recordStep(
        `Корень дерева DFS ${v} имеет <=1 ребенка, он
        v,
    );
}
};

for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
        dfs(i);
    }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
    desc: "",

```

```

        >
        {isPlaying ? {
            <Pause className="w-4 h-4 ml-1" />
        } : {
            <Play className="w-4 h-4 ml-1" />
        }}
        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 g
    <div className="lg:col-span-2 relative h-96 lg:l
        <svg className="w-full h-full" viewBox="0 0 8
            {/* Edges */}
            {edges.map((edge, i) => {
                const u = nodes.find((n) => n.id === edge
                const v = nodes.find((n) => n.id === edge

            const isBridge =
                (edge.u === 3 && edge.v === 4) ||
                (edge.u === 4 && edge.v === 3);

            return (
                <line
                    key={"edge-" + i}
                    x1={u.x}
                    y1={u.y}
                    x2={v.x}
                    y2={v.y}

```

```

        className="transition-all duration-300"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y - r - 15}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"
        >
          tin:{" "}
          {currentStep.tin[node.id] >= 0
            ? currentStep.tin[node.id]
            : "-"}
        </text>
        <text
          x={node.x}
          y={node.y + r + 20}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"
        >
          low:{" "}
          {currentStep.low[node.id] >= 0

```

```

        <div className="w-8 h-1 bg-rose-500 border"
        Мост
        </div>
    </div>

    <div>
        <h5 className="text-xs text-slate-500 font"
        ЛОГ ДЕЙСТВИЙ:
        </h5>
        <div className="space-y-2">
            {steps
                .slice(0, currentStepIndex + 1)
                .reverse()
                .map((step, idx) => {
                    <div
                        key={idx}
                        className={`text-xs p-2 rounded $
xt-slate-500`} `}
                    >
                        {step.desc}
                    </div>
                })}
        </div>
    </div>
    <div className="mt-4 shrink-0">
        <div className="w-full bg-slate-800 h-2 rounded"
        <div
            className="bg-rose-500 h-full transition"
            style={{
                width: `${(currentStepIndex / Math.ma
            }}
        />
        </div>
    </div>
</div>
</div>
</div>

```

```

const getEdges = (cycle: boolean): Edge[] => {
  const defaultEdges = [
    { u: 'S', v: 'A', w: 4 },
    { u: 'S', v: 'B', w: 5 },
    { u: 'B', v: 'C', w: -2 },
    { u: 'A', v: 'C', w: 1 },
    { u: 'C', v: 'D', w: 3 },
    { u: 'C', v: 'E', w: 4 },
    { u: 'D', v: 'F', w: 2 },
    { u: 'E', v: 'F', w: 1 },
  ];
  if (cycle) {
    return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
  } else {
    return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
  }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
  const list: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (list[n.id] = []));
  ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
  return list;
};

const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start)' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    for i from 1 to V - 1: // |V| - 1 iterations' },
  { line: 4, text: '        for (u, v, weight) in E:' },
  { line: 5, text: '            if dist[u] + weight < dist[v]' },
  { line: 6, text: '                dist[v] = dist[u] + weight' },
  { line: 7, text: '    for (u, v, weight) in E: // Π' },
  { line: 8, text: '        if dist[u] + weight < dist[v]' }
];

```

```

        });
    }

    if (!anyUpdate && iter < vCount - 1) {
        simulationSteps.push({
            iteration: iter, checkingEdge: null, distance:
            log: ` Ранний выход после итерации ${iter}.
            activeCodeLine: 10
        });
        break;
    }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v, w: edge.w },
                distances: { ...dist }, previous: { ...prev },
                log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v} с весом ${edge.w}.
                activeCodeLine: 9,
            });
            break;
        }
    }

    if (!cycleFound) {
        simulationSteps.push({
            iteration: vCount, checkingEdge: null,
            distances: { ...dist }, previous: { ...prev },
            log: ` Проверка завершена. Ни одно ребро не вызвало цикла.
            activeCodeLine: 10,
        });
    }
}
}

```

```

<button onClick={() => { setIsPlaying(false);
  g-slate-600 text-white px-3 py-2 rounded-lg
<button onClick={() => setIsPlaying(!isPlaying
  sition-colors text-white ${isPlaying ? 'bg-
<button onClick={() => { setIsPlaying(false);
  ; }} className="flex items-center gap-1 bg-l
  s">⏮ar </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-blue-950/20
    y-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-blue
      ld shadow-sm">
      Итерация {currentStep.iteration} (⏮ar {curr
    </div>

    {currentStep.hasNegativeCycle && (
      <div className="absolute top-3 right-3 bg-r
        animate-pulse shadow-lg shadow-rose-600/50
        ▲ Отрицательный цикл!
      </div>
    )}

  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-bf-normal" markerWidth=
      <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
      <marker id="arrow-bf-active" markerWidth=
      <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      <marker id="arrow-bf-cycle" markerWidth=
      path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
    </defs>

    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id ===

```

```

        split(' ')[0]}</text>
      </g>
    );
  }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2 text-sm font-mono text-white">
    {Object.keys(adjList).map(u => {
      <div key={u} className="flex flex-wrap gap-2">
        <span className="font-bold text-white">
          {adjList[u].map((edge, idx) => {
            const isChk = currentStep.checkingEdge === u;
            return (
              <span key={idx} className={`${px-2} ${
                edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
              } ${edge.v} (${edge.w})`>
            </span>
          )
        )
      )
    })}
  </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-4 rounded-2xl">

```

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
  { id: 0, label: '0', x: 300, y: 50 },
  { id: 1, label: '1', x: 150, y: 150 },
  { id: 2, label: '2', x: 450, y: 150 },
  { id: 3, label: '3', x: 75, y: 250 },
  { id: 4, label: '4', x: 225, y: 250 },
  { id: 5, label: '5', x: 375, y: 250 },
  { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}
```

```

        currentNode: curr,
        logs: `Достаем из очереди (${curr}).`
    });

    const neighbors = ADJ[curr];
    if (neighbors.length > 0) {
        for (const n of neighbors) {
            if (!visited.has(n)) {
                steps.push({
                    activeLine: 7,
                    queue: [...queue],
                    visited: Array.from(visited),
                    currentNode: curr,
                    logs: `Рассматриваем соседа: ${n}. Он еще не
                });

                visited.add(n);
                queue.push(n);

                steps.push({
                    activeLine: 9,
                    queue: [...queue],
                    visited: Array.from(visited),
                    currentNode: curr,
                    logs: `Помечаем ${n} как посещенную и добавляем
                });
            }
        }
    } else {
        steps.push({
            activeLine: 6,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: curr,
            logs: `Соседей нет или все уже посещены.`
        });
    }
}

```

```

return (
  <div className="bg-slate-900 border-2 border-slate-
    <div className="flex flex-col md:flex-row items-c
      <div>
        <h3 className="text-2xl font-bold text-white
          <span className="text-blue-400"> </span> Ал
        </h3>
        <p className="text-slate-400 text-sm mt-1">Ви
      </div>
      <div className="flex bg-slate-800 p-1.5 rounded
        <button
          onClick={() => { setIsPlaying(false); setSt
          className="p-2 text-slate-400 hover:text-wh
          title="Сброс"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
        <button
          onClick={() => setIsPlaying(p => !p)}
          className="p-2 text-blue-400 hover:text-bl
          title={isPlaying ? 'Пауза' : 'Воспроизведен
        >
          {isPlaying ? <Pause className="w-5 h-5" />
        </button>
        <button
          onClick={handleNext}
          disabled={stepIdx >= STEPS.length - 1}
          className="p-2 text-emerald-400 hover:text-
          ed:hover:bg-transparent"
          title="Шаг вперед"
        >
          <StepForward className="w-5 h-5" />
        </button>
      </div>
    </div>
    <div className="grid grid-cols-1 lg:grid-cols-2 g

```

```

    } else if (isVisited) {
      fill = '#0f172a'; // slate-900
      stroke = '#3b82f6'; // blue-500
    }

    return (
      <g key={n.id} className="transition-tran
        {isCurrent && {
          <circle cx={n.x} cy={n.y} r="30" fi
        }}
        <circle
          cx={n.x} cy={n.y}
          r="20"
          fill={fill}
          stroke={stroke}
          strokeWidth="3"
          className="transition-colors durati
        />
        <text
          x={n.x} y={n.y}
          textAnchor="middle"
          dy=".3em"
          className="fill-white font-bold tex
        >
          {n.label}
        </text>
      </g>
    );
  }
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-sl
    er">
    <div className="bg-slate-800 text-slate-400
      se tracking-wider">

```

```

        enter justify-center rounded-md shadow-lg s
            {qId}
        </span>
    ))
  })
</div>
<div className="mt-2 text-sm text-amber-300
ms-center justify-center">
  "{step.logs}"
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/82: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const imageMap: Record<string, { src: string; alt: string }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',

```

```

    prev?: Chapter;
    next?: Chapter;
    index: number;
    total: number;
    onGo: (id: string) => void;
    compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
  if (compact) {
    return (
      <div className="flex items-center gap-2 text-xs">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-2.5 py-1
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
        >
          <ChevronLeft className="w-4 h-4" />
          <span className="hidden sm:inline">Назад</span>
        </button>
        <span className="text-slate-500 font-mono tabular-nums">
          {index + 1} / {total}
        </span>
        <button
          disabled={!next}
          onClick={() => next && onGo(next.id)}
          title={next ? next.title : "Это последняя тема"}
          className="flex items-center gap-1 px-2.5 py-1
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
        >
          <span className="hidden sm:inline">Вперёд</span>
          <ChevronRight className="w-4 h-4" />
        </button>
      </div>
    );
  }
  return null;
};

```

```
};
```

ФАЙЛ 32/82: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointer } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";
```

```
interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
  useTermHints(contentRef, [chapter.id]);
```

```
  const cancelHide = useCallback(() => {
```

```

    if (isTouch()) return;
    const el = (e.target as HTMLElement).closest<HTMLElement>('a');
    if (el) open(el);
  });

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>('a');
  if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>('a');
  if (!el) return;
  e.preventDefault();
  if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>('a');
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border border-gray-300 hover:text-white hover:border-emerald-500"
        />
      </div>
    </article>
  </>
)

```



```

    {viz && {
      <section id="chapter-viz" className="mt-8 screen">
        <div className="flex items-center justify-between">
          <h3 className="flex items-center gap-2 text-indigo-600">
            <Sparkles className="w-4 h-4 text-indigo-600" />
            Демонстрация: {viz.title}
          </h3>
          <button
            onClick={() => onOpenSimulator(chapter.viz)}
            className="flex items-center gap-1.5 text-white border-indigo-600
            px-3 py-2 rounded-md shadow-sm hover:opacity-90"
          >
            <Maximize2 className="w-3.5 h-3.5" />
            {viz.title}
          </button>
        </div>
        {viz.hint && <p className="text-xs text-slate-500">
          {viz.hint}
        </p>
        <viz.Component />
      </section>
    }}

    <ChapterNav prev={prev} next={next} index={index} />
  </article>

  <TermPopover
    anchor={anchor}
    onClose={() => setAnchor(null)}
    onOpenSimulator={(id) => {
      setAnchor(null);
      onOpenSimulator(id);
    }}
    onCardEnter={cancelHide}
    onCardLeave={scheduleHide}
  />
</>
);
};

```

```

    visitedIndices: number[];
    tin: Record<number, number>;
    low: Record<number, number>;
    stack: number[];
    bridges: string[];
    log: string;
    activeEdge: [number, number] | null;
    backEdge: [number, number] | null;
    activeCodeLine: number[];
}

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
  }
];

```

```

currentNode: 3, visitedIndices: [0,1,6,2,3],
tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
activeEdge: [2,3], backEdge: null,
log: "Идём C→D. tin[D]=5, low[D]=5.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4],
tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
activeEdge: [3,4], backEdge: null,
log: "D→E. tin[E]=6, low[E]=6.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: [4,5], backEdge: null,
log: "E→F. tin[F]=7, low[F]=7.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: [5,3],
log: "Из F видим D (посещён). Обратное ребро (F,D).
activeCodeLine: [7,8,9]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,

```

```

useEffect(() => {
  if (dfsAuto) {
    dfsTimer.current = setInterval(() => {
      setDfsIdx(prev => {
        if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
        return prev + 1;
      });
    }, 2500);
  }
  return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">Шаг {step}</h4>
      <div className="flex items-center gap-1 bg-slate-800 p-1">
        <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
          disabled={dfsIdx === 0}
          className="p-1.5 hover:bg-slate-700 rounded text-white">
          <Undo className="h-3.5 w-3.5" />
        </button>
        <button onClick={() => setDfsAuto(!dfsAuto)}
          className={`px-2.5 py-1 rounded text-[10px] ${
            dfsAuto ? "bg-rose-600 text-white" : "bg-slate-800 text-white"
          }`}
          >{dfsAuto ? <><Pause className="h-3 w-3" /> : <><Play className="h-3 w-3" /></>
        </button>
        <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1); }}
          disabled={dfsIdx === DFS_STEPS.length - 1}
          className="p-1.5 hover:bg-slate-700 rounded text-white">
          <SkipForward className="h-3.5 w-3.5" />
        </button>
        <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
          className="p-1.5 hover:bg-slate-700 rounded text-white">
          <RefreshCw className="h-3.5 w-3.5" />
        </button>
      </div>
    </div>
  </div>
);

```



```

    <div className="bg-indigo-950/10 border borde
      <p className="text-[11px] text-slate-300 le
        <div className="mt-2 pt-2 border-t border-s
          <span className="text-slate-400">Мосты:</
          <span className="font-mono font-bold text
            {step.bridges.length > 0 ? step.bridges
          </span>
        </div>
      </div>
    </div>
  </div>
</div>
);
}

```

ФАЙЛ 34/82: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },

```

```

    { line: 10, text: '                pq.push((dist[v]
    { line: 11, text: '    return dist // Все кратчайши
};

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999
  const visited: Record<string, boolean> = { A: false
  const prev: Record<string, string | null> = { A: nu

```

```

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: '    Дейкстра готов к доставке. Начинаем из Пи
  activeCodeLine: 2,
});

```

```

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }
}

```

```

if (!u) break;

```

```

visited[u] = true;
simulationSteps.push({
  currentNode: u, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited }
  log: `    Выбираем вершину ${u} с мин. непосещённ

```

```

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border-1" style={{borderColor: 'white'}}>
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justify-between">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white rounded">
          <span className="text-2xl"> </span>
        </div>
      </div>
      <div>
        <h3 className="text-2xl font-bold text-white">
          <p className="text-sm text-indigo-300">Полный код
        </p>
      </div>
    </div>

    <div className="flex items-center gap-2">
      <button
        onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
        className="flex items-center gap-1 bg-slate-700 text-white px-3 py-2 rounded"
      >

```



```

        <path d="M0,0 L6,3 L0,6 z" fill="#10b98"
      </marker>
    </defs>
    {/* Пёбра */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === u)
      const vNode = nodes.find((n) => n.id === v)
      const isChecking =
        currentStep.checkingEdge &&
        (currentStep.checkingEdge.u === edge.u &&
         currentStep.checkingEdge.v === edge.v)

      const isOptimal = currentStep.previous[edge.id] === 'optimal'

      let marker = 'url(#arrow-dh-normal)';
      if (isChecking) marker = 'url(#arrow-dh-check)';
      else if (isOptimal) marker = 'url(#arrow-dh-optimal)';

      return (
        <g key={idx}>
          <line
            x1={uNode.x} y1={uNode.y}
            x2={vNode.x} y2={vNode.y}
            stroke={isChecking ? '#f59e0b' : isOptimal ? '#1e293b' : '#10b98'}
            strokeWidth={isChecking ? 5 : isOptimal ? 1 : 1}
            markerEnd={marker}
            className="transition-all duration-300"
            strokeDasharray={isChecking ? '4,4' : isOptimal ? '0' : '0'}
          />
          <circle
            cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
            r="12" fill="#1e293b"
            stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b98' : '#10b98'}
            strokeWidth={isChecking ? 2 : isOptimal ? 1 : 1}
          />
          <text
            x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2) + 10}
            fill={isChecking ? '#f59e0b' : isOptimal ? '#10b98' : '#10b98'}
            fontSize="11" fontWeight="bold" text={edge.label}
          />
        </g>
      )
    })}
```

```

        <p className="min-h-[40px] flex items-center justify-center">
      </div>
    </div>

```

```

    { /* Список смежности */ }
    <div className="w-full lg:w-1/3 bg-emerald-950/50 p-3 rounded-lg">
      <h4 className="text-lg font-bold text-emerald-500">Смежные узлы</h4>
      <div className="space-y-2.5 flex-1 overflow-y-auto">
        {Object.keys(adjList).map((u) => {
          const isCurrentNode = currentStep.currentNode === u
          return (
            <div key={u} className={`p-3 rounded-lg ${
              isCurrentNode ? 'bg-emerald-900/60 text-white' : 'bg-slate-300'
            }`} >
              <div className="flex items-center gap-2">
                <span className={`px-2 py-0.5 rounded ${
                  isCurrentNode ? 'bg-emerald-900/60 text-white' : 'bg-slate-300'
                }`} >
                  {u}
                </span>
                <span className="text-slate-400">→</span>
              </div>
              <div className="flex flex-wrap gap-2">
                {adjList[u].map((edge, idx) => {
                  const isCheckingThis = currentStep.currentNode === edge.v
                  return (
                    <span key={idx} className={`px-2 py-0.5 rounded ${
                      isCheckingThis ? 'bg-amber-500 text-white' : 'bg-slate-300'
                    }`} >
                      {edge.v}
                    </span>
                    <span className="text-slate-400">→</span>
                    <span className={isCheckingThis ? 'bg-amber-500 text-white' : 'bg-slate-300'}>
                      {edge.w}
                    </span>
                  </div>
                ))}
              </div>
            </div>
          )
        })}
      </div>
    </div>

```

```

        <td className="py-2.5 px-3 text
            {prev || '-'}
        </td>
    </tr>
    );
    }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
    <div>
        <h4 className="text-lg font-bold text-slate
        <div className="bg-slate-950/80 rounded-lg
            {codeLines.map({item} => {
                const isActive = currentStep.activeCode
                return (
                    <div key={item.line} className={`py-1
                        isActive ? 'bg-indigo-900/80 text
                    }`}>
                    <span className="text-slate-600 sel
                    <span className="whitespace-pre fle
                        {isActive && <span className="text-
                    </span>}
                </div>
            )};
            }}}
        </div>
    </div>
</div>
</div>
</div>
);
}

```

```

const memoMap: { [key: number]: number } = {};

function simulateFib(n: number): number {
  generatedSteps.push({
    id: stepId++,
    n,
    action: 'call',
    desc: `Вызов fibМемо(${n}). Проверяем наличие в
    codeLine: 2,
    memoState: { ...memoMap }
  });

  if (n in memoMap) {
    generatedSteps.push({
      id: stepId++,
      n,
      action: 'memo_hit',
      desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
      codeLine: 2,
      memoState: { ...memoMap }
    });
    return memoMap[n];
  }

  if (n <= 2) {
    generatedSteps.push({
      id: stepId++,
      n,
      action: 'base_case',
      desc: `Базовый случай: n <= 2 (n=${n}). Возвр
      codeLine: 3,
      memoState: { ...memoMap }
    });
    return 1;
  }

  generatedSteps.push({
    id: stepId++,

```



```

        onClick={() => { setIsPlaying(false); set
        disabled={currentStepIndex === steps.length
        className="p-2 text-slate-400 hover:text-
        title="Шаг вперед"
      >
        <ChevronRight className="w-5 h-5" />
      </button>
    </div>
  </div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
  <h4 className="text-xs font-bold uppercase trac
  <div className="flex flex-wrap items-center gap
    {Array.from({ length: targetN }, (_, i) => i
    const isCached = num in memoState || num <=
    const val = num <= 2 ? 1 : memoState[num];
    const isCurrent = currentStep?.n === num;

    return (
      <div
        key={num}
        className={`flex flex-col items-center p
          isCurrent
            ? 'bg-emerald-950 border-emerald-500
            : isCached
            ? 'bg-slate-900 border-emerald-500/
            : 'bg-slate-900/50 border-slate-800
          `}
      >
        <span className="text-xs text-slate-400
        <span className={`text-xl md:text-2xl f
          {isCached ? val : '0'}
        </span>
        {num <= 2 && <span className="text-[9px
      </div>
    );

```

Интерактивный код

```

</button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between" >
  {activeTab === 'table' && (
    <div className="bg-slate-950 p-6 rounded-2xl border" >
      <h5 className="font-bold text-white text-sm" >
        <span className="text-emerald-400" > </span>
      </h5>
      <p className="text-sm text-slate-400" >
        В классической рекурсии для  $N=\{targetN\}$  по
        вычисленное значение в таблице, мы момента.
      </p>
      <div className="p-4 bg-slate-900 rounded-xl border
        t-slate-300">
        <span>Всего шагов симуляции: {steps.length}</span>
        <span className="text-emerald-400 font-bo
      </div>
    </div>
  )}

  {activeTab === 'code' && (
    <div className="bg-slate-950 rounded-2xl border" >
      <div className="bg-slate-900 px-6 py-3 border" >
        <span className="text-xs font-bold font-m" >
        <span className="text-xs text-emerald-400" >
      </div>
      <pre className="p-6 font-mono text-sm space" >
        {DP_CODE_LINES.map((line, idx) => {
          const lineNum = idx + 1;
          const isActive = currentStep?.codeLine :
          return (
            <div
              key={lineNum}
              className={`flex items-center px-4 p
              isActive

```

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Клики на вершину");
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п");
      return;
    }
  }
}
```



```

    </button>
  </div>

  <div className="bg-slate-950 rounded-xl p-4 border
    -hidden">
    <div className="absolute inset-0 bg-[radial-gra
    <svg className="w-full h-[260px] z-10 relative"
      {C1_EDGES.map((e, i) => {
        const u = C1_NODES.find(n => n.id === e.u)!
        const v = C1_NODES.find(n => n.id === e.v)!
        const key = `${Math.min(e.u,e.v)}-${Math.ma
        const used = c1VisitedEdges.includes(key);
        return <line key={i} x1={u.x} y1={u.y} x2={
          stroke={used ? "#6366f1" : "#475569"}
          strokeWidth={used ? 4 : 2}
          className="transition-all duration-300" />
      })}
      {C1_NODES.map(n => {
        const isLast = c1Path[c1Path.length-1] === n
        const visited = c1Path.includes(n.id);
        return <g key={n.id} className="cursor-poin
          {isLast && <circle cx={n.x} cy={n.y} r={1
          <circle cx={n.x} cy={n.y} r={11}
            fill={isLast ? "#6366f1" : visited ? "#
            stroke={isLast ? "#fff" : visited ? "#6
            strokeWidth={2.5}
            className="transition-all duration-200 l
          <text x={n.x} y={n.y+4} textAnchor="middl
          bel}</text>
        </g>;
      })}
    </svg>
    <div className="absolute top-2 left-2 bg-slate-
      Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.leng
    </div>
  </div>

  <div className={`p-3 rounded-xl border text-sm ${

```

```

        'Мы вызываем алгоритм Дейкстры для поиска нового
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане
        Это нарушит структуру дерева.'
},
{
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор
    options: [
        'Потому что он написан только для положительных ч
        'Он жадный и считает, что каждый следующий шаг де
            закрытые вершины.',
        'Отрицательные ребра создают гравитационный коллапс
        'Дейкстра просто не любил отрицательные балансы н
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты прие
        ля чего нужен алгоритм Беллмана-Форда.'
},
{
    id: 3,
    question: 'Какова ключевая фишка алгоритма Борувки
    options: [
        'Он использует меньше памяти благодаря коммунисти
        'Он умеет работать с отрицательными весами без ци
        'Он позволяет выполнять шаги слияния колхозов (ко
        'Он был разработан в Токио и поэтому самый быстрый
    ],
    correctAnswer: 2,
    explanation: 'Абсолютно верно! В Борувке каждая ком
        лелить вычисления на GPU или многопроцессорных кл
},
{
    id: 4,
    question: 'В чем главная суть Динамического програм
    options: [
        'Писать код очень быстро, динамично нажимая на кл

```

```

    };

    const handleNext = () => {
      if (currentQIndex + 1 < quizQuestions.length) {
        setCurrentQIndex(prev => prev + 1);
        setSelectedOption(null);
        setIsAnswered(false);
      } else {
        setShowResults(true);
      }
    };

    const handleRestart = () => {
      setCurrentQIndex(0);
      setSelectedOption(null);
      setIsAnswered(false);
      setScore(0);
      setShowResults(false);
    };

    if (showResults) {
      return (
        <div className="bg-slate-900/90 border border-slate-800 p-12">
          <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
            <Award className="w-10 h-10 text-white" />
          </div>
          <h3 className="text-3xl font-bold text-white mb-4">Поздравляем!</h3>
          <p className="text-slate-400 text-sm mb-6">Ты правильно ответил на все вопросы!</p>

          <div className="bg-slate-950 border border-slate-800 p-6">
            <p className="text-slate-400 text-sm uppercase">Итоговый результат:</p>
            <p className="text-5xl font-black text-indigo-400">{score}</p>
            <p className="text-slate-300 text-sm">
              {score === quizQuestions.length
                ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
                : score >= 3
              }
            </p>
          </div>
        </div>
      );
    }
  }
}

```

```

{currentQ.options.map((option, idx) => {
  const isSelected = selectedOption === idx;
  const isCorrect = idx === currentQ.correctA

  let optionStyle = "bg-slate-800/80 hover:bg-
  if (isAnswered) {
    if (isCorrect) {
      optionStyle = "bg-emerald-950/80 border
    } else if (isSelected) {
      optionStyle = "bg-rose-950/80 border-ro
    } else {
      optionStyle = "bg-slate-800/40 border-s
    }
  }
})

return (
  <div
    key={idx}
    onClick={() => handleSelectOption(idx)}
    className={"flex items-start gap-4 p-4"}
  >
    <div className="mt-0.5 shrink-0">
      {isAnswered && isCorrect ? (
        <CheckCircle2 className="w-6 h-6 te
      ) : isAnswered && isSelected && !isCo
        <XCircle className="w-6 h-6 text-ro
      ) : (
        <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
          {String.fromCharCode(65 + idx)}
        </div>
      )}
    </div>
    <div className="flex-1 text-sm md:text-l
      {option}
    </div>
  </div>
);

```

```
import { useState, useEffect } from 'react';
```

```
interface Step {  
  k: number; i: number; j: number;  
  matrix: number[][]; updated: boolean;  
  log: string; activeCodeLine: number;  
}
```

```
export default function FloydViz() {  
  const [steps, setSteps] = useState<Step[]>([]);  
  const [currentStepIndex, setCurrentStepIndex] = useSt  
  const [isPlaying, setIsPlaying] = useState(false);
```

```
  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',  
  const nodesSvg = [  
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },  
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },  
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },  
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },  
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },  
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },  
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },  
  ];
```

```
  const initialMatrix = [  
    [0, 4, 999, 2, 999, 999, 999],  
    [999, 0, 3, 999, 999, 3, 999],  
    [999, 999, 0, 999, 2, 999, 999],  
    [999, 999, 999, 0, 4, 2, 999],  
    [999, 999, 999, 999, 0, 999, 1],  
    [999, 999, 999, 999, 999, 0, 5],  
    [999, 1, 999, 999, 999, 999, 0]  
  ];
```

```
  const adjList: { [key: number]: { v: number; w: numbe  
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
```

```

    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
        dist[i][j] = viaVal;
        simulationSteps.push({
          k, i, j, matrix: dist.map(r => [...r]), up: 0,
          log: ` Улучшение пути [${i}]→[${j}] через [${k}].`,
          activeCodeLine: 7,
        });
      }
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), up: 0,
  log: ` Все пары отработаны. Алгоритм Флойда завершён.`,
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

        <marker id="arrow-fw-normal" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#475569" />
        <marker id="arrow-fw-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
        <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></marker>
</defs>
{Object.keys(adjList).flatMap((uStr) => {
  const u = parseInt(uStr);
  return adjList[u].map((edge) => {
    const uNode = nodesSvg.find((n) => n.id === u);
    const vNode = nodesSvg.find((n) => n.id === v);
    const isTarget = currentStep.i === u && currentStep.j === v;
    const isVia = (currentStep.i === u && currentStep.j !== v) ||
    let strokeColor = '#475569'; let markerId = 'arrow-fw-normal';
    if (isTarget) { strokeColor = '#10b981'; markerId = 'arrow-fw-active'; }
    else if (isVia) { strokeColor = '#818cf8'; markerId = 'arrow-fw-via'; }
    return (
      <g key={`-${u}-${edge.v}`}>
        <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
markerEnd={markerId} strokeDasharray={isTarget ? [5, 5] : []}
        <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)}
isVia ? '#818cf8' : '#64748b'} strokeWidth={2} />
        <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)}
8fafc'} fontSize="10" fontWeight="bold" text={edge.v} />
      </g>
    );
  });
});
}}
}
nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748b';
  if (isK) { fill = '#4f46e5'; stroke = '#a1a8b8'; }
  else if (isI) { fill = '#b45309'; stroke = '#c9c9c9'; }
  else if (isJ) { fill = '#047857'; stroke = '#000000'; }
  return (
    <rect x={node.x} y={node.y} width={node.w} height={node.h}
    <circle cx={node.x + node.w / 2} cy={node.y + node.h / 2}
    <text x={node.x + node.w / 2} y={node.y + node.h / 2}
  );
});

```

```

        dow' : isVia ? 'bg-indigo-500 text-white bo
            {edge.v} ({edge.w})
        </span>
    )
    }}}
</div>
);
}}
</div>
</div>
</div>

```

```

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border">
        <thead><tr className="bg-rose-950/40 text-white">
          <th className="p-2 border-r border-rose-950">
            {nodeNames.map((n, idx) => <th key={idx}
              -400 font-bold' : ''}>{n.split(' ')[0]} {
            </tr></thead>
            <tbody className="text-xs font-mono">
              {currentStep.matrix.map((row, i) => (
                <tr key={i} className="border-b border-rose-950">
                  <td className={`p-2 bg-rose-950/40 text-white`
                    tStep.k === i ? 'text-amber-400' : ''}}>
                    {nodeNames[i].split(' ')[0]} {i}
                  </td>
                  {row.map((val, j) => {
                    const isUpd = currentStep.i === i
                    const isVia = currentStep.k !== i
                    const isDest = currentStep.j === j});
                    let bg = 'bg-slate-900/40 text-slate-500';
                    if (isUpd) bg = 'bg-emerald-900/80 text-emerald-500';
                    if (isVia) bg = 'bg-indigo-900/80 text-indigo-500';
                    if (isDest) bg = 'bg-rose-900/80 text-rose-500';
                    <div className={bg}>{val}</div>
                  )}
                </tr>
              )}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

```



```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```
type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};
```

```

    ray.from(vis)] });

while (q.length > 0) {
  const v = q[0];
  steps.push({ type: 'process', node: v, desc: `Д  

    q], visitedNodes: [...Array.from(vis)] });
  q.shift();

  for (const u of adj[v]) {
    if (!vis.has(u)) {
      vis.add(u);
      q.push(u);
      steps.push({ type: 'visit', node: u, de  

        tack: [...q], visitedNodes: [...Array.from(  

        }
    }
  }

  steps.push({ type: 'backtrack', node: '', desc: `Оч  

    om(vis)] });

  return steps;
}

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("df
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? genera

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);

  useEffect(() => {

```



```

    }
    return a;
}

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
    return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT PRESETS = [

```

```

-----
const insertPatient = useCallback(({name: string, prio
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выпис
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);

  const preset = PATIENT_PRESETS[Math.floor(Math.rand
  const finalName = name || preset.name.split(' ')[0]
  const finalEmoji = preset.emoji;
  const finalSymptom = preset.symptom;

  const newPatient: Patient = {
    id: `p${uid++}`,
    name: finalName,
    emoji: finalEmoji,
    symptom: finalSymptom,
    priority,
    animState: 'in',
  };

  const nextHeap = heapInsert(heap, newPatient).map(x
  // Mark specifically this new patient as 'in' in th
  const target = nextHeap.find(x => x.priority === pr
  const marked = nextHeap.map(x => x.id === (target?.

  setHeap(marked);
  addLog(` Поступил пациент: ${finalName} с тяжестью

  setTimeout(() => {
    setHeap(nextHeap);
    setBusy(false);
  }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {

```

```

        const a = prev.map(x => ({ ...x }));
        a[0] = { ...a[a.length - 1] };
        a.pop();
        return siftDown(a).map(x => ({ ...x, animStat
    }));

    // Patient gets healed in the bed
    setTimeout(() => {
        setHealing(prev => prev ? { ...prev, animStat
        addLog(` Успех! Пациент ${topPatient.name} пр
        setTimeout(() => {
            setHealing(null);
            setBusy(false);
        }, 600);
    }, 800);

    }, 350);
    }, 650);
}, [busy, heap]);

const reset = () => {
    setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
    setLog([' База данных скорой помощи перезагружена.
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
    const res: React.ReactElement[] = [];
    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < heap.length) {
        const lp = nodePos(l);
        res.push(
            <line key={`el${idx}`}
                x1={` ${p.x}%` } y1={` ${p.y + 4}%` }

```

```

    {/* Добавить случайного */}
    <button
      onClick={handleRandomInsert}
      disabled={busy || heap.length >= 15}
      className="flex-1 min-w-[150px] bg-gradient-to-r from-teal-600 to-teal-500
        opacity-40 disabled:cursor-not-allowed text-white font-medium p-4
        ve:scale-95 transition-all cursor-pointer focus:outline-none"
    >
      <span> Привезти по Скорой (INSERT)</span>
    </button>

    {/* Добавить кастомного */}
    <form onSubmit={handleCustomInsert} className="flex flex-col gap-4"
    >
      <input
        type="text"
        required
        value={customName}
        onChange={e => setCustomName(e.target.value)}
        placeholder="Имя пациента"
        className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
      />
      <div className="flex flex-col items-center min-h-[40px]"
      >
        <span className="text-[9px] font-mono text-slate-400"> Приоритет</span>
        <input
          type="range"
          min="10"
          max="99"
          value={customPriority}
          onChange={e => setCustomPriority(Number(e.target.value))}
          className="w-20 accent-emerald-500 cursor-pointer"
        />
      </div>
      <button
        type="submit"
        disabled={busy || !customName.trim() || !customPriority}
        className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white font-medium
          px-4 rounded-xl shadow-lg active:scale-95 transition-all"
      >
        Добавить
      </button>
    </form>
  </div>
</div>

```



```
</svg>
```

```
{heap.map((patient, idx) => {
  const pos = nodePos(idx);
  const isRoot = idx === 0;
  return (
    <div
      key={patient.id}
      className={`
        absolute flex flex-col items-center
        -translate-x-1/2 -translate-y-1/2 t
        ${patient.animState === 'in' ? 'anim
        ${patient.animState === 'winner' ?
        ${patient.animState === 'out' ? 'an
        ${isRoot ? 'bg-rose-950/80 border-r
      `}
      style={{
        left: `${pos.x}%`,
        top: `${pos.y}%`,
        width: isRoot ? 60 : 52,
        height: isRoot ? 60 : 52,
      }}
    >
      <span className="text-2xl leading-non
      <span className="text-[10px] font-mon
        {patient.priority}%
      </span>

      {/* Tooltip */}
      <div className="absolute bottom-full
        <div className="bg-slate-950 border
0 shadow-xl">
        <div className="font-bold text-wh
        <div className="text-emerald-400"
        </div>
      </div>
    </div>
  </div>
);
```

```

        </div>
      )}
    </div>

    <div className="w-full h-3 bg-gradient-to-r f
  </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-emerald-
        <span>{entry}</span>
      </div>
    }}}
  </div>
</div>
);
};

```

ФАЙЛ 41/82: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { createContext, useContext, useEffect, u
```

```

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

```

```
};
```

```
export const Svg: React.FC<{ children: React.ReactNode;
  const paused = useContext(DemoPauseContext);
  return (
    <div className="w-full">
      <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
        {children}
      </svg>
      {caption && (
        <div className="text-[10px] text-slate-400 text-center">
          {caption}
        </div>
      )}
      <div className="text-[9px] text-slate-600 text-center">
        {paused ? "пауза – курсор на карточке" : "наведи"}
      </div>
    </div>
  );
};
```

```
export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
  );
```

```

        caption={
          step === 0
            ? "k ещё запрещена: путь  $i \rightarrow j = 9$ "
            : step === 1
              ? "Разрешаем пересадку через k:  $3 + 4 = 7$ "
              : " $7 < 9 \rightarrow D[i][j] = 7$ "
        }
      >
      <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
      <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
      <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

      <text x={130} y={112} textAnchor="middle" fontSize=
        9
      </text>
      <text x={72} y={54} textAnchor="middle" fontSize=
        3
      </text>
      <text x={188} y={54} textAnchor="middle" fontSize=
        4
      </text>

      <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
      <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
      <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
    </Svg>
  );
};

```

/** Компоненты связности: несколько «островов», которые

```
export const ComponentsDemo: React.FC = () => {
```

```
  const comps: [string, string][][] = [
    [
      ["A", "B"], ["B", "C"],
      ["D", "E"],
      ["F", "G"], ["G", "H"], ["F", "H"],
    ],
  ];
```

```
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
```

```

        const a = pos[u];
        const b = pos[v];
        return (
          <g key={i}>
            <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
              {step >= 2 && (
                <text
                  x={({a[0] + b[0]) / 2}
                  y={({a[1] + b[1]) / 2 - 4}
                  textAnchor="middle"
                  fontSize={10}
                  fontWeight="700"
                  fill={C.warn}
                >
                  {w}
                </text>
              )}
            </g>
          );
        }
      )
    {Object.entries(pos).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={C.idle} />
    ))}
  </Svg>
);
};

```

/** Сжатие координат: разреженные числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

```

```

  return (
    <Svg
      caption={

```

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

```
const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, st
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id
```

```
  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }>
          <circle r={13} fill={color(id)} stroke={C.idl} />
          <text textAnchor="middle" dominantBaseline="c
            {id}
          </text>
        </g>
      ))}
    </Svg>
  )
```

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "A"],
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order[step - 1] : [];
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
          stroke={wave.includes(k) ? "#a5b4fc" : undefined} />
      })}
    />
  );
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
```

```

        <text x={n.x} y={n.y} textAnchor="middle" d
        {idx >= 0 && <circle cx={n.x + 30} cy={n.y
        {idx >= 0 && (
            <text x={n.x + 30} y={n.y - 12} textAncho
            x + 1}</text>
        )}
    </g>
  );
  }}}
  <text x={224} y={62} textAnchor="middle" fontSize=
  <text x={224} y={74} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
      <Edge a={[60, 70]} b={[200, 70]} color={improved
      <text x={130} y={54} textAnchor="middle" fontSize=
      <Node x={60} y={70} r={20} fill={C.active} label=
      <Node x={200} y={70} r={20} fill={improved ? C.do
      <text x={60} y={106} textAnchor="middle" fontSize=
      <text x={200} y={106} textAnchor="middle" fontSiz
      <text x={130} y={22} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [

```



```
export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без ц`
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
              <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
                textAnchor="middle" fontSize={9} fontWeigh
              </g>
            );
          }}}
      {Object.entries(pos).map(([k, [x, y]]) => (<Node
    </Svg>
  );
};
```

```
export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
```

```

const states = [["A"], ["A", "B"], ["A", "B", "C"], [
const step = useLoop(states.length, 800);
const items = states[step];
const popping = step >= 3;
return (
  <Svg caption={popping ? "pop: забираем ВЕРХНИЮ tape."
    <rect x={90} y={22} width={80} height={96} rx={6}
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={94} y={100 - i * 26} width={72} height={26}
          fill={i === items.length - 1 ? (popping ? C
        <text x={130} y={111 - i * 26} textAnchor="middle"
      </g>
    ))}
    <text x={130} y={16} textAnchor="middle" fontSize={12} />
  </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к
    <defs>
      <marker id="mdArrow" markerWidth="6" markerHeight="6"
        <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
      </marker>
    </defs>
    <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
    <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={30 + i * 54} y={56} width={46} height={26}
        <text x={53 + i * 54} y={71} textAnchor="middle"
      </g>
    ))}
  );
};

```

```

    <Svg caption="Путь от корня по буквам = слово; общий
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]}}
      {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
    </Svg>
  );
};

```

```

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, M
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]}}
      {TRIE_NODES.map((n) => (
        <Node key={n.id} x={n.x} y={n.y} label={n.ch}
        fill={wave.includes(n.id) ? C.active : doneId
      )}}
      <text x={6} y={12} fontSize={9} fill={C.dim}>очер
    </Svg>
  );
};

```

```

export const SuffixLinkDemo: React.FC = () => {
  const links: [number, number][] = [[1, 0], [2, 0], [3
  const step = useLoop(links.length + 1, 900);
  return (
    <Svg caption="Суффиксная ссылка: «куда откатиться,
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,

```

```
};
```

```
export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
      : `о префиксные суммы P`} >
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={12}
            fill={show && i >= l && i <= r ? C.active : C.dim} />
          <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
        </g>
      ))}
      <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
      {pref.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 33} y={78} width={29} height={12}
            fill={show && i === l ? C.bad : show && i > l ? C.dim : C.idle}
            stroke={C.idleStroke} style={{ transition: "fill 0.2s" }} />
          <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
        </g>
      ))}
    </Svg>
  );
};
```

```
export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
```

```

    <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
    <text x={92} y={64} textAnchor="middle" dominantB
    <text x={206} y={34} textAnchor="middle" fontSize
    <circle cx={196} cy={84} r={13} fill={step === 1
    <text x={196} y={84} textAnchor="middle" dominant
        ?"}</text>
    <text x={130} y={124} textAnchor="middle" fontSize
</Svg>
  );
};

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} hei
            fill={step > 0 && (idx < len || (idx > i -
              style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="mi
            ext>
          <text x={32 + idx * 33} y={70} textAnchor="mi
            fill={idx <= i && step > 0 ? C.warn : C.idle
        </g>
      )}}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</t
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</t
    <text x={130} y={106} textAnchor="middle" fontSize
      π[i] – длина самого длинного «префикс = суффикс
    </text>
    </Svg>
  );
};

```

```
import {
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
```

```
interface Props {
  vizId: string | null;
  onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-en
    <div className="absolute inset-0 bg-slate-950/80
    <div className="relative w-full sm:max-w-5xl max-l
      ded-2xl shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm
              {viz.hint && <p className="text-[11px] text
            </div>
          <button
            onClick={onClose}
            className="p-2 rounded-lg bg-slate-800 hove
            aria-label="Заккрыть симулятор"
```

```

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({ top: 0, left: 0 });

  useEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });

    useEffect(() => {
      if (!anchor) return;
      const onKey = (e: KeyboardEvent) => {
        if (e.key === "Escape") onClose();
      };
      window.addEventListener("keydown", onKey);
      window.addEventListener("scroll", onClose, true);
      return () => {
        window.removeEventListener("keydown", onKey);
        window.removeEventListener("scroll", onClose, true);
      };
    }, [anchor]);
  }, [anchor]);
};

```



```
      {entry.formal && {
        <p className="text-[11.5px] leading-relaxed"
          {entry.formal}
        </p>
      }}

      {entry.complexity && {
        <div className="text-[11px] font-mono text-0"
          Сложность: {entry.complexity}
        </div>
      }}

      {viz && simulatorId && {
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-center
            rounded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      }}
    </div>
  </div>,
  document.body
);
};
```

ФАЙЛ 48/82: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
```

```

        if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
        if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
        return "stroke-slate-700 stroke-dasharray-4";
    }

    if (e.id === 'AB') {
        if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

```

```
const getDesc = () => {
  switch (step) {
    case 0: return (
      <div>
        <b>Исходный граф.</b> Имеется ребро
        Если мы запустим быстрый алгоритм Д
      </div>
    );
    case 1: return (
      <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
      </div>
    );
    case 2: return (
      <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1
      </div>
    );
    case 3: return (
      <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый в
        <div className="text-center font-mo
        Это повышает или понижает вес ребра
      </div>
    );
    case 4: return (
      <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-r
        Вычисляем:  $-2 + 0 - (-2) =$  <span cl
      </div>
    );
  }
}
```

```

        <defs>
            <marker id="arrow-slate" viewBox="0 0 10 5" style="vertical-align: middle; color: slategray;">
                <path d="M 0 0 L 10 5 L 0 10" />
            </marker>
            <marker id="arrow-slate-dim" viewBox="0 0 10 5" style="vertical-align: middle; color: slategray; opacity: 0.5;">
                <path d="M 0 0 L 10 5 L 0 10" />
            </marker>
            <marker id="arrow-amber" viewBox="0 0 10 5" style="vertical-align: middle; color: #FFC300;">
                <path d="M 0 0 L 10 5 L 0 10" />
            </marker>
            <marker id="arrow-pink" viewBox="0 0 10 5" style="vertical-align: middle; color: #FF69B4;">
                <path d="M 0 0 L 10 5 L 0 10" />
            </marker>
            <marker id="arrow-emerald" viewBox="0 0 10 5" style="vertical-align: middle; color: #3CB371;">
                <path d="M 0 0 L 10 5 L 0 10" />
            </marker>
        </defs>

        {/* Edges */}
        {edges.map(({e, i}) => {
            const u = nodes.find(n => n.id === e.u);
            const v = nodes.find(n => n.id === e.v);

            const isS = e.u === 'S';
            if (isS && (step === 0 || step === 1)) {
                return null;
            }

            const midX = (u.x + v.x) / 2;
            const midY = (u.y + v.y) / 2;

            const edgeColorClass = getEdgeColorClass(e);
            const textHigh = isEdgeTextHigh(e);
            const textEmer = isEdgeTextEmer(e);
        })}
```

```

        -white">
                                {n.label}
                                </text>
                                {pot !== null && {
                                <g transform={`
fade-in zoom-in duration-500">
                                <rect x="-1
k-500 stroke-1" />
                                <text x="0"
font-bold fill-pink-400">
                                {pot}
                                </text>
                                </g>
                                }}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

                                <div className="w-full md:w-[320px] flex
                                {/* Log Box */}
                                <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
                                {getDesc()}
                                </div>

                                {/* Controls */}
                                <div className="flex bg-slate-900 b
                                <button onClick={() => setStep(1
                                className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
                                <RotateCcw strokeWidth={3}
                                </button>
                                <button onClick={() => setStep(1
                                className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на

```

```

interface GNode {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId

```

```

    recordStep(
      "init",
      "Инициализация алгоритма Косарайю. Начинаем первый",
      null,
      false,
      -1,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    initialEdges.forEach((e) => adj[e.u].push(e.v));

    // DFS 1: post-order list
    const dfs1 = (u: number) => {
      visited.add(u);
      recordStep(
        "dfs1",
        `DFS 1: зашли в вершину ${u}, помечаем посещение`,
        u,
        false,
        -1,
      );

      for (const to of adj[u]) {
        if (!visited.has(to)) {
          dfs1(to);
        }
      }

      order.push(u);
      recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Записываем`,
        u,
        false,
        -1,
      );
    };
  };

```

```

        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,
    true,
    sccId,
);

```



```

    });

    return (
      <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <Layers className="w-5 h-5 text-emerald-400"
              Визуализатор Алгоритма Косарайю (Поиск SCC)
            </h3>

          <div className="flex items-center gap-2 bg-slate-
            <button
              onClick={togglePlay}
              className="flex items-center gap-2 px-3 py-
                d-500 text-white"
            >
              {isPlaying ? (
                <Pause className="w-4 h-4 ml-1" />
              ) : (
                <Play className="w-4 h-4 ml-1" />
              )}
              {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button
              onClick={reset}
              className="flex items-center justify-center
                ors"
            >
              <RotateCcw className="w-4 h-4" />
            </button>
          </div>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-12
          {/* Playfield SVG */}
          <div className="lg:col-span-8 bg-[#0B1120] rela
            <svg className="w-full h-full max-w-[650px]"

```

```

{initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id === edge.u);
  const v = initialNodes.find((n) => n.id === edge.v);

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =
    step.sccMap[edge.u] !== step.sccMap[edge.v] ||
    step.sccMap[edge.u] === step.sccMap[edge.v] &&
    step.sccMap[edge.u] !== step.sccMap[edge.v];
  let stroke = "#475569";
  let markerId = "arrow";

  if (step.transposed) {
    stroke = isSccEdge ? "#818cf8" : "#4f4682";
    markerId = "arrow-transposed";
  } else {
    if (
      step.currentNode === edge.u ||
      step.currentNode === edge.v
    ) {
      stroke = "#10b981";
      markerId = "arrow-active";
    }
  }
}

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
  <line

```

```

        />
        <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
        >
            {node.label}
        </text>

        {sccId && (
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        )}
    </g>
  );
  }}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
    1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
  )}
</div>

```

```

<div>
  <span className="text-[10px] text-slate-500">
    ЛОГ ОПЕРАЦИЙ:
  </span>
  <div className="space-y-1.5 max-h-[150px]">
    {steps
      .slice(0, currentStepIdx + 1)
      .reverse()
      .slice(0, 5)
      .map((s, idx) => {
        <div
          key={idx}
          className={`text-[11px] p-1.5 rounded`
        >
          {s.desc}
        </div>
      )}}
    </div>
  </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    № {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1.5">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-800 px-2 py-1"
    >
      Назад
    </button>
    <button
      disabled={currentStepIdx >= steps.length - 1}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-800 px-2 py-1"
    >
      Далее
    </button>
  </div>
</div>

```

```

    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
  }

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdge

```

```

        title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
        description: 'Оно соединяет две другие бесцветны
        type: 'info'
    }, ...prev]));
},
// Step 4: Include D-E
() => {
    setVertices(prev => prev.map(v => v.id === 3 || v
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-E в осто́ве',
        description: 'Вершины D и E окрашены в синий цв
        type: 'success'
    }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в осто́ве',
        description: 'Вершина C перекрашена в красный ц
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{

```

```

        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.'
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{

```

```

        description: 'Оба конца E и H уже фиолетовые. С
        type: 'warning'
    }, ...prev]);
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро E-H.',
        type: 'error'
    }, ...prev]);
},
// Step 21: Inspect H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю
        type: 'info'
    }, ...prev]);
},
// Step 22: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
        type: 'success'
    }, ...prev]);
},
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E

```



```

        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6)
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {

```

```
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (Е) тянет',
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. Р
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
```

```

        setLogs(prev => [{
            title: 'Успех: Вершина H захвачена',
            description: 'В кучу добавлено H-I(12).',
            type: 'success'
        }, ...prev]));
    },
    // Step 20: Pop E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Куча выдает E-H (вес 11)',
            description: 'Обе вершины E и H уже в плесени.',
            type: 'warning'
        }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {

```

```

// Component 1: {A, B, C} (colored red)
// Component 2: {D, E, F, G, H, I} (colored gold)
setVertices(prev => prev.map(v =>
  [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
));
setEdges(prev => prev.map(e => {
  if (['0-1', '1-2'].includes(e.id)) {
    return { ...e, status: 'included', color: 'red' }
  }
  if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
    return { ...e, status: 'included', color: 'blue' }
  }
  return e;
}));
setLogs(prev => [{
  title: 'Слияние: Деревни объединились в колхозы',
  description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
  type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    status: 'included', color: 'red'
  } : e));
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост',
    description: 'Теперь Красный и Синий колхозы объединились в один колхоз. Самый дешевый мост A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в наметке.',
    type: 'warning'
  }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
  setEdges(prev => prev.map(e => ({ ...e, status: 'included', color: 'blue' })));
  setLogs(prev => [{
    title: 'Завершение: Все деревни объединились в один колхоз',
    description: 'Все дороги построены. Колхозы объединились в один колхоз.',
    type: 'success'
  }, ...prev]);
}
}

```

```

    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин графа
                    уже покрашены, и тебе нужно выбрать
                    ребро (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else if (algo === 'prim') {
        setLogs([
            {
                title: 'Введение в Плесень (Прима)',
                description: 'Логика «Плесени»: выбираем стар
                    едь (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else {
        setLogs([
            {
                title: 'Введение в Коллективизацию (Борувка)',
                description: 'Логика «Борувки»: Каждая из 9 д
                    ет выбрать соседа, чтобы объединиться в колхоз.',
                type: 'info',
            },
        ]);
    }
};

```

```

const getColorClass = (color: string, id: number) => {
    switch (color) {
        case 'red': return 'bg-red-500 border-red-300 tex
        case 'blue': return 'bg-blue-500 border-blue-300
        case 'purple': return 'bg-purple-600 border-purple
        case 'mold': return 'bg-emerald-500 border-emerald
        default:
            if (activeAlgo === 'prim' && id === 4 && stepIn

```

```

    { /* Algorithm Tabs */ }
    <div className="flex flex-wrap items-center gap-1" />
    <div className="flex items-center bg-slate-400 p-2" />
    <button
      onClick={() => handleReset('kruskal')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'kruskal'
          ? 'bg-indigo-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <GitGraph className="w-3.5 h-3.5" />
      <span>Краскал (Билет 18)</span>
    </button>
    <button
      onClick={() => handleReset('prim')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'prim'
          ? 'bg-emerald-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <Layers className="w-3.5 h-3.5" />
      <span>Прима (Билет 19)</span>
    </button>
    <button
      onClick={() => handleReset('boruvka')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'boruvka'
          ? 'bg-rose-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >

```

```
<div className="lg:col-span-2 bg-slate-950 rounded-2xl p-4 active min-h-[480px] overflow-hidden">
```

```
  { /* Legend */ }
```

```
  <div className="absolute top-4 left-4 bg-slate-900 p-4 text-center gap-3 text-xs">
```

```
    <div className="flex items-center gap-1.5">
```

```
      <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
    </div>
```

```
    {activeAlgo === 'kruskal' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```
      </>
```

```
    ) : activeAlgo === 'prim' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-1 bg-sky-600">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```
      </>
```

```
    ) : (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-900">
```

```
        </div>
```

```

        fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
        fontSize="4.5"
        fontFamily="monospace"
        fontWeight="bold"
        textAnchor="middle"
        dominantBaseline="middle"
        className="select-none filter drop
dy="-1.5"
    >
        {edge.weight}
    </text>
</g>
    );
  }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: v
      className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'p
        <span className="text-[8px] block -
      }}
    </div>
  )}}
</div>

```

```

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom
shadow-xl z-20">

```



```

    <div className="flex-1 p-4 overflow-y-auto"
      {logs.map((log, idx) => {
        <div
          key={idx}
          className={
            "p-4 rounded-xl border transition-a
            {log.type === 'success'
              ? 'bg-emerald-950/40 border-emera
              : log.type === 'warning'
              ? 'bg-amber-950/40 border-amber-5
              : log.type === 'error'
              ? 'bg-rose-950/40 border-rose-500
              : 'bg-slate-900/60 border-slate-7
            }
          }
        >
          <div className="flex items-center gap
            {log.type === 'success' && <CheckCi
            {log.type === 'warning' && <HelpCir
            {log.type === 'error' && <XCircle c
            {log.type === 'info' && <Play class
            <h5 className="font-bold text-sm le
          </div>
          <p className="text-xs text-slate-300
        </div>
      })}
    </div>
  </div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
  <div className="flex flex-col sm:flex-row items
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-40
      <h4 className="text-xl font-bold text-white
    </div>

```

```

        <Award className="w-3.5 h-3.5 text-indigo-500">
    </p>
    <p className="text-2xl font-black text-indigo-500">
    <p className="text-slate-400 text-xs mt-2">
</div>
<div className="bg-slate-950 p-4 rounded-2xl">
    <p className="text-slate-400 text-xs font-medium">
ожении:</p>
    <p className="text-xs text-slate-300 leading-relaxed">
ди и красим вершины в один цвет, бракуя циклы.
    </div>
</div>
<div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
            <Code className="w-3.5 h-3.5 text-indigo-500">
            </p>
            <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
```

```
def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
```

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'boruvka' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

p>

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r

```
    </div>
  );
};
```

ФАЙЛ 51/82: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
```

}

ФАЙЛ 52/82: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```
interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}
```

```
/**
```

```
 * Мобильное содержание: липкая панель снизу-сверху + в
```

```
 * На десктопе не рендерится (там обычный сайдбар).
```

```
 */
```

```
export const MobileToc: React.FC<Props> = ({ selectedChapterId }) => {
  const [open, setOpen] = useState(false);
```

```
  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);
```

```
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
    window.addEventListener("keydown", onKey);
```

```
        <div className="flex-1 min-h-0">
          <Sidebar
            selectedChapterId={selectedChapterId}
            setSelectedChapterId={setSelectedChapterId}
            onNavigate={() => setOpen(false)}
          />
        </div>
      </div>
    </div>
  )}
</>
);
};
```

ФАЙЛ 53/82: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab }) => {
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
```

```

        id="tab-simulator-btn"
        onClick={() => setActiveTab('simulator')}
        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'simulator'
                ? 'bg-indigo-600 text-white shadow-lg s
                : 'text-slate-400 hover:text-white'
        }`}
    >
        <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <a
        id="download-pdf-btn"
        href="/export/full_code_all_pages.pdf"
        download="full_code_all_pages.pdf"
        target="_blank"
        rel="noreferrer"
        className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
        title="Скачать полный PDF сборник всех стра
    >
        <FileDown className="w-4 h-4" /> Скачать PD
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
        className="flex items-center justify-center
        :bg-sky-600/20 border border-sky-500/30 tra
        title="Скачать полный TXT файл со всем кодо
    >
        <FileText className="w-4 h-4" /> TXT
    </a>
    <button
        id="download-html-btn"
        onClick={saveBook}

```

```

    ];
    const edges = [
      [0,1],[0,2],[0,3],[0,4],
      [1,2],[1,3],[1,4],
      [2,3],[2,4],
      [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
      if (a===c||a===d||b===c||b===d) return
      const p1=findPt(a),p2=findPt(b),p3=find
      function ccw(ax:number,ay:number,bx:num
        return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
      }
      return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
      for(let j=i+1;j<edges.length;j++) {
        if (intersect(edges[i][0],edges[i][1]
          if (!crosses.includes(i)) crosses.p
          if (!crosses.includes(j)) crosses.p
        }
      }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
      const p1=findPt(edges[i][0]),p2=findPt(
      const xing = crosses.includes(i);
      lines.push(<line key={`l${i}`} x1={p1.x
        stroke={xing ? "#f43f5e" : "#4f46e5"}
      }
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
      circles.push(<g key={`c${p.id}`}>
        <circle cx={p.x} cy={p.y} r={8} fill=
        <text x={p.x} y={p.y+3} textAnchor="m

```



```

    for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
      if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
        if (!crosses.includes(i)) crosses.push(i);
        if (!crosses.includes(j)) crosses.push(j);
    }
  }
  const lines = []; const circles = [];
  for(let i=0;i<edges.length;i++) {
    const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
    const xing=crosses.includes(i);
    lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
      stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
  }
  const labels = ["\u04141","\u04142","\u04143","\u04144"];
  for(const p of pts) {
    circles.push(<g key={`c${p.id}`}>
      <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
      <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
    </g>);
  }
  return <>{lines}{circles}</>;
}
}
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20">
  V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950/40 padding-10">
  ▲ Запомни: K5 и K3,3 – два кита непланарности.
  гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
</div>
</div>
);
}

```

```

        голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён',    emoji: '👨‍💻', color: 'red',
      ишу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис',      emoji: '🐈', color: 'blue',
      ез сметаны, плиз.', animState: 'idle' },
  ]);

```

```

const [servedHistory, setServedHistory] = useState<string[]>([]);
const [isServing, setIsServing]         = useState(false);
const [shakeEmpty, setShake]             = useState(false);
const [log, setLog]                      = useState<string[]>([]);

```

```

const addLog = (msg: string) => setLog(prev => [...prev, msg], {
  shouldUpdate: true
});

```

```

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяжёло!');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
}, [isServing]);

```

```

const preset = PRESETS[counter % PRESETS.length];
counter++;

```

```

const newGuest: Customer = {
  id: Date.now().toString(),
  ...preset,
  animState: 'in',
};

```

```

setQueue(prev => [...prev, newGuest]);
addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

```

```

setTimeout(() => {
  setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c));
}, 1000);

```

```

    { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', text: 'Я  

      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', text: 'Я  

      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я  

      без сметаны, плиз.', animState: 'in' },
  ]));
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!']);
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-indigo-950/40 border border-indigo-500 p-6">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
          <p className="text-xs text-slate-300 mt-1 leading-4">
            В очереди за супом всё честно. Кто первый встал, тот и ест.  

            Новые гости встают строго <b>в конец</b> очереди.  

            Здесь нет обхода и блата — только строгий порядок.
          </p>
        </div>
      </div>

      {/* УПРАВЛЕНИЕ */}
      <div className="flex items-center gap-3 flex-wrap">
        <button
          onClick={enqueue}
          disabled={isServing}
          className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600  

            text-white font-medium rounded-md opacity-40 disabled:cursor-not-allowed transition-all  

            hover:scale-95 transition-all cursor-pointer">
            Добавить в очередь
          </button>
      </div>
    </div>
  );
}

```

```

<div className="relative">
  <div className="absolute -top-10 left-0">
    <div className="w-1.5 h-6 bg-slate-400">
      <div className="w-2 h-4 bg-slate-400">
      </div>
    </div>
    <span className="text-5xl block animate-pulse">
    </span>
  </div>
  <div className="text-center">
    <span className="text-3xl block">
    </span>
    <span className="text-[10px] font-bold font-mono tracking-wider">
    </span>
    <div>
      <div>
        ПОВАР
      </div>
    </div>
  </div>
</div>

```

{/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}

```

<div className="flex flex-col items-center">
  <span className="text-2xl animate-pulse">
  </span>
  <span className="text-[8px] font-mono tracking-wider">
  </span>
</div>

```

{/* СПИСОК ОЧЕРЕДИ */}

```

<div className="flex-1 flex items-end gap-4">
  {queue.length === 0 ? (
    <div className="flex-1 flex flex-col">
      <span className="text-4xl mb-1">
      </span>
      <p className="text-xs font-bold font-mono">
      </p>
      <p className="text-[10px]">Все сыты
      </p>
    </div>
  ) : (
    queue.map((customer, idx) => {
      const isHead = idx === 0;
      const isTail = idx === queue.length - 1;
      return (
        <div
          key={customer.id}
          className={`

```

```

        });
      })
    })
  </div>

</div>
</div>

  { /* Пол кухни */ }
  <div className="w-full h-3 bg-gradient-to-r f
</div>

  { /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
  <div className="md:col-span-3 bg-slate-900 round
    <div className="absolute top-4 left-4 text-[1
      Сытые гости (Архив FIFO)
    </div>

    <div className="absolute top-4 right-4 flex i
      order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pu
      <span>СЫТЫ </span>
    </div>

    <div className="flex-1 flex flex-col justify-f
      {servedHistory.length === 0 ? (
        <div className="flex-1 flex flex-col item
          <span className="text-5xl mb-2"> </span>
          <p className="text-xs font-bold font-mo
          <p className="text-[10px] mt-1">Здесь 6
        </div>
      ) : (
        <div className="flex flex-col gap-1.5 w-f
          {servedHistory.map((item, idx) => (
            <div
              key={idx}
              className="w-full py-1.5 px-3 round
            0 flex items-center gap-2 text-[11px] text-

```

```

interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([
  const [currentNode, setCurrentNode] = useState<number
  const [speechText, setSpeechText] = useState<string>('
    'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
      буль!')
  });
  const [isTalking, setIsTalking] = useState<boolean>(f
  const [isBlinking, setIsBlinking] = useState<boolean>

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

```

```

    }
  });

  let currentX = 0;
  const paddingX = 70;
  const paddingY = 80;

  const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
      newNodes[u].x = 40 + currentX * paddingX;
      currentX++;
    } else {
      let leftX = -1;
      let rightX = -1;
      childKeys.forEach(v => {
        layoutDFS(v, depth + 1);
        if (leftX === -1) leftX = newNodes[v].x!;
        rightX = newNodes[v].x!;
      });
      newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX / 2;
      if (leftX === -1) currentX++;
    }
  };

  layoutDFS(0, 0);

  setNodes(newNodes);
  setSimulationStep(0);
  setBfsQueue([]);
  setCurrentNode(null);
  setSpeechText(
    `Отлично! Я построил базовое префиксное дерево (БПД) для строки S, чтобы расставить fail-ссылки!`
  );
};

```

```

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на гл
        очке суффикса просто нет. Жми «Следующий шаг»!'
    );
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полны
            ь полюбоваться таблицей переходов!'
        );
        return;
    }

    const r = bfsQueue[0];
    const newQueue = bfsQueue.slice(1);
    const updatedNodes = { ...nodes };
    const rNode = updatedNodes[r];
    const childrenIds = Object.values(rNode.children);

    let logs = `Рассматриваем вершину #${r} (${rNode.cha

    for (const [char, u] of Object.entries(rNode.children)
        newQueue.push(u);
        let v = rNode.fail;
        while (v !== 0 && updatedNodes[v].children[char]
            v = updatedNodes[v].fail;
        }
        const failTarget = updatedNodes[v].children[char]
        updatedNodes[u].fail = failTarget;
    }

```



```

onClick={() => buildInitialTrie(words)}
className="flex items-center gap-1 bg-slate
ransition-colors"
title="Сбросить автомат"
>
  <RotateCcw className="w-4 h-4" /> Сбросить
</button>
</div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
    <svg
      viewBox="0 0 200 200"
      className={`w-36 h-36 transition-transform
        isTalking ? 'scale-105 drop-shadow-[0_0_
        ]`}
    >
      {/* Тело лосося */}
      <ellipse cx="100" cy="100" rx="70" ry="45"
      <path d="M 40 85 Q 100 55 160 85 Q 100 115
      </path>

      {/* Хвост */}
      <path
        d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
        fill="#be123c"
        className="origin-[35px_100px] animate-
      />

      {/* Плавники */}
      <path d="M 90 55 L 70 30 L 110 55 Z" fill=
      <path d="M 90 145 L 70 170 L 110 145 Z" f
      <path

```

```

<div className="md:col-span-2 bg-slate-800 border-
  tween min-h-[140px]">
  {/* Треугольник баббля */}
  <div className="absolute -left-3 top-1/2 -tran
    slate-800 border-b-[12px] border-b-transpar

<div className="flex items-center space-x-2 t
  <Volume2 className={`w-4 h-4 ${isTalking ?
    <span>Прямое вещание из аквариума:</span>
</div>

<p className="text-slate-200 text-base leading
  {speechText}
</p>

<div className="mt-4 pt-3 border-t border-sla
  <div className="text-xs text-slate-400 flex
    <Info className="w-3.5 h-3.5 text-blue-400
    Статус: {simulationStep === 0 ? 'Бор ротон
    оен!'}
  </div>

  <div className="flex gap-2">
    {simulationStep === 0 && (
      <button
        onClick={startBFS}
        className="bg-emerald-600 hover:bg-em
        -1 shadow-lg shadow-emerald-900/40 transition
      >
        <Play className="w-4 h-4" /> Запустить
      </button>
    )}
    {simulationStep === 1 && (
      <button
        onClick={stepBFS}
        className="bg-blue-600 hover:bg-blue-5
        dow-lg shadow-blue-900/40 transition-transf
      >

```

```

        <polygon points="0 0, 6 3, 0 6" fill=
    </marker>
    <marker id="fail-arrow" markerWidth="6"
        <polygon points="0 0, 6 3, 0 6" fill=
    </marker>
</defs>

{/* Draw Edges */}
{Object.values(nodes).map(node => {
  if (node.id === 0) return null;
  const parent = nodes[node.parent];
  if (!parent.x || !parent.y || !node.x || !node.y) return null;

  const isCurrent = currentNode === node.id;

  return (
    <g key={`edge-${node.id}`}>
      <line
        x1={parent.x}
        y1={parent.y + 16}
        x2={node.x}
        y2={node.y - 16}
        stroke={isCurrent ? '#60a5fa' : '#ccc'}
        strokeWidth="2"
        markerEnd="url(#arrowhead)"
      />
      <text
        x={({parent.x + node.x} / 2 - 10)}
        y={({parent.y + node.y} / 2)}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
      >
        {node.char}
      </text>
    </g>
  );
})}

```

```

if (isCurrent) {
    fill = '#2563eb';
    stroke = '#60a5fa';
} else if (isInQueue) {
    fill = '#b45309';
    stroke = '#fbbf24';
} else if (node.is_terminal) {
    fill = '#059669';
    stroke = '#34d399';
}

return (
  <g key={`node-${node.id}`}>
    <circle
      cx={node.x}
      cy={node.y}
      r="16"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
    />
    <text
      x={node.x}
      y={node.y + 4}
      fill="white"
      fontSize={isRoot ? "10" : "12"}
      fontWeight="bold"
      textAnchor="middle"
    >
      {isRoot ? 'R' : node.char}
    </text>
    {!isRoot && (
      <text
        x={node.x + 12}
        y={node.y - 12}
        fill="#94a3b8"
        fontSize="9"
      >

```

```

    <form onSubmit={handleAddWord} className="flex flex-col gap-2" >
      <input
        type="text"
        value={newWord}
        onChange={(e) => setNewWord(e.target.value)}
        placeholder="НОВОЕ СЛОВО..."
        maxLength={8}
        className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
      />
      <button
        type="submit"
        className="bg-slate-700 hover:bg-slate-600 transition-colors"
      >
        <Plus className="w-3.5 h-3.5" /> Добавить
      </button>
    </form>
  </div>

```

```

  {/* Таблица Вершин и суффиксных ссылок */}
  <div className="lg:col-span-2 bg-slate-900/50 p-4" >
    <h5 className="text-white font-bold mb-3 flex items-center gap-1.5" >
      <span> </span> Таблица переходов и fail-ссылок
      <span className="text-xs text-slate-400 font-normal" >
        Всего вершин: {Object.keys(nodes).length}
      </span>
    </h5>

```

```

    <div className="max-h-[220px] overflow-y-auto" >
      <table className="w-full text-left border-collapse" >
        <thead>
          <tr className="border-b border-slate-700" >
            <th className="py-2 px-3" >ID Вершины</th>
            <th className="py-2 px-3" >Символ</th>
            <th className="py-2 px-3" >Родитель</th>

```

```

      <span className="bg-slate-800 b
        {node.char}
      </span>
    </td>
    <td className="py-2.5 px-3 text-s
      {node.id === 0 ? '-' : `#${node
    </td>
    <td className="py-2.5 px-3">
      {node.id === 0 ? (
        <span className="text-slate-5
      ) : (
        <span className="bg-indigo-95
          #{node.fail}
        </span>
      )}
    </td>
    <td className="py-2.5 px-3">
      {node.id === 0 || node.term_lin
        <span className="text-slate-5
      ) : (
        <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
          #{node.term_link}
        </span>
      )}
    </td>
    <td className="py-2.5 px-3 text-s
      {Object.keys(node.children).len
        ? 'ø'
        : Object.entries(node.children)
          .map(([c, id]) => `${c}→#
          .join(', ')}
    </td>
    <td className="py-2.5 px-3">
      {node.words.length === 0 ? (
        <span className="text-slate-6
      ) : (
        <span className="bg-emerald-9

```

```

function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};

  const CODE = [
    "build(v, l, r) {", // 1
    "  if (l === r) {", // 2
    "    tree[v] = A[l]; return", // 3
    "  }", // 4
    "  mid = (l + r) >> 1", // 5
    "  build(2v, l, mid)", // 6
    "  build(2v+1, mid+1, r)", // 7
    "  tree[v] = tree[2v]+tree[2v+1]", // 8
    "}", // 9
  ];
  void CODE;

```

```

function go(v: number, l: number, r: number) {
  if (l === r) {

```

```

        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v] });
        return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`, codeLine: 6, treeState: tree, highlightNodes: [v] });
    const left = go(2 * v, l, m, qL, qR);
    const right = go(2 * v + 1, m + 1, r, qL, qR);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}:`, codeLine: 8, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
    return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`, codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;
    // Build iterative tree: leaves at [n..2n-1]
    const tree: Record<number, number> = {};
    for (let i = 0; i < n; i++) tree[n + i] = arr[i];
    for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

    const steps: Step[] = [];
    let id = 0;
    let l = qL + n;
    let r = qR + n + 1; // half-open [l, r)
    let res = 0;

    steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

    while (l < r) {

```



```
const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
```

```
];
const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
```

```
];
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
  "    if (l & 1) res += tree[l++];",
  "    // -----",
  "    if (r & 1) res += tree[--r];",
  "    l >>= 1; r >>= 1;",
  "  }",
  "  return res; }"
```

```
];
```

// ===== PLAYER COMPONENT =====

```
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
```

```

const has = nd.val !== null;

let cls = 'bg-slate-800 border-slate-700 text-slate
if (isHigh) cls = `${clr.ring} text-white ring-2 sc
else if (isChild) cls = `${clr.childRing} text-ambe
else if (has) cls = `${clr.filled} text-slate-200`;

return (
  <div key={nd.v} className="flex flex-col items-center"
    <div className={`flex flex-col items-center justify
      ${cls}`}>
        <span className="text-[7px] opacity-60 font-mono"
        <span className="text-[9px] font-bold">[{nd.l
        <span className="text-xs font-extrabold font-mono"
      </div>
      {(nd.left || nd.right) && (
        <div className="flex flex-col items-center mt-2"
          <div className="w-px h-2 bg-slate-700" />
          <div className="flex justify-around w-full"
            {nd.left && renderNode(nd.left)}
            {nd.right && renderNode(nd.right)}
          </div>
        </div>
      )}
    </div>
  );
}, [step, clr]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
    /* Header */
    <div className={`px-4 py-2.5 flex items-center justify-between
      -emerald-950/50' : 'bg-amber-950/50'} border-b-2
      <h4 className="font-bold text-sm text-white flex-grow-1"
      <span className="text-[10px] font-mono text-slate-200"
    </div>

```

```

px]">
<span className={`mt-0.5 inline-block w-2 h-2 rounded`
<span className="leading-relaxed">{step.desc}</span>
{step.result !== undefined && {
  <span className={`ml-auto shrink-0 font-mono text-
    ' : color === 'emerald' ? 'text-emerald-300' :
    = {step.result}
  </span>
}}
</div>

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t
  <div className="flex items-center bg-slate-900
    <button onClick={() => { setPlaying(false); s
      unded" title="Сброс"><RotateCcw className="
    <button onClick={() => { setPlaying(false); s
      0 hover:text-white hover:bg-slate-800 rounded
    <button onClick={() => setPlaying(!playing)}
      white transition-all ${playing ? 'bg-rose-5
      {playing ? <Pause className="w-3 h-3" /> :
    </button>
    <button onClick={() => { setPlaying(false); s
      .5 text-slate-400 hover:text-white hover:bg
    </button>
  </div>
  <select value={speed} onChange={e => setSpeed(N
    text-[9px] font-bold rounded px-1.5 py-1 outl
    <option value={1200}>Медл.</option>
    <option value={700}>Норма</option>
    <option value={350}>Быстро</option>
    <option value={120}>Турбо</option>
  </select>
</div>

{/* Progress bar */}
<div className="h-1 bg-slate-950"><div className=
  + 1) / total) * 100 : 0}%` }} /></div>

```

```

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-
    {/* ---- TOP BAR: array editor + query range ---- */}
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        {/* Array input */}
        <form onSubmit={handleApply} className="flex-
          <div className="flex items-center justify-b
            <label className="text-[10px] font-bold up
              label>
                <button type="button" onClick={randomize}
              "><RefreshCw className="w-3.5 h-3.5" /></bu
            </div>
            <div className="flex gap-2">
              <input
                value={customInput}
                onChange={e => setCustomInput(e.target.v
                className="flex-1 bg-slate-900 border b
              one focus:border-indigo-500"
                placeholder="5, 8, 3, 12, 7, 2"
              />
              <button type="submit" className="bg-indigo
            ition-colors">Применить</button>
          </div>
          <div className="flex flex-wrap gap-1.5 pt-1
            {arr.map((v, i) => (
              <span key={i} className="bg-slate-900 b
                <span className="text-slate-500">A[{i
              </span>
            </span>
          </div>
        </div>
        {/* Query range */}

```

```

        -colors ${view === 'build' ? 'border-indigo-500
        -slate-200'}`}>
        <Hammer className="w-3.5 h-3.5" /> Построить
    </button>
    <button onClick={() => setView('queries')} class
        on-colors ${view === 'queries' ? 'border-emera
        er:text-slate-200'}`}>
        <Search className="w-3.5 h-3.5" /> Запрос: св
    </button>
    <button onClick={() => setView('theory')} class
        n-colors ${view === 'theory' ? 'border-blue-500
        te-200'}`}>
        <Info className="w-3.5 h-3.5" /> Почему / Спра
    </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
    <SimPanel
        key={`build-${arr.join('-')}`}
        title="Построение дерева (рекурсия)"
        icon=" "
        steps={buildSteps}
        code={BUILD_CODE}
        color="indigo"
        arr={arr}
    />
)}

{view === 'queries' && (
    <div className="grid grid-cols-1 xl:grid-cols-2"
        <SimPanel
            key={`qtd-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Сверху
            icon="↑ "
            steps={queryTDSteps}
            code={QUERY_TD_CODE}
            color="emerald"

```

```

<div className="grid grid-cols-1 xl:grid-cols-2" style={{width: 1000px; height: 1000px; border: 1px solid #ccc; padding: 10px; position: relative; background-color: #f0f0f0; border-radius: 10px; margin: 10px auto;}}
  <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%; background-color: #000000; color: #ffffff; border-radius: 10px; margin: 10px auto;}}
    <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md" style={{width: 100%; height: 100%; background-color: #008000; border: 2px solid #008000; border-radius: 10px; margin: 10px auto;}}
      <div style="text-align: center; padding: 10px 0 0 0;">
        † Запрос Сверху-вниз (Рекурсия)
      </div>
      <p style="text-align: center; margin: 10px 0 0 0;">
        Начинаем с корня. Если отрезок узла полон, то идем в левую ветку. Иначе спускаемся в обоих детей.
      </p>
      <div className="space-y-1.5 text-xs" style="margin-top: 10px;">
        <div className="flex justify-between border border-slate-950 padding: 5px" style="border: 1px solid #000000; margin-bottom: 5px;">
          <span className="text-rose-400 font-mono">0(
            <div className="flex justify-between border border-slate-950 padding: 5px" style="border: 1px solid #000000; margin-bottom: 5px;">
              <span className="text-rose-400 font-mono">>
              <div className="flex justify-between" style="width: 100%;">
                <span className="text-emerald-400 font-bold">д-400 font-bold">✓ легко</span></div>
              </div>
            </div>
          <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%; background-color: #000000; color: #ffffff; border-radius: 10px; margin: 10px auto;}}
            <div className="absolute -top-3 left-4 bg-amber-500 border border-amber-500 shadow-md" style={{width: 100%; height: 100%; background-color: #ffcc00; border: 2px solid #ffcc00; border-radius: 10px; margin: 10px auto;}}
              <div style="text-align: center; padding: 10px 0 0 0;">
                † Запрос Снизу-вверх (Итерация)
              </div>
              <p style="text-align: center; margin: 10px 0 0 0;">
                Стартуем с двух указателей (l и r) на левом и правом узлах. Идем к левому соседу. Поднимаемся пока l {<
              </p>
              <div className="space-y-1.5 text-xs" style="margin-top: 10px;">
                <div className="flex justify-between border border-slate-950 padding: 5px" style="border: 1px solid #000000; margin-bottom: 5px;">
                  <span className="text-emerald-400 font-mono">
                    <div className="flex justify-between border border-slate-950 padding: 5px" style="border: 1px solid #000000; margin-bottom: 5px;">
                      <span className="text-emerald-400 font-mono">>
                      <div className="flex justify-between" style="width: 100%;">
                        <span className="text-emerald-400 font-bold">00 font-bold">x сложно</span></div>
                      </div>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

const q = query.trim().toLowerCase();
const filtered = q ? chapters.filter((c) => c.title
const map = new Map<string, typeof chapters>();
for (const c of filtered) {
  const key = c.category?.trim() || "Прочие темы";
  if (!map.has(key)) map.set(key, []);
  (map.get(key) as typeof chapters).push(c);
}
return Array.from(map.entries());
}, [query]));

return (
  <aside className="w-full bg-slate-900/80 lg:bg-tran
    <div className="p-4 pb-3 shrink-0">
      <h3 className="text-xs font-bold text-slate-400
        <Compass className="w-4 h-4 text-indigo-400"
      </h3>
      <div className="relative">
        <Search className="w-4 h-4 text-slate-500 abs
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск по темам.."
          className="w-full bg-slate-800/70 border bo
          500 focus:outline-none focus:border-indigo-!
        />
        {query && (
          <button
            onClick={() => setQuery("")}
            className="absolute right-2 top-1/2 -tran
            aria-label="Очистить поиск"
          >
            <X className="w-3.5 h-3.5" />
          </button>
        )}
      </div>
    </div>
  </div>

```

```

        }}}
      </div>
    </div>
  }}}
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-300 to-indigo-400
] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex items-center">
    <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по нажатию на иконку
    симулятора.
  </p>
  <p className="text-slate-400 italic">Стрелки ← и → позволяют переключаться
  </p>
</div>
</aside>
);
};

```

ФАЙЛ 59/82: `src/components/Simulator.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  icon: string;
}

interface Person {
  id: string;

```



```
// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '=' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]);
  setPopMessage(" Стопка тарелок восстановлена.");
};
```

```
// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(`Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
  setHeapMessage(`Выбран самый "тяжелый" кандидат: ${best.text}`);
};

const resetHeap = () => {
  setHeap([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
    { id: '3', text: 'Кандидат: "Привет, я испанский язык"' }
  ]);
  setHeapMessage("Гипотезы восстановлены.");
};
```

```

<button
  onClick={() => setActiveTab('queue')}
  className={`flex items-center justify-between
    activeTab === 'queue'
      ? 'bg-indigo-600/20 border-indigo-500 text-white'
      : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'} font-size-4`}>
      <div className="text-left">
        <div className="font-bold text-sm text-white">FIFO
        <div className="text-xs opacity-80">FIFO
      </div>
    </div>
    <span className="text-xs bg-indigo-500/20 text-white">BFS
  </span>
</div>
</button>

<button
  onClick={() => setActiveTab('heap')}
  className={`flex items-center justify-between
    activeTab === 'heap'
      ? 'bg-emerald-600/20 border-emerald-500 text-white'
      : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
  <div className="flex items-center gap-3">
    <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'} font-size-4`}>
      <div className="text-left">
        <div className="font-bold text-sm text-white">Приоритетная
        <div className="text-xs opacity-80">Приоритетная
      </div>
    </div>
    <span className="text-xs bg-emerald-500/20 text-white">Beam Search
  </span>
</div>
</button>

```

```

        >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {popMessage && {
    <div className="bg-blue-950/60 border border-
    imate-fade-in">
      <span className="inline-block w-2 h-2 round
      {popMessage}
    </div>
  }}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
  >
    {stack.length === 0 ? {
      <div className="text-center py-12 text-sla
      <div className="text-5xl mb-3"> </div>
      <p className="text-base font-bold">Стор
      <p className="text-xs mt-1">Добавьте гр
    </div>
    } : {
      <div className="w-full max-w-sm flex flex
      <div className="absolute top-0 text-xs
      full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map(({plate, ind
      const isTop = index === 0;
      return {
        <div
          key={plate.id}
          className={`w-full flex items-cen

```

```

    /span>
  </h3>
  <p className="text-slate-400 text-xs mt-1">
    Кто первым пришел, тот первым получил суп
  </p>
</div>
<div className="flex flex-wrap items-center">
  <button
    onClick={addPerson}
    className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
  >
    <Plus className="w-4 h-4" /> Встать в очередь
  </button>
  <button
    onClick={servePerson}
    className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
  >
    Налить суп первому
  </button>
  <button
    onClick={resetQueue}
    title="Сбросить"
    className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
  >
    <RotateCcw className="w-5 h-5" />
  </button>
</div>
</div>

{dequeueMessage && (
  <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
    <span className="inline-block w-2 h-2 round
    {dequeueMessage}
  </div>

```

```

        Первый (Head)
        </span>
    }}
    <span className="text-4xl mb-2">{
    <span className={`font-bold text-
        {person.name}
    </span>
    <span className={`text-[10px] font
        isFirst ? 'bg-indigo-500/30 tex
    }`}>
        {isFirst ? 'Получает сун' : `В
    </span>
    </div>
    });
    }}}

    <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
        <Plus className="w-6 h-6 mb-1" />
        <span className="text-xs font-mono up
    </div>
    </div>
    })
    <div className="mt-6 text-xs text-slate-500
        ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
    </div>
    </div>
    </div>
    })

    { /* HEAP TAB */ }
    { activeTab === 'heap' && {
        <div className="space-y-6">
            <div className="bg-slate-800/80 p-5 rounded-x
                tify-between gap-4">
                <div>
                    <h3 className="text-lg font-bold text-whi
                    <span> Приоритетная очередь / Куча</sp

```

```

        />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emerald-500">
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e.target.value))}
        className="w-full accent-emerald-500 cursor-pointer"
      />
    </div>
    <div className="md:col-span-3">
      <button
        type="submit"
        disabled={!newCustomText.trim()}
        className="w-full flex items-center justify-center text-sm font-medium text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
      >
        <Plus className="w-4 h-4" /> Добавить в список
      </button>
    </div>
  </form>

  {heapMessage && (
    <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">
      <span className="inline-block w-2 h-2 rounded-full bg-emerald-500/40" />
      {heapMessage}
    </div>
  )}

  {/* Visualization */}

```

```

        {isTop && (
          <span className="text-[10px]
-0.5 rounded inline-block mt-1">
            Вершина Кучи (Root) • Буд
          </span>
        )}
      </div>
    </div>

    <div className="flex items-center
    /* Custom progress bar */>
      <div className="hidden sm:block
        <div
          className={`h-full rounded-
          style={{ width: `${percent}%
        ></div>
      </div>
      <span className={`font-mono font
        isTop ? 'text-emerald-400 tex
      }`>
        {percent}%
      </span>
    </div>
  </div>
    </div>
  );
  }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
  НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
  )}
</div>
);
};

```



```

        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q)
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
}, [query]));

return (
    <div>
        <div className="mb-6">
            <h2 className="text-xl sm:text-2xl font-extrabold">
                <p className="text-sm text-slate-400 leading-relaxed">
                    Все интерактивные демонстрации в одном месте.
                    здесь их можно открыть напрямую.
                </p>
            </div>

            <div className="relative mb-6 max-w-md">
                <Search className="w-4 h-4 text-slate-500 absolute">
                    <input
                        value={query}
                        onChange={(e) => setQuery(e.target.value)}
                        placeholder="Поиск тренажёра..."
                        className="w-full bg-slate-900 border border-
                            focus:outline-none focus:border-indigo-500"
                    />
                    {query && (
                        <button
                            onClick={() => setQuery("")}
                            className="absolute right-2 top-1/2 -translate-y-1/2"
                            aria-label="Очистить поиск"
                        >
                            <X className="w-4 h-4" />
                        </button>
                    )}
                </div>

                {items.length === 0 && <p className="text-slate-500">

```

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, RotateCw, Undo, } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно.
```

```
 * только после того, как узел окажется наверху (или по крайней мере в середине).
```

```
 */
```

```
export interface TNode {
```

```
  key: number;
```

```
  left: TNode | null;
```

```
  right: TNode | null;
```

```
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
```

```
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просечка (zig-zag-zig-zag).
```

```
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } =
```

```
{
```

```
  name: "Линия влево",
```

```
  hintCase: "Zig-Zig",
```

```
  target: 1,
```

```
  build: () => {
```

```
    const n1 = leaf(1);
```

```
    const n2: TNode = { key: 2, left: n1, right: leaf(3) }; // 2 - левый ребенок 1, 3 - правый
```

```
    const n4: TNode = { key: 4, left: n2, right: leaf(5) }; // 4 - левый ребенок 2, 5 - правый
```

```

    while (cur) {
        path.push(cur);
        if (key === cur.key) return path;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

```

```

    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2,
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  });
}

```

/* ----- КОМПОНЕНТ -----

```

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.build());
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; col: number; row: number }>([]);
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() => preset, [preset]);
  const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
  const expected = useMemo(() => (solved ? null : canonicalPath(target)), [target, solved]);

  const clickNode = (key: number) => {
    if (solved) return;
  }

```

```

<div className="flex gap-2 mb-3 overflow-x-auto" style={{ width: '100%' }}
  {PRESETS.map((p, i) => {
    <button
      key={p.name}
      onClick={() => reset(i)}
      className={`px-3 py-1.5 rounded-lg text-[11pt]
        ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-900"}
      `}
    >
      {p.name}
    </button>
  ))}
</div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-800" style={{ width: '100%' }}
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420) }}
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
        />
      )
    })}
  </svg>

```

```

        className="flex items-center gap-1.5 px-3 py-3
        -white disabled:opacity-40 text-xs font-bold"
      >
        <Undo2 className="w-3.5 h-3.5" /> Отменить
      </button>
      <button
        onClick={() => reset()}
        className="flex items-center gap-1.5 px-3 py-3
        text-xs font-bold transition-colors"
      >
        <RotateCcw className="w-3.5 h-3.5" /> Заново
      </button>
      <button
        onClick={() => setShowAnswer((s) => !s)}
        className={`flex items-center gap-1.5 px-3 py-3
        showAnswer
          ? "bg-amber-600/20 border-amber-500 text-amber-500"
          : "bg-slate-800 border-slate-700 text-slate-200"
        }`
      >
        {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
      </button>

      <span className="text-[11px] text-slate-500 ml-2" >
        поворотов: {moves.length} • минимум: {minimalMoves}
      </span>
    </div>

    {showAnswer && !solved && expected && (
      <div className="mt-3 text-[12px] bg-amber-950/40 p-3" >
        Сейчас splay крутил бы узел{" "}
        <span className="font-mono font-bold">{expected}
      </div>
    )}

    {solved && (
      <div

```

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
```

```
 *
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
```

```
 * * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОшаговый;
```

```
 * * это и есть доказательство, что поворот ничего не ломает;
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает;
```

```
 * * на кадре «результат» остаются призраки – пунктирные линии
```

```
 * * местах и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь
```

```
 * * идёт в своём темпе. Автопрокрутка включается кнопкой
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
  pos: Pos;
```

```
  edges: [NodeId, NodeId][];
```

```
  kind: "start" | "aim" | "result";
```

```
  title: string;
```

```
  text: string;
```

```
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
```

```
  focus?: [NodeId, NodeId];
```

```
  /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
```

```
  moved?: NodeId[];
```

```
  ms: number;
```

```
}
```

```
interface Case {
```

```
  key: "zig" | "zig-zig" | "zig-zag";
```

```

    },
    {
        ...ZIG_BEFORE,
        kind: "aim",
        title: "Прицел: крутим ребро 40 – 20",
        text: "Ничего пока не двигается. Смотри на подсве
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [160, 50], A: [80, 130], p: [245, 130],
        edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
        kind: "result",
        title: "Стало: 20 – корень",
        text: "40 стало правым сыном двадцатки. Поддерев
            ева от 40 – это одно и то же место.",
        moved: ["x", "p", "B"],
        ms: RESULT_MS,
    },
    ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
    edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
    pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
    edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",
    when: "x и p – дети с ОДНОЙ стороны (оба слева или об
    rotations: "2 поворота, первым – верхний",

```



```

pos: { x: [100, 45], A: [45, 118], p: [188, 118],
edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 в корне, бамбук стал кустом",
text: "Сравни с первым кадром: А и В были на глуб
        дерево, а не только достаёт ключ.",
moved: [{"x", "p", "A", "B"}],
ms: RESULT_MS,
    },
],
};

```

/* ----- Zig-Zag -----

```

const ZG_S0 = {
pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
key: "zig-zag",
label: "Zig-Zag",
when: "x и p – дети с РАЗНЫХ сторон (змейка)",
rotations: "2 поворота, первым – нижний",
keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
{
...ZG_S0,
kind: "start",
title: "Было: 60 → влево на 20 → вправо на 40",
text: "Узлы идут «змейкой»: сначала шаг влево, по
",

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```
const COLOR: Record<string, { fill: string; stroke: string }> = {
  x: { fill: "#6366f1", stroke: "#a5b4fc" },
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};
```

```
const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};
```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```
const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C }> = ({
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
  const ghosts = frame.kind === "result" ? (moved ?? []) : [];
```

```
  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" refX="10" refY="0">
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>
```

```
    { /* призраки: где узел стоял до поворота */
```

```
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
```

```

    });
  }}}

  {(Object.keys(pos) as NodeId[]).map((id) => {
    const p = pos[id];
    if (!p) return null;
    const sub = isSubtree(id);
    const c = COLOR[sub ? "sub" : id];
    const r = sub ? 15 : 20;
    const hot = Boolean(focus && focus.includes(id))
    return (
      <g
        key={id}
        style={{
          transform: `translate(${p[0]}px, ${p[1]}px)`,
          transition: MOVE,
          opacity: dim(id) ? 0.28 : 1,
        }}
      >
        {hot && <circle r={r + 7} fill="none" stroke="black" />}
        <circle r={r} fill={c.fill} stroke={c.stroke} />
        <text
          textAnchor="middle"
          dominantBaseline="central"
          fontSize={sub ? 12 : 15}
          fontWeight="700"
          fill={sub ? "#94a3b8" : "#fff"}
        >
          {sub ? id : keys[id]}
        </text>
        {!sub && (
          <text textAnchor="middle" y={-r - 7} font-size={10}>
            {id}
          </text>
        )}
      </g>
    );
  })}
}
```

```

        >
        {c.label}
      </button>
    )))
  </div>

<div className="mb-3 text-xs sm:text-[13px] text-slate-400">
  <span className="font-bold text-indigo-300">Корр.
  <span className="text-slate-500">({active.rotation})
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
<div className="mb-3 rounded-xl border border-slate-200">
  <div className="flex items-center gap-2 mb-1 flex-wrap">
    <span
      className={`text-[10px] font-bold uppercase
        current.kind === "aim"
          ? "bg-amber-500/20 text-amber-300"
          : current.kind === "start"
            ? "bg-slate-700 text-slate-300"
            : "bg-indigo-500/20 text-indigo-300"
        }`>
    >
    {current.kind === "aim" ? "прицел" : current.title}
  </span>
  <span className="text-sm font-bold text-white">
    Шаг {idx + 1} из {total}. {current.title}
  </span>
</div>
<p className="text-[13px] text-slate-400 leading-relaxed">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-200">
  <Tree frame={current} prev={prevPos} keys={activeKeys}>

    {/* обход слева направо – одинаковый на всех кадрах */}
    <div className="mt-2 pt-2 border-t border-slate-200">
      <div className="flex items-center gap-1.5 flex-wrap">

```

```

        d: hover: text-slate-300 text-xs font-bold transition-colors
      >
        ← Назад
    </button>
    <button
      onClick={() => {
        setPlaying(false);
        setFrame((f) => (f + 1) % total);
      }}
      className="px-4 py-2 rounded-lg bg-indigo-600
        w-indigo-900/40"
    >
      {last ? "⏮ Сначала" : "Дальше →"}
    </button>
    <button
      onClick={() => setPlaying((p) => !p)}
      className="flex items-center gap-1.5 px-3 py-2
        text-xs font-bold transition-colors"
    >
      {playing ? <Pause className="w-3.5 h-3.5" />
        : {playing ? "Стоп" : "Авто"}}
    </button>
    <button
      onClick={() => setSlow((s) => !s)}
      title="Автопрокрутка ещё медленнее"
      className={`flex items-center gap-1.5 px-3 py-2
        slow
        ? "bg-emerald-600/20 border-emerald-500 transition-colors"
        : "bg-slate-800 border-slate-700 text-slate-300"
      }`}
    >
      <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
    </button>
    <button
      onClick={() => {
        setFrame(0);
        setPlaying(false);
      }}

```

```

    key: number;
    left: SplayNode | null;
    right: SplayNode | null;
    x?: number;
    y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
    if (!root) return { key, left: null, right: null };
    if (key < root.key) {
        root.left = insertBST(root.left, key);
    } else if (key > root.key) {
        root.right = insertBST(root.right, key);
    }
    return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
    const y = x.left;
    if (!y) return x;
    x.left = y.right;
    y.right = x;
    return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
    const y = x.right;
    if (!y) return x;
    x.right = y.left;
    y.left = x;
    return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
    if (!node) return null;
    return {
        key: node.key,
        left: cloneTree(node.left),
        right: cloneTree(node.right),
    };
};

```

```

// For visualization logs, we do a bottom-up explanation
let path: { node: SplayNode; dir: "left" | "right" }
let current: SplayNode | null = root;

// Find the element or the element where search failed
let lastNode: SplayNode = root;
while (current) {
  lastNode = current;
  if (key === current.key) {
    path.push({ node: current, dir: null });
    break;
  } else if (key < current.key) {
    path.push({ node: current, dir: "left" });
    current = current.left;
  } else {
    path.push({ node: current, dir: "right" });
    current = current.right;
  }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
  steps.push(`Поиск ${key}: элемент отсутствует в дереве`);
  steps.push(
    `Поиск споткнулся на узле [${lastNode.key}]. Инициализируем подъем`
  );
} else {
  steps.push(
    `Поиск ${key}: элемент найден! Запускаем подъем`
  );
}

// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,

```

```

        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.right, target);
        node = leftRotate(node);
        steps.push(
            `Вращение Zag-Zag (Левый поворот родителя для правого ребенка)`
        );
    }

    if (!node.right) return node;
    steps.push(
        `Финальный поворот Zag (Левое вращение корня)`
    );
    return leftRotate(node);
}
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
    setHighlightedNode(key);
    const { newRoot, steps } = splayStepByStep(tree, key);
    if (newRoot) {
        setTree(newRoot);
    }
    setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;

    // Standard BST insert, then splay!
    let newTree = cloneTree(tree);
    newTree = insertBST(newTree, val);

```



```

    }
  };

  const drawTree = cloneTree(tree);
  computeCoordinates(drawTree, 300, 40, 140);

  // Render lines and nodes
  const renderLines = (node: SplayNode | null): React.ReactNode[] => {
    if (!node) return [];
    let lines: React.ReactNode[] = [];
    if (node.left && node.left.x && node.left.y) {
      lines.push(
        <line
          key={`l-${node.key}-${node.left.key}`}
          x1={node.x}
          y1={node.y}
          x2={node.left.x}
          y2={node.left.y}
          stroke="#475569"
          strokeWidth="2"
        />,
      );
      lines = lines.concat(renderLines(node.left));
    }
    if (node.right && node.right.x && node.right.y) {
      lines.push(
        <line
          key={`r-${node.key}-${node.right.key}`}
          x1={node.x}
          y1={node.y}
          x2={node.right.x}
          y2={node.right.y}
          stroke="#475569"
          strokeWidth="2"
        />,
      );
      lines = lines.concat(renderLines(node.right));
    }
  };

```

```

        circles = circles.concat(renderNodes(node.left));
        circles = circles.concat(renderNodes(node.right));
        return circles;
    };

    return (
      <div className="bg-slate-900 rounded-xl border border-
        { /* Header */ }
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <BookOpen className="w-5 h-5 text-indigo-400"
              Интерактивное Splay-дерево
            </h3>
            <p className="text-xs text-slate-400">
              Кликните на вершины дерева, чтобы «вытащить»
              Splay.
            </p>
          </div>

          <div className="grid grid-cols-1 lg:grid-cols-12
            { /* Playfield */ }
            <div className="lg:col-span-7 bg-[#0f172a] p-4
              <svg
                className="w-full h-full max-w-[600px] max-h-[320px]
                viewBox="0 0 600 320"
              >
                {renderLines(drawTree)}
                {renderNodes(drawTree)}
              </svg>

              { /* Quick interactive controls */ }
              <div className="absolute bottom-4 left-4 right-4
                <form onSubmit={handleFind} className="flex
                  <input
                    type="number"
                    placeholder="Поиск..."
                    value={inputValue}
                    onChange={(e) => setInputValue(e.target

```

```

    </div>
  </div>

  {/* Logs */}
  <div className="lg:col-span-5 bg-slate-950 border-1
    to overflow-y-auto custom-scrollbar">
    <h4 className="text-xs font-bold text-slate-400">
      Ход вращения (Splay-логи)
    </h4>
    <div className="flex-1 space-y-2 mb-4 overflow-y-auto">
      {log.map((step, idx) => (
        <div
          key={idx}
          className={`text-xs p-2.5 rounded transition
            idx === log.length - 1
              ? "border-l-2 border-indigo-500 bg-slate-900"
              : "text-slate-400 bg-slate-900/50"
            }`}
        >
          {step}
        </div>
      ))}
    </div>

    <div className="border-t border-slate-800 pt-3">
      <p>
        <span className="font-semibold text-slate-400">
          у самого корня. Делаем 1 вращение.
        </span>
      </p>
      <p>
        <span className="font-semibold text-slate-400">
          оба левые или оба правые. Вращаем деда, по
        </span>
      </p>
      <p>
        <span className="font-semibold text-slate-400">
          колено. Вращаем узел в разные стороны.
        </span>
      </p>
    </div>
  
```

```

<p className="font-bold text-slate-300 mb-2">
  2. Эффект качелей (Swing)
</p>
<p className="text-slate-400 leading-relaxed">
  Если агент запрашивает по очереди{" "}
  <code className="text-rose-400">[10]</code>
  <code className="text-rose-400">[60]</code>
  углы), дерево будет превращаться в палку
  сторону. Первая операция Splay(60) стоит
  <strong className="text-white">O(n)</strong>
  глубину n. Вторая Splay(10) заставляет
  Постоянный свинг убивает производительность
  сложности <strong className="text-white">O(n)</strong>
</p>
</div>
<div className="mt-3 text-[10px] text-rose-400">
  Кэшировать свингующие пары на концах — фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l" style="background-color: #1a2b3c; color: white; padding: 10px; border-radius: 10px 10px 0 0;">
  <div>
    <p className="font-bold text-slate-300 mb-2">
      3. Амортизация O(log n)
    </p>
    <p className="text-slate-400 leading-relaxed">
      Парадокс: один запрос за{" "}
      <code className="text-emerald-400">O(n)</code>
      всю структуру вдвое более плоской за счет
      при двойных вращениях {
      <code className="text-emerald-400 font-mono">O(n)</code>
      }. Дорогая операция инвестирует в дешевые
      операций. Поэтому амортизированное время
      <strong className="text-white">O(log n)</strong>.
    </p>
  </div>
  <div className="mt-3 text-[10px] text-emerald-400">
    Каждое "растяжение" компенсируется "склад

```

```
const [dirtyStack, setDirtyStack] = useState<Plate[]>(
  [
    { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
    { id: 'p3', name: 'Блюдо от торта', emoji: '🍰', state: 'idle' },
  ]
);
```

```
const [cleanRack, setCleanRack] = useState<Plate[]>(
  []
);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>(
  []
);
```

```
const addLog = (msg: string) => setLog(prev => [...prev, msg],
```

```
// PUSH: Положить грязную тарелку сверху стопки
```

```
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут ронзиться');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };
  setDirtyStack(prev => [...prev, newPlate]);
}, [isWashing, dirtyStack.length, addLog, setShake, preset]);
```

```
const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
counter++;
```

```
const newPlate: Plate = {
  id: Date.now().toString(),
  ...preset,
  isDirty: true,
  animState: 'in',
};
```

```
setDirtyStack(prev => [...prev, newPlate]);
```

```

        addLog(`    Готово! Чистая тарелка ${topPlate.emoji}
    }, 850);
}, [dirtyStack, isWashing]));

```

```

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝',
      State: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕',
      imState: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰',
      mState: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog(['    Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = dirtyStack.length - 1;

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950/40 p-4">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-tight">
          Ты складываешь грязные тарелки друг на друга.
          А когда приходит время мыть, ты берёшь именно последнюю.
          Попробуй вытащить нижнюю – всё рухнет!
        </p>
      </div>
    </div>
  </div>

  {/* УПРАВЛЕНИЕ */}

```

```
</div>
```

```
{/* Визуализация раковины */}
```

```
<div className="absolute top-4 right-4 flex items-center justify-center" style={{
  background-color: 'r-slate-800/80', text: '[10px] font-mono'}>
  <span className="w-1.5 h-1.5 rounded-full background-color: 'r-slate-500'" />
  <span>РАКОВИНА </span>
</div>
```

```
{/* Мыльные пузыри при мытье */}
```

```
{showSoap && (
  <div className="absolute inset-0 pointer-events-none" style={{
    background-color: 'r-slate-800/80', text: '[10px] font-mono'}>
    <div className="absolute w-6 h-6 rounded-full background-color: 'r-slate-500'" />
    <div className="absolute w-4 h-4 rounded-full background-color: 'r-slate-500'" />
    <div className="absolute w-5 h-5 rounded-full background-color: 'r-slate-500'" />
    <div className="absolute text-4xl anim-curve" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    </div>
  )}
```

```
{dirtyStack.length === 0 ? (
```

```
  <div className="flex-1 flex flex-col items-center justify-center" style={{
    background-color: 'r-slate-800/80', text: '[10px] font-mono'}>
    <span className="text-6xl mb-3 anim-pulse" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    <p className="text-white font-black text-2xl" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    <p className="text-slate-500 text-xs mt-1" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    </div>
```

```
) : (
```

```
  <div className={`flex-1 flex flex-col justify-center items-center`} style={{
```

```
    /* Указатель TOP */
```

```
    <div
```

```
      className="absolute right-8 flex items-center justify-center" style={{
        background-color: 'r-slate-800/80', text: '[10px] font-mono',
        style: {{ bottom: `${24 + topIndex * 44}px` }}
      >
```

```
        <span className="text-xs font-mono text-white" style={{
          background-color: 'r-slate-500', text: '[10px] font-mono'}>
        </span>
      </div>
    </div>
  </div>
  <div className="flex-1 flex flex-col justify-center items-center" style={{
    background-color: 'r-slate-800/80', text: '[10px] font-mono'}>
    <span className="text-6xl mb-3 anim-pulse" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    <p className="text-white font-black text-2xl" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    <p className="text-slate-500 text-xs mt-1" style={{
      background-color: 'r-slate-500', text: '[10px] font-mono'}>
    </div>
  )}
```

```

    { /* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */
    <div className="md:col-span-5 bg-slate-900 rounded-2xl" >
      <div className="absolute top-4 left-4 text-[10px] font-mono" >
        Сушилка для чистой посуды
      </div>

      <div className="absolute top-4 right-4 flex items-center justify-center gap-2" >
        <div className="order-emerald-800/80 text-[9px] font-mono" >
          <Sparkles className="w-3.5 h-3.5 animate-pulse" />
          <span>ЧИСТО </span>
        </div>

        <div className="flex-1 flex flex-col justify-between" >
          {cleanRack.length === 0 ? (
            <div className="flex-1 flex flex-col item-center justify-center" >
              <span className="text-5xl mb-2" > </span>
              <p className="text-xs font-bold font-mono" >Нет тарелок
              <p className="text-[10px] mt-1" >Здесь будет сушилка
            </div>
          ) : (
            <div className="flex flex-col gap-2 w-full" >
              {cleanRack.map(plate => (
                <div
                  key={plate.id}
                  className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-0.5" >
                    <div>
                      <span className="text-sm" > {plate.name}
                      <span className="text-[10px] text-emerald-400" > {plate.status}
                    </div>
                  </div>
                </div>
              )
            </div>
          )
        </div>

      </div>

    { /* Столешница сушилки */ }
    <div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-700" />
  </div>
}

```



```

-----
    for (let i = 1; i < n; i++) {
        let j = pi[i - 1];

        steps.push({ i, j, pi: [...pi], highlight: [i,
            ;

        while (j > 0 && s[i] !== s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
            яд назад: откат j = pi[`${j} - 1`] = `${pi[j] -
            j = pi[j - 1];
        }

        if (s[i] === s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
            ска увеличивается.` });
            j++;
        } else {
            steps.push({ i, j, pi: [...pi], highlight:
            овпадений нет.` });
        }

        pi[i] = j;
        steps.push({ i, j, pi: [...pi], desc: `Записывае

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

```

```

        }
    }
    return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
    const [mode, setMode] = useState<"kmp" | "z">("kmp");
    const [pattern, setPattern] = useState("aba");
    const [text, setText] = useState("abacaba");

    // Ensure inputs are valid dynamically
    const safePattern = pattern || " ";
    const safeText = text || " ";

    const { s, steps, patternLength } = useMemo(() => {
        if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
        else return generateZSteps(safePattern, safeText);
    }, [mode, safePattern, safeText]);

    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);
    const timer = useRef<NodeJS.Timeout | null>(null);

    // Reset when inputs or mode change
    useEffect(() => {
        setStepIdx(0);
        setAutoPlay(false);
    }, [s, mode]);

    // Timer loop for autoPlay
    useEffect(() => {
        if (autoPlay) {
            timer.current = setInterval(() => {
                setStepIdx(prev => {
                    if (prev >= steps.length - 1) { setAutoPlay(false); }
                    return prev + 1;
                });
            }, 1200);
        }
    }, [autoPlay, steps]);
}

```

```

        </div>
    </div>

    <div className="bg-slate-950 rounded-xl border-2 border-slate-900 px] flex items-center justify-start">
        <div className="flex gap-1.5 px-4 h-full">
            {s.split("").map((char, index) => {
                const isPattern = index < pattern.length;
                const isSeparator = index === pattern.length;

                let borderColor = "border-slate-900";
                let bgColor = "bg-slate-900";
                let textColor = isSeparator ? "text-slate-50" : "text-slate-400";

                // KMP Logic
                if (mode === "kmp") {
                    if (step.highlight?.include(char)) {
                        borderColor = "border-emerald-400";
                        bgColor = step.match === char ? "bg-emerald-400" : "bg-slate-900/40";
                    }
                }

                // Z Logic
                if (mode === "z") {
                    if (index >= step.l && index <= step.r && char === step.s.charAt(index)) {
                        bgColor = "bg-emerald-900/40";
                        borderColor = "border-emerald-400";
                    }
                    if (step.zTarget === index && char === step.s.charAt(index)) {
                        borderColor = "border-emerald-400";
                        bgColor = step.match === char ? "bg-emerald-400" : "bg-slate-900/40";
                    }
                }

                return (
                    <div key={index} className="flex flex-col items-center justify-center align-center gap-0.5">

```

```

                                <span className="z-finger" style={zFingerStyle} />
                            </div>
                        ))
                    { /* Z fingers */
                    {mode === "z" && step.i === 0}
                        <div className="abs" style={zFingerStyle} />
e z-20">
                                <span className="z-finger" style={zFingerStyle} />
                            </div>
                        ))
                    {mode === "z" && step.z === 0}
                        <div className="abs" style={zFingerStyle} />
                                <span className="z-finger" style={zFingerStyle} />
                            </div>
                        ))
                    {mode === "z" && step.z === 1}
                        <div className="abs" style={zFingerStyle} />
                                <span className="z-finger" style={zFingerStyle} />
                            </div>
                        ))
                    { /* Found flash effect */
                    {step.found && mode === "z"}
                        <div className="abs" style={zFingerStyle} />
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
                        </div>
                    {step.found && mode === "z"}
                        <div className="abs" style={zFingerStyle} />
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
                        </div>
                    </div>
                )
            }
        </div>
    </div>

<div className="flex flex-col sm:flex-row g

```

Обрати внимание на хитрость с решёт
Мы пишем **Шаблон + # + Текст**.
Когда значени {mode === 'kmp' ? "пр
о было найдено!

Пройди шаги вручную, посмотри, как
ля пропуска работы (в Z).

</p>

</div>

{/* Code Sneak Peek */}

<div className="bg-slate-900 border border-

<div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">

Код C++ ({mode === 'kmp' ? 'КМП

</div>

<pre className="p-4 text-xs font-mono te

{mode === 'kmp' ? `vector<int> pi(s.length());

for (int i = 1; i < s.length(); i++) {

int j = pi[i-1];

while (j > 0 && s[i] != s[j]) j = pi[j-1];

if (s[i] == s[j]) j++;

pi[i] = j;

}` : `int l = 0, r = 0;

for (int i = 1; i < n; i++) {

if (i <= r) z[i] = min(r - i + 1, z[i - l]);

while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]

if (i + z[i] - 1 > r) {

l = i;

r = i + z[i] - 1;

}

}`}

</pre>

</div>

</div>

);

}

```

    { u: 2, v: 3 },
    { u: 3, v: 4 },
  ];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
      });
    };

    recordStep(
      "Начало топологической сортировки. Используем кла

```

```

    for (let i = 0; i < 5; i++) {
      if (!visited.has(i)) {
        recordStep(
          `Запускаем новый обход DFS от нетронутой верши
          null,
        );
        dfs(i);
      }
    }

    recordStep(
      "Топологическая сортировка завершена! Полученный
      null,
    );
    setSteps(listSteps);
    setCurrentStepIdx(0);
  }, []));

const step = steps[currentStepIdx] || {
  desc: "Запуск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

```

```

        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
    { /* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] relative" >
        <svg className="w-full h-full max-w-[650px]" >
            <defs>
                <marker
                    id="dag-arrow"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
                </marker>
                <marker
                    id="dag-arrow-active"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
                </marker>
            </defs>

            { /* Edges */}

```



```

    } else if (isProcessed) {
        fill = "#022c22";
        stroke = "#059669";
        textColor = "text-emerald-300";
    } else if (isVisited) {
        fill = "#3730a3";
        stroke = "#6366f1";
        textColor = "text-indigo-200";
    }

    return (
        <g key={node.id}>
            <rect
                x={node.x - 55}
                y={node.y - 22}
                width="110"
                height="44"
                rx="8"
                fill={fill}
                stroke={stroke}
                strokeWidth="2"
                className="transition-all duration-!
            />
            <text
                x={node.x}
                y={node.y - 4}
                textAnchor="middle"
                fill="white"
                fontSize="11px"
                fontWeight="bold"
                className="pointer-events-none"
            >
                ({node.label}) {node.name.split(" "
            </text>
            <text
                x={node.x}
                y={node.y + 12}
                textAnchor="middle"

```

```

        getTopoListString()
    })
</div>
</div>

{/* Color key */}
<div className="space-y-1">
  <span className="text-[10px] text-slate-500">
    ВЕРШИНЫ В DFS:
  </span>
  <div className="flex items-center gap-2">
    <span className="w-3.5 h-3.5 rounded bg-slate-400">
      <span className="text-slate-400 text-[10px]">
        Серые (зашли)
      </span>
    </span>
  </div>
  <div className="flex items-center gap-2">
    <span className="w-3.5 h-3.5 rounded bg-slate-400">
      <span className="text-slate-400 text-[10px]">
        Черные (готово)
      </span>
    </span>
  </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-800 text-white"
    >
      Назад
    </button>
  </div>

```

*/

```
export interface Pair {
  key: number;
  grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
  id: NodeId;
  key: number;
  grade: number;
  left: TNode | null;
  right: TNode | null;
}

export const POINTS: Pair[] = [
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
  { key: 7, grade: 2 },
  { key: 12, grade: 0 },
  { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
  { key: 3, grade: 5 },
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 1, grade: -4 },
  { key: 7, grade: 2 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
```

```

    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `${t.key}
        о левое поддереву.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
    root: TNode | null;
    trace: { atId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; note: string };
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
    const trace: MergeResult["trace"] = [];
    const go = (a: TNode | null, b: TNode | null): TNode | null {
        if (!a || !b) {
            trace.push({ atId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: `
                уное целиком.` });
            return a ?? b;
        }
        if (a.grade > b.grade) {
            trace.push({ atId: a.id, bId: b.id, chosenId: a.id, note: `
                ем его правое поддереву с (${b.key}, ${b.grade})` });
            a.right = go(a.right, b);
            return a;
        }
        trace.push({ atId: a.id, bId: b.id, chosenId: b.id, note: `
            (${a.key}, ${a.grade}) с его левым поддеревом.` });
        b.left = go(a, b.left);
        return b;
    };
    return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): MergeResult {
    const trace: { atId: NodeId | null; note: string }[] = [];

```

```

    };
    walk(root, -Infinity, Infinity, null);
    return { bst, heap };
}

/* ----- раскладка сцены дерева -----

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
  };
  walk(root);
  return es;
};

interface TreeViewProps {
  root: TNode | null;
  width?: number;
  height?: number;
  nodeColor?: (id: NodeId) => string;
  nodeStroke?: (id: NodeId) => string;
}

```

```

    <circle r={pulse ? 24 : 20} fill={nodeColor}
    } strokeWidth={pulse ? 4 : 2} className="triangle"
    <text y={-2} fill="#fff" fontSize="13" fontStyle="italic">{label}</text>
    <text y={13} fill="#cbd5e1" fontSize="10">{label}</text>
  </g>
  );
  )))
</svg>
{subtitle && <p className="text-xs text-slate-400">{label}</p>
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 6);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];
  pending: Pair | null;
  interactive?: boolean;
  selected?: Pair | null;
  cursor?: { x: number; y: number } | null;
  badEdge?: { a: Pair; b: Pair } | null;
  onPointDown?: (p: Pair) => void;
  onPointUp?: (p: Pair) => void;
  onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}
const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,

```

```

    <line
      x1={PX(selected.key)} y1={PY(selected.grade)}
      stroke="#818cf8" strokeWidth={2.5} strokeDash
    />
  })

{points.map((p) => {
  const done = drawnEdges.some((e) => (e.a.key === p.key))
  const isSel = selected && selected.key === p.key
  return (
    <g
      key={`pp-${p.key}:${p.grade}`}
      className="transition-all duration-500"
      onPointerDown={interactive && onPointDown ? () : null}
      onPointerUp={interactive && onPointUp ? () : null}
      style={interactive ? { cursor: "pointer" } : null}
    >
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
        stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
        strokeDash={done ? [5, 5] : null}
      />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size="12"
        font-family="monospace">{p.key}</text>
    </g>
  );
});
}}}

{pending && (
  <g className="transition-all duration-500">
    <circle cx={PX(pending.key)} cy={PY(pending.grade)} r={10}
      stroke="#ccc" strokeDash={done ? [5, 5] : null}
    />
    <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-size="12"
      font-family="monospace">{pending.key}</text>
  </g>
)}
</svg>
);
};

```

```

const roots = all.filter((p) => !hasParent.has(gpid(p)
const ok = (n: Pair | undefined, lo: number, hi: number) => {
  if (!n) return true;
  if (n.key <= lo || n.key > hi) return false;
  const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
  const left = ch.filter((c) => c.key < n.key);
  const right = ch.filter((c) => c.key > n.key);
  if (left.length > 1 || right.length > 1) return false;
  return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
};
return roots.every((r) => ok(r, -Infinity, Infinity))
}

```

/** Полный вердикт по текущим линиям. */

```

export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade);
  const oneParent = new Set(edges.map((e) => gpid(e.b)));
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.key));
  const complete = edges.length === all.length - 1 && roots.length === 1;
  return { heap, oneParent, acyclic, bst, complete, win: true };
}

```

/** Попытка хода: почему нельзя – или ОК. */

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
  if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
  if (a.grade === b.grade) {
    return { ok: false, reason: "«!» равны – «кто выше?»" };
  }
  const parent = a.grade > b.grade ? a : b;
  const child = a.grade > b.grade ? b : a;
  if (parentOf(edges, gpid(child))) {
    return { ok: false, reason: `У точки (${child.key}) уже есть родительская линия.` };
  }
  if (reaches(edges, parent, child)) {
    return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
  }
  const next = [...edges, { a: parent, b: child }];
  if (!forestIsBst(next, POINTS)) {
    return { ok: false, reason: "Не является бинарным деревом" };
  }
  return { ok: true, reason: "ОК" };
}

```



```

-----
/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
  a: Pair;
  b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
  edges: GameEdge[],
  x: Pair,
  y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
  if (x.key === y.key && x.grade === y.grade) return { ok: false, reason: "такая линия уже проведена" };
  for (const e of edges) {
    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
        (pid(e.a) === pid(y) && pid(e.b) === pid(x)))
      return { ok: false, reason: "такая линия уже проведена" };
  }
  if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
  if (edges.some((e) => pid(e.b) === pid(child)))
    return { ok: false, reason: `у точки ${child.key} уже есть родитель` };
  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже имеет родителя)
  const childComp = new Set<string>([pid(child)]);
  let grew = true;
  while (grew) {
    grew = false;
    for (const e of edges) {
      if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a)))
        childComp.add(pid(e.a));
      grew = true;
    }
  }
  if (childComp.has(pid(parent))) return { ok: false, reason: "уже есть родитель" };
  return {
    ok: true,
    edge: { a: parent, b: child },
  };
}

```

```

const seen = new Set<string>();
const inorder: number[] = [];
const walk = (p: Pair): void => {
  if (seen.has(pid(p))) {
    bstOk = false; // цикл
    return;
  }
  seen.add(pid(p));
  const kids = children.get(pid(p)) ?? [];
  if (kids.length > 2) {
    bstOk = false;
    return;
  }
  const sorted = [...kids].sort((u, v) => u.key - v.key);
  let left: Pair | null = null;
  let right: Pair | null = null;
  if (sorted.length === 2) {
    if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
      bstOk = false; // двое детей с равным x или с
      return;
    }
    left = sorted[0];
    right = sorted[1];
  } else if (sorted.length === 1) {
    // одиночный ребёнок: сторона важна – определяет
    if (sorted[0].key <= p.key) left = sorted[0];
    else right = sorted[0];
  }
  if (left) walk(left);
  inorder.push(p.key);
  if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
  if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };

```

```

const [order, setOrder] = useState<ConnectOrder>("y-desc");
const [sortIdx, setSortIdx] = useState(0);
const [connIdx, setConnIdx] = useState(0);
const [playing, setPlaying] = useState(false);
const [sortCost, setSortCost] = useState<string | null>(null);

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
  const es: { a: Pair; b: Pair }[] = [];
  const walk = (node: TNode | null) => {
    if (!node) return;
    if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }; }
    if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }; }
  };
  walk(userTree);
  return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].filter(e => e.a.group < e.b.group));

useEffect(() => {
  if (!playing || phase === "game") return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.length : canonEdges.length / 2;
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 1000);
}, [playing, phase, algo, sortIdx, connIdx, order]);

```

```

const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
    window.setTimeout(() => setBadEdge(null), 900);
  }
};

const pointDown = (p: Pair) => {
  if (phase !== "game") return;
  if (!sel) {
    setSel(p);
    setGameMsg(`Выбрана (${p.key}; ${p.grade}) – тене
  } else if (sel.key === p.key && sel.grade === p.grade) {
    setSel(null);
    setGameMsg(null);
  } else {
    attemptConnect(sel, p);
    setSel(null);
  }
};

const pointUp = (p: Pair) => {
  if (phase !== "game" || !sel) return;
  if (sel.key === p.key && sel.grade === p.grade) return;
  attemptConnect(sel, p);
  setSel(null);
  setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
  if (!sel) return;

```

```

    })
    {parsed.errors.length === 0 && (parsed.equalX |
      <div className="mt-2 text-xs text-amber-300 b
        {parsed.equalY && <div>✗ Есть <b>равные y</b>
          кто <b>раньше в списке</b> (сортировка ста
        {parsed.equalX && <div>✗ Есть <b>равные x</b>
          de.key → left).</div>}
      </div>
    })
  </div>

  {/* Координатная плоскость – НАВЕРХУ */}
  <div className="bg-emerald-950/20 rounded-xl p-4
    <div className="flex flex-wrap items-center jus
      <h4 className="text-base font-bold text-emera
        <div className="flex items-center gap-2">
          {phase === "sort" ? {
            <div>
              <span className="text-xs text-slate-400
                {[ "quicksort", "counting" ] as const}.m
              <button
                key={a}
                onClick={() => { setAlgo(a); setSort
                  Math.round(n * Math.log2(n)) : 0} сравнений
                disabled={n < 2}
                className={`px-3 py-1.5 rounded-lg
                  600 text-white" : "bg-slate-700 text-slate-
                >
                  {a === "quicksort" ? "quicksort (сп
                </button>
              )}}
            </div>
          ) : {
            <div>
              <span className="text-xs text-slate-400
              <button
                onClick={() => startConnect("y-desc")}
                className={`px-3 py-1.5 rounded-lg te

```

```

        <>Ввод приходит <b>как попало</b> ({n} па
ть</b>.</>
    ) : sortIdx < 3 ? (
        algo === "quicksort" ? (
            <>✂ quicksort: счётчик сравнений: <b cl
{comparisons}</b> (n·log2n).</>
        ) : (
            <> counting: точки в корзины по значени
, потому что у – маленькие целые.</>
        )
    ) : (
        <>Отсортировано ({sortCost ?? "способ не
<b>Игру    </b></>
    )
) : phase === "game" ? (
    gameWon ? (
        <> Готово! {hintUsed ? "Решение подсказа
ево: линии вниз по у, обход по х отсортиров
    ) : gameMsg ? (
        <>△ {gameMsg}</>
    ) : (
        <>Кликни точку-родителя (выше по «!»), за
    )
) : connDone ? (
    <> Проведено {connEdges.length} линий в по
</b>: точки прибиты, дерево одно. А теперь
) : (
    <>Линий проведено: {connIdx} из {connEdgesA
    )}
</div>
</div>

```

```

/* Чек-лист инвариантов */
<div className={`rounded-xl p-4 border transition
er-slate-700 bg-slate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    {phase === "game" ? (

```


14

```

/* ===== РЕЖИМ MERGE ===== */

```

```
const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const { l, r } = useMemo(() => splitTree(buildTree(P0), P1), [P0, P1]);
  const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
  const steps = merged.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;
  const cur = steps[idx];

  return (
    <div className="space-y-4">
      <div className="flex items-center gap-3">
        <button onClick={() => setStepIdx(Math.max(0, idx - 1))} className="bg-slate-700 text-white disabled:opacity-30">⏮
        <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))} className="font-bold bg-indigo-600 text-white disabled:opacity-30">⏭
        <span className="text-xs text-slate-400">mar {idx}
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
          <h5 className="text-sm font-bold text-sky-300">Left
          <TreeView root={l} width={480} height={220} nodeSize={16} />
        </div>
        <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
          <h5 className="text-sm font-bold text-rose-300">Right
          <TreeView root={r} width={480} height={220} nodeSize={16} />
        </div>
      </div>

      <div className="text-xs text-slate-300 bg-slate-900 p-4">
        {cur?.chosenId ? (
          <div>Ⓜ {cur.note}</div>
        ) : (

```



```

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mo
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4"
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только вер
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4"
        <h4 className="text-sm font-bold text-slate-300">
        <TreeView root={res.root} subtitle="Валидность"
        <ul className="mt-2 space-y-1 text-sm">
          <li> обход слева-направо по-прежнему сортир
          <li> каждое ребро вниз по y (куча целая) —
        </ul>
      </div>
    )}

    <div className="text-xs text-slate-300 bg-slate-900/40 p-2">
      {steps[idx]?.note}
    </div>

```

```

{pick === "heap" ? {
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-slate-300" >
      4>
    <div className="flex gap-2 overflow-x-auto mb-2" >
      {heapArr.slice(1).map((v, i) => {
        <div key={i} className="px-3 py-2 rounded" >
          <div className="text-slate-500 text-[10px]" >
            <div className="text-white font-bold">{v}</div>
            <div className="text-emerald-400 text-[10px]" >
              {i}</div>
          </div>
        </div>
      })}
    </div>
    <p className="text-xs text-slate-400">Полное  $2i/2i+1$  попадает в узел.</p>
  </div>
) : {
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-rose-300" >
      >
    <TreeView root={full} subtitle="Попробуй «посмотреть»" />
    <div className="flex gap-2 overflow-x-auto mt-2" >
      {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
        const id = slots[i];
        return (
          <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-900" >
            <div className="text-slate-500 text-[10px]" >
              {id ? <div className="text-white font-bold">{v}</div> : v}
            </div>
          </div>
        );
      })}
    </div>
    <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
  </div>
}
}

```

```

        нашли за ${steps.length + 1} сравнение(й).` }));
        if (sortedByKey[mid].key < target) { steps.push({
            ` } — вся левая половина выброшена.` })); lo = mid
        else { steps.push({ lo, hi, mid, found: false, no
            ` })); hi = mid - 1; }
    }
    return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
    <div className="space-y-4">
        <div className="flex gap-2">
            {stage < 2 && (
                <button onClick={() => { setStage(stage === 0
                    digo-600 text-white">
                        {stage === 0 ? "Найти y для x=7 бинарным по
                    </button>
                )}
            {stage === 2 && (
                <button onClick={() => setBidx(Math.min(bsStep
                    -1.5 rounded-lg text-xs font-bold bg-indigo
                    Шаг поиска →
                </button>
            )}
            <button onClick={() => { setStage(0); setBidx(0
                Сброс</button>
        </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
    {(stage === 2 ? sortedByKey : shuffled).map((p,
        const highlight = stage === 1 && i === wrongMid
        const inBs = stage === 2 && (() => { const s
        const isMid = stage === 2 && (() => { const s
        const isFound = stage === 2 && bidx >= bsStep
        return (

```



```

        <text x={PX(p.key)} y={PY(p.grade) + 24}
      </text>
    </g>
  );
  }}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
  ate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    <li>{v.heap ? " " : " "} каждое ребро – вниз
    <li>{v.oneParent ? " " : " "} у каждой точки
    <li>{v.acyclic ? " " : " "} циклов нет</li>
    <li>{v.bst ? " " : " "} обход слева-направо –
    <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
  </ul>
  {win && {
    <p className="mt-3 text-sm font-bold text-еме
      ения просто не существует.</p>
  }}
</div>

  <div className="text-xs text-slate-300 bg-slate-9
</div>
  );
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge  " },

```

```
        {tab === "merge" && <MergeMode />}
        {tab === "erase" && <EraseMode />}
        {tab === "layers" && <LayersMode />}
        {tab === "search" && <SearchMode />}
        {tab === "game" && <GameMode />}
    </div>
  );
};
```

ФАЙЛ 68/82: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
```

```
/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветитс
    Component: SparseTableViz,
  },
  treap: {
```

```

    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем",
    Component: () => (
      <>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
      </>
    ),
  },
  "bellman-ford": {
    title: "Форд-Беллман",
    Component: () => (
      <>
        <ChapterImage vizType="bellman-ford" />
        <BellmanFordViz />
      </>
    ),
  },
  floyd: {
    title: "Флойд-Уоршелл",
    Component: () => (
      <>
        <ChapterImage vizType="floyd" />
        <FloydViz />
      </>
    ),
  },
  "johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
  "mst-kruskal": { title: "Краскал и DSU", Component: () => (
  "mst-prima": { title: "Прим", Component: () => (
  "mst-boruvka": { title: "Борувка", Component: () => (
  "string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
  "string-z-func": { title: "Z-функция", Component: () => (
  "aho-corasick": {
    title: "Ахо-Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">

```



```

const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'Ø', x: 400, y: 60, depth: 0,
    // Ветка H -> E -> R -> S
    1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,
      2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,
        3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
          4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4,
            // Ветка H -> I -> S
            5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2,
              6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3,
                // Ветка S -> H -> E
                7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1,
                  8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2,
                    9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3,
                      = 2 (HE)
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>('');

```

```

const char = textToScan[currentIndex];
let curr = currentNode;
let logMsg = `[Символ '${char}']`;

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
  const nextNode = TRIE_TRANSITIONS[curr][char];
  logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}).`;
  setJumpPath({ from: curr, to: nextNode, type: 'normal' });
  curr = nextNode;
  setActiveSearchCodeLine(3);
} else {
  setActiveSearchCodeLine(2);
  let jumps = 0;
  while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
    curr = WATERFALL_NODES[curr].fail;
    jumps++;
  }
  const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
  if (originalCurr === 0) {
    logMsg += `В корне нет перехода по '${char}'. Лист`;
    setJumpPath(null);
  } else {
    logMsg += `Поток по '${char}' у вершины #${originalCurr}`;
    logMsg += `рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}).`;
    setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
  }
  curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

```

```

        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -  

            просто не существует. fail[H] автоматически нап
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 7,
        title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
        desc: 'Обрабатываем дочернюю вершину #7 (S) на De
            ,
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 2,
        title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
        desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
            а-разведчик плывет в ROOT и ищет ветку по "E".
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'E',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
            H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
        codeLine: 3,
    }
}

```

```

    },
    {
      targetNodeId: 9,
      title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С',
      desc: '  МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
        ба плывет в #1 (H) и ищет ветку "E". У вершины
      codeLine: 4,
      scoutPos: 1,
      inspectedParentFail: 1,
      checkTargetChar: 'E',
      checkingBranchesFrom: 1,
    },
    {
      targetNodeId: 4,
      title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
      desc: 'Финальная вершина #4 (HERS) на Depth 4. По
        #7 (S). fail[HERS] = #7 (S). Автомат полностью
      codeLine: 4,
      scoutPos: 0,
      inspectedParentFail: 0,
      checkTargetChar: 'S',
      checkingBranchesFrom: 0,
    },
  ],
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNodeId];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border" >
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center space-x-3" >
        <span className="text-3xl" > </span>
        <div>
          <h4 className="text-2xl font-bold text-white" >
            Анимация водопада: Полное обучение автомата

```

```

    { /* Управление шагами обучения */ }
    <div className="bg-slate-900/90 p-5 rounded-x
    <div>
      <h5 className="text-indigo-300 font-bold
        <span> </span> Полный обход в ширину (B
      </h5>
      <p className="text-xs text-slate-300 mb-3
        Показаны все шаги очереди BFS (начиная
      ar 8 (SHE → HE)</strong>, где наглядно виде
      </p>
      <div className="space-y-1.5 mb-6 overflow
        {TRAINING_STEPS_INFO.map((step, idx) =>
          <button
            key={idx}
            onClick={() => setTrainStep(idx)}
            className={`w-full text-left px-3 p
      ${
        idx === trainStep
          ? 'bg-indigo-600 text-white fon
          : 'bg-slate-800 text-slate-400
        }`}
      >
        <span className="line-clamp-1">{step
        {idx === trainStep && <span classNa
      </button>
      }}}
    </div>
  </div>

  <div className="flex gap-2">
    <button
      onClick={() => setTrainStep((prev) => M
      disabled={trainStep === 0}
      className="flex-1 bg-slate-700 hover:bg
ion-all"
    >
      Назад
    </button>

```

```

        l-4 border-amber-500 text-amber-300 font-bo
            <span>3. while (v !== root && trie[v]
                {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
r-l-4 border-emerald-500 text-emerald-300 f
            <span>4. if (trie[v][char] !== undefin
/span>
                {currentTrainInfo.codeLine === 4 && <
ал IF!</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
-4 border-blue-500 text-blue-300 font-bold'
            <span>5. else fail[u] = root; // Bero
                {currentTrainInfo.codeLine === 5 && <
root</span>}
            </div>
        </div>
    </div>
    <div>
        <h5 className="text-slate-200 font-bold ml
            <Lightbulb className="w-4 h-4 text-ambe
        </h5>
        <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
            {currentTrainInfo.desc}
        </div>
    </div>
</div>
</div>
) : (
    /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
    <div className="grid grid-cols-1 lg:grid-cols-3
        <div className="bg-slate-900/60 p-5 rounded-x
            <div>
                <h5 className="text-white font-bold mb-2

```

```

    </div>

    <button
      onClick={stepSearchSimulation}
      disabled={isSearchDone || textToScan.length === 0}
      className={`w-full py-2.5 rounded-lg font-medium transition-colors
        ${isSearchDone || textToScan.length === 0
          ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
          : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
      `}
    >
      <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}</span>
      <ArrowRight className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
  <div>
    <h5 className="text-white font-bold mb-3">Текущее действие в коде</h5>
    <div className="bg-slate-950 p-4 rounded-2xl">
      <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
        <span>1. curr = state; char = text[i];
        {activeSearchCodeLine === 1 && <span>инициализация</span>}
      </div>
      <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold`} >
        <span>2. while (curr !== root && !trie[curr].next)
        {activeSearchCodeLine === 2 && <span>прыжок по fail!</span>}
      </div>
      <div className={`p-1.5 rounded flex items-center border border-blue-400 text-blue-200 font-bold`} >

```

```

<div className="flex items-center justify-between" style={{width: 1000px}}>
  <h5 className="text-white font-bold text-base">
    <span> </span> Ветвящийся синий водопад (Борис)
  </h5>
  <div className="flex flex-wrap gap-4 text-xs">
    <span className="flex items-center gap-1 text-white">
      <span className="w-3 h-0.5 bg-blue-500 inline-block"></span>
    </span>
    <span className="flex items-center gap-1 text-white">
      <span className="w-3 h-0.5 border-t border-blue-500 inline-block"></span>
    </span>
  </div>
</div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
  <svg viewBox="0 0 800 520" className="w-full h-full">

    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' },
    ]}.map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="14" height="24"

```



```

        y={{fromNode.y + toNode.y} / 2 +
        fill={isTrainingExamined ? (isMatch
        fontSize="12"
        fontWeight="bold"
        textAnchor="middle"
    >
        {char}
    </text>

    {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
    {isTrainingExamined && (
        <g transform={`translate(${(fromNode
        .y) / 2})`}>
            <rect x="-15" y="-12" width="30
strokeWidth="2" />
            <text x="0" y="5" fontSize="14"
                {isMatchBranch ? ' ' : ' '}
            </text>
        </g>
    )}
    </g>
    );
    });
    }}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
    if (node.id === 0) return null;
    const targetNode = WATERFALL_NODES[node.fromNode];

```

```

    // В режиме обучения показываем fail-ссылку
    // Найдём индекс шага, на котором обработали
    const stepIdxForNode = TRAINING_STEPS_INFO[node.id];
    if (activeMode === 'training' && trainStep === stepIdxForNode)

```

```

        let isHighlighted = activeMode === 'search' || activeMode === 'fail';
        let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
    }
    return (

```

```

    {isCurrent && {
      <circle cx={node.x} cy={node.y} r="10" fill="black" />
    }}
    {isTargetChild && {
      <circle cx={node.x} cy={node.y} r="10" fill="white" stroke="black" />
    }}

    {/* Основной круг вершины */}
    <circle
      cx={node.x}
      cy={node.y}
      r={isCurrent || isTargetChild ? '24' : '10'}
      fill={isCurrent ? '#2563eb' : isTargetChild ? 'white' : 'black'}
      stroke={isCurrent ? 'black' : isTargetChild ? 'black' : 'white'}
      strokeWidth={isCurrent || isTargetChild ? '2' : '1'}
      className="transition-all duration-300"
    />

    {/* Текст внутри вершины */}
    <text
      x={node.x}
      y={node.y + 5}
      fill="white"
      fontSize={isCurrent || isTargetChild ? '12' : '10'}
      fontWeight="bold"
      textAnchor="middle"
    >
      {node.label}
    </text>

    {/* Метка искомой вершины в режиме обхода */}
    {isTargetChild && {
      <g transform={`translate(${node.x}, ${node.y})`} >
        <rect x="-35" y="-12" width="70" height="24" fill="white" stroke="black" />
        <text x="0" y="2" fontSize="10" fill="white" >
          ИЩЕМ FAIL
        </text>
      </g>
    }}
  </div>
</pre>

```

```

        </text>
      </g>

    </svg>
  </div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
  <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{backgroundImage: 'url(/img/gradient.svg)'}}>
    <h5 className="text-white font-bold mb-3 text-sm">
      <span> </span> Улов лосося (Найденные слова)
    </h5>
    {foundMatches.length === 0 ? (
      <p className="text-sm text-slate-400 italic">Пока ничего не поймали. Запустите шаги симуляции</p>
    ) : (
      <div className="flex flex-wrap gap-2">
        {foundMatches.map((match, idx) => (
          <div
            key={idx}
            className="bg-emerald-950 border border-emerald-500 text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
            style={{backgroundImage: 'url(/img/gradient.svg)'}}
          >
            <CheckCircle2 className="w-4 h-4 text-emerald-500"/>
            <span>{match.word}</span>
            <span className="text-[10px] bg-emerald-950 px-2">
              Индекс {match.index}
            </span>
          </div>
        ))}
      </div>
    )}
  </div>
)}
</div>
);

```

```

const [mode, setMode] = useState<"1d" | "2d">("1d");
return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
      {(
        [
          ["1d", "1. Одномерные суммы"],
          ["2d", "2. Двумерные суммы"],
        ] as const
      ).map(([key, label]) => (
        <button
          key={key}
          onClick={() => setMode(key)}
          className={`px-3 py-2 rounded-lg text-xs sm:text-sm
            mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
          }`}
        >
          {label}
        </button>
      ))}
    </div>
    {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
  </div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]], []),
    []
  );
};

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]

```

const [hover, setHover] = useState<{ kind: "pref" | "el"; i: number}>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number}>({ l: 0, r: 0 });

```

```

const covered =

```

```

        : "bg-slate-800 text-slate-300"
      }` }
      title={`a[${i}] = ${v}`}
    >
      {v}
    </div>
  );
}
}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right">
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" &&
      const isL = i === range.l;
      const isR = i === range.r + 1;
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind:
          onMouseLeave={() => setHover(null)}
          className={`${cellBase} cursor-help $
            active
              ? "bg-emerald-500 text-white ring
              : isR
                ? "bg-emerald-700 text-white"
                : isL
                  ? "bg-rose-700 text-white"
                  : "bg-slate-900 text-slate-300"
            }` }
            title={`P[${i}] = сумма a[0..${i} - 1]}
          >
            {v}
          </div>
        );
      )
    })}
  </div>

```

```

    {/* Запрос */}
    <div className="bg-slate-900 border border-indigo
      <div className="text-xs text-slate-400 mb-2">
        Запрос суммы на отрезке (кликайте по массиву
      </div>
      <p className="font-mono text-sm sm:text-base te
        sum(a[{range.l}..{range.r}]) = P[{range.r + 1
        <span className="text-emerald-400">{pref[range.l
        <span className="text-rose-400">{pref[range.l
        <span className="text-indigo-300 font-bold">{
      </p>
      <p className="text-[11px] text-slate-500 mt-1">
        Один вычитание вместо цикла: запрос за O(1) п
      </p>
    </div>
  </div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i
      }
    }
    return p;
  }, [n, m]);

  const [hover, setHover] = useState<{ i: number; j: num
  const [rect, setRect] = useState({ r1: 1, c1: 1, r2:

```

```

        </div>
      );
    }
  }
</div>
  )
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    <div className="inline-block">
      {P.map((row, i) => {
        <div key={i} className="flex gap-1.5 mb-1">
          {row.map((v, j) => {
            const corner = cornerOf(i, j);
            const isHover = hover?.i === i && hover?.j === j;
            const cornerColor =
              corner === "A"
                ? "bg-emerald-600 text-white"
                : corner === "D"
                ? "bg-sky-600 text-white"
                : corner === "C"
                ? "bg-rose-600 text-white"
                : "bg-slate-900 border border-slate-700";
            return (
              <div
                key={j}
                onMouseEnter={() => setHover({ i, j })}
                onMouseLeave={() => setHover(null)}
                className={` ${cellBase} cursor-pointer ${
                  isHover ? "bg-amber-500 text-white" : ""
                }`}
              >
                {v}
                {corner && !isHover && (
                  <span className="absolute -top-1.5 -left-1.5 font-size-0.8">
                    ht">

```

ФАЙЛ 71/82: src/components/visualizers/SparseTableViz.t

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
  }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
```



```

}

export const SparseTableViz: React.FC = () => {
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d")

  return (
    <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
      <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
        <button
          onClick={() => setTab("1d")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
        >
          1. Сворачивание 1D
        </button>
        <button
          onClick={() => setTab("2d_build")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
        >
          2. Сворачивание 2D (Построение)
        </button>
        <button
          onClick={() => setTab("2d_query")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
        >
          3. Запрос 2D (Формула)
        </button>
      </div>

      <AnimatePresence mode="wait">
        {tab === "1d" ? (
          <Viz1D key="1d" />
        ) : tab === "2d_build" ? (
          <Viz2DBuild key="2d_build" />
        ) : (
          <Viz2DQuery key="2d_query" />
        )}
      </AnimatePresence>
    </div>
  )
}

```

```

        <ChevronLeft size={20} />
      </button>
      <button
        onClick={() => setStep(Math.min(maxStep, step + 1))}
        className="p-2 bg-indigo-600 hover:bg-indigo-500"
        disabled={step === maxStep}
      >
        <ChevronRight size={20} />
      </button>
      <button
        onClick={() => setStep(0)}
        className="p-2 bg-slate-800 hover:bg-slate-700"
      >
        <RotateCcw size={20} />
      </button>
    </div>
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <div className="text-[11px] uppercase tracking-wider" >
      <div className="flex gap-1 sm:gap-1.5 overflow-x-auto" >
        {arr1D.map((v, idx) => {
          const inCover = !!covered && idx >= covered - 1;
          return (
            <div key={idx} className="flex flex-col items-center justify-center" >
              <div
                className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em)]}
                ${inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-400"}
              </div>
              <span className="text-[9px] text-slate-400">{v}</span>
            </div>
          );
        })}
      </div>
    </div>
  </div>

```

```
<div className="text-center font-bold text-  
rap">
```

```
  2^2 (L=4)
```

```
</div>
```

```
<div className="text-center font-bold text-  
rap">
```

```
  2^3 (L=8)
```

```
</div>
```

```
{/* Matrix */}
```

```
{Array.from({ length: N }).map((_, i) => {
```

```
  <React.Fragment key={i}>
```

```
    <div className="text-slate-500 font-mon
```

```
      {i}
```

```
    </div>
```

```
    {Array.from({ length: LOG }).map((_, j)
```

```
      const isValid = i + (1 << j) <= N;
```

```
      const isVisible =
```

```
        isValid &&
```

```
        (j === 0 ||
```

```
          computeSteps.findIndex((s) => s.i  
            step));
```

```
    // Active computing state
```

```
    let isActive = false;
```

```
    let isSrc1 = false;
```

```
    let isSrc2 = false;
```

```
    if (step > 0 && step <= maxStep) {
```

```
      const currentStep = computeSteps[ste
```

```
      if (currentStep.i === i && currentS
```

```
        isActive = true;
```

```
      if (currentStep.j === j + 1 && curre
```

```
        isSrc1 = true;
```

```
      if (
```

```
        currentStep.j === j + 1 &&
```

```
        currentStep.i === i - (1 << (curre
```

```
      )
```

```

        >
          {st1D[i][j]}
        </motion.div>
      }}

      {/* Arrows visualization */}
      {(isSrc1 || isSrc2) && (
        <motion.svg
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          className="absolute pointer-e
          style={{
            width: 100,
            height: isSrc2 ? 40 + (1 <<
            right: -50,
            top: 24,
            overflow: "visible",
          }}
        >
          <path
            d={`M 0 0 C 30 0, 30 ${isSrc
            fill="none"
            stroke={isSrc1 ? "#10b981"
            strokeWidth="3"
          />
        </motion.svg>
      )}
    </div>
  );
  }}}
</React.Fragment>
  )})
</div>
</div>
</div>

  {/* Formula helper text */}
  <div className="bg-slate-900 border border-slate-

```

```

const Viz2DBuild: React.FC = () => {
  const [k, setK] = useState(1);
  const [hover, setHover] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
  const [locked, setLocked] = useState<{ r: number; c: number }>({ r: 0, c: 0 });

  const L = 1 << k;
  const activeCell = locked || hover;

  return (
    <div className="flex flex-col xl:flex-row gap-6">
      <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
        <h3 className="text-xl font-bold text-indigo-400">Визуализация
        <p className="text-sm text-slate-400 mb-6 text-wrap">
          Для наглядности показываем только <b className="text-slate-400">
            <br/>
            <span className="text-rose-300 animate-pulse">
              иц!</span>
        </p>

        <div className="flex bg-slate-950 p-1 rounded-l-1">
          {[0, 1, 2, 3].map(step => (
            <button
              key={step}
              onClick={() => { setK(step); setHover(null); setLocked(null); }}
              className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
            >
              {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
            </button>
          ))}
        </div>

        <div className="flex flex-col items-center gap-6 no-scrollbar">
          {[...Array(k + 1)].map((_, i) => k - i).map((step, i) => {
            const stepL = 1 << step;
            const stepSize = 8 - stepL + 1;

```

```

highlightClass = 'bg-ros
md:scale-[1.08]';
    } else if (isTop && !isLeft
        highlightClass = 'bg-blur
] md:scale-[1.08]';
    } else if (!isTop && isLeft
        highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
    } else {
        highlightClass = 'bg-amb
3)] md:scale-[1.08]';
    }
    zIndex = 'z-10';
  }
}

return {
  <div
    key={idx}
    onMouseEnter={() => { if(is
    onMouseLeave={() => { if(is
    onClick={() => {
      if(isCurr) {
        if(locked && locked.
        else setLocked({r, c
      }
    }}
    className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
>
    {ST2D[r][c][step][step]}
  </div>
  );
  }}}
</div>
</div>
</div>

```

```

        &nbsp;&nbsp;<span className="text-amber-500">c + pL</span>][k-<span className="text-slate-500">// Правый Низ</span><b>
        {'}'}));
    </div>

    {activeCell ? (
      <div className="bg-slate-950 p-5 rounded-2xl mt-2 transition-all">
        <p className="text-slate-300 mb-3">
          <span>Пример для ячейки:</span>
          {locked && <span className="text-slate-500 border border-rose-500/20"><Lock size={
        </p>
        <p className="text-indigo-300 border">
          <span className="font-bold text-white">
            }}[k]][k]</span>
          <span className="text-slate-400">
        </p>

        <div className="space-y-2 pt-3 pl-2">
          {(() => {
            const prevL = 1 << (k - 1);
            return (
              <div className="flex items-center">
                <span className="flex-shrink-0 h-8px #f43f5e"></div> ST[{activeCell.r}[activeCell.c][k-1][k-1]}</span>
              </div>
                <div className="flex items-center">
                  <span className="flex-shrink-0 h-8px #3b82f6"></div> ST[{activeCell.r}[activeCell.c + prevL][k-1][k-1]}
                </div>
                <div className="flex items-center">

```



```

        </div>
      </label>
      <label className="flex items-center" >
        <span className="text-slate-500" >
          <div className="flex gap-2">
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
          </div>
        </label>
      </div>
    </div>
  </div>

```

```

<div className="flex-1 bg-slate-900 p-6 rounded" >
  <h3 className="text-xl font-bold text-emerald-400" >
    <p className="text-sm font-sans text-slate-400" >
      Мы знаем, что максимальная степень дв
    rounded text-white">{Lx}</span>. Аналогично
    te">{Ly}</span>.
  </p>
  <p className="text-sm font-sans text-slate-400" >
    Чтобы покрыть весь прямоугольник, мы
    Name="font-mono bg-[#0a0f1e] px-1 rounded b
    роса. (Цветные прямоугольники слева).
  </p>

```

```

    <div className="bg-[#0a0f1e] rounded-xl p-6" >
      text-slate-300 shadow-inner overflow-x-auto" >
        <div className="absolute right-3 top-3" >
          <span className="text-slate-500">// 1.
          <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
          <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
          <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>

```

```

        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;

        let color = "bg-slate-800 text-white";
        let txt = ST2D[r][c][kx][ky].text;

        let badge = null;
        if (isTL) { color = "bg-rose-400 text-white"; }
        else if (isTR) { color = "bg-blue-300 text-white"; }
        else if (isBL) { color = "bg-emerald-300 text-white"; }
        else if (isBR) { color = "bg-amber-300 text-white"; }

        return (
            <div key={idx} className={color}
            duration-300 relative shrink-0 ${c === c1 ? "w-10" : "w-12"}
            {txt}
            {badge && <div className="text-white font-bold rounded-sm px-1" />}
        )
    )
}
}
}
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white" />
<div className="text-center font-bold text-white" />
    <span className="text-slate-400 font-sans" />
    <span className="text-rose-400 font-sans" />
    <span className="text-blue-400 font-sans" />

```

```
      : 'px-3 py-1 rounded text-sm font-bold'

const aside = document.querySelector('aside')
const headerMain = document.querySelector('h1')
const questionsContainer = document.getElementById('questions')
const wtfContainer = document.getElementById('wtf')

if (mode === 'wtf') {
  document.body.classList.add('wtf-mode')
  if (aside) aside.style.display = 'none'
  if (headerMain) headerMain.style.display = 'none'
  if (questionsContainer) questionsContainer.style.display = 'none'
  if (wtfContainer) wtfContainer.classList.remove('hidden')

  // Remove the URL hash to avoid scrolling
  history.pushState("", document.title, window.location.pathname + window.search)
  window.scrollTo(0, 0);
} else {
  document.body.classList.remove('wtf-mode')
  if (aside) aside.style.display = '';
  if (headerMain) headerMain.style.display = '';
  if (questionsContainer) questionsContainer.style.display = '';
  if (wtfContainer) wtfContainer.classList.add('hidden')
}

function setTheme(theme) {
  const body = document.body;
  body.classList.remove('theme-dark', 'theme-light');
  body.classList.add('theme-' + theme);

  // Just some visual feedback on buttons
  document.querySelectorAll('[id^="theme-"]').forEach(btn => {
    btn.classList.remove('outline', 'outline-1');
  });

  if (theme === 'dark') document.getElementById('dark-button').classList.add('active')
  if (theme === 'light') document.getElementById('light-button').classList.add('active')
```

```

        setTimeout(() => drawerOverlay.
    }
    document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
    mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
    closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
    drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
    link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.c
            toggleDrawer();
        }
    }));
});

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
};

const observer = new IntersectionObserver((
    entries.forEach(entry => {
        if (entry.isIntersecting && !document
            // Highlight active TOC link lo
            const id = entry.target.id;
            document.querySelectorAll('aside
                if (link.getAttribute('href

```

```

    };
</script>
<style>
    @import url('https://fonts.googleapis.com/css2?
    @reference './src/index.css';
    .math-font { font-family: 'Fira Code', monospace; }
    .uno-card {
        width: 70px; height: 105px; border-radius: 10px;
        box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
        align-items: center; justify-content: center; color: white;
        overflow: hidden; flex-shrink: 0; text-shadow: 2px 2px 0 #000,
        -1px -1px 0 #000; font-family: 'Arial Black', Impact, sans-serif;
    }
    .uno-card::before, .uno-card::after {
        content: attr(data-val); position: absolute; text-shadow: 1px 1px 0 #000,
        -1px -1px 0 #000;
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
    .uno-card.mini { width: 45px !important; height: 60px !important; }
    .uno-card.mini::before, .uno-card.mini::after {
        font-size: 0.8em;
    }

    /* Global formula scroll settings */
    .formula-scroll {
        overflow-x: auto;
        overflow-y: hidden;
        -webkit-overflow-scrolling: touch;
        scrollbar-width: none; /* Firefox */
        -ms-overflow-style: none; /* IE and Edge */
    }
    .formula-scroll::-webkit-scrollbar {
        display: none;
    }

    mjx-container {
        max-width: 100% !important;
    }

```

```

    .card-blue { background-color: #004dcf; } .card
    .card-green { background-color: #339e35; } .card
    .card-yellow { background-color: #ffc400; } .ca
    .card-black { background-color: #222; } .card-b
    .card-black::before, .card-black::after { text-
    .info-block { @apply bg-slate-800/80 border-l-4
    .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
    ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
    king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
    r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
    important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
    background-color: #fcf8eb !important; /* li
    color: #3b2818 !important; /* brown ink */
    background-image: repeating-linear-gradient
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Se

```

```

</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

  <div class="sticky top-0 z-50 w-full bg-slate-950/90
    center items-center gap-4 sm:gap-8 transition-colors">
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">Режим</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setMode('normal')" id="btn-normal"
          transition-colors">Список тем</button>
        <button onclick="setMode('wtf')" id="btn-wtf"
          transition-colors">WTF</button>
      </div>
    </div>
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">Тема</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setTheme('dark')" id="btn-dark"
          transition-colors">Темная</button>
        <button onclick="setTheme('light')" id="btn-light"
          transition-colors">Светлая</button>
        <button onclick="setTheme('notebook')" id="btn-notebook"
          transition-colors">Заметки</button>
      </div>
    </div>
  </div>

  <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
  <svg width="0" height="0" class="absolute">
    <defs>
      <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="yellow">
        <path d="M0,0 L6,3 L0,6 Z" fill="yellow">
      </marker>
      <marker id="arrow-orange" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="orange">
        <path d="M0,0 L6,3 L0,6 Z" fill="orange">
      </marker>
      <marker id="arrow-pink" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="pink">
        <path d="M0,0 L6,3 L0,6 Z" fill="pink">
      </marker>
    </defs>
  </svg>

```

```

        <path d="M 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#333"
        <!-- Выход -->
        <path d="M 5 45 L -5 45 L -5 55 L 5 55"
        <circle cx="25" cy="50" r="15" fill="#f
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="10"
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform
        <svg xmlns="http://www.w3.org/2000/svg" class="
            <path stroke-linecap="round" stroke-linejoin
        </svg>
    </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0
    background-color: #000; opacity: 0.5; z-index: 1000;

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col
    flex-grow: 1;

    <!-- Левое меню (Оглавление) -->
    <aside id="mobile-drawer" class="fixed inset-y-0 left-0
        translate-x-full transition-transform duration-300
        w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
        <div class="h-full flex flex-col p-6 overflow-y-auto

```



```
        <a href="#section-2-2" class="block">
        <a href="#section-2-3" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">
        <a href="#question-3" class="block">
            3. Тригонометрическая форма
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-teal-300">
        <a href="#section-3-1" class="block">
        <a href="#section-3-2" class="block">
        <a href="#section-3-3" class="block">
        <a href="#section-3-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
        <a href="#question-4" class="block">
            4. Степени и корни
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-amber-300">
        <a href="#q4-def" class="block">
        <a href="#q4-moivre" class="block">
        <a href="#q4-root" class="block">
        <a href="#q4-count" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
        <a href="#question-5" class="block">
```

```
<div class="pl-6 space-y-2 text
  <a href="#q7-1" class="bloc
  <a href="#q7-2" class="bloc
  <a href="#q7-3" class="bloc
  <a href="#q7-4" class="bloc
  <a href="#q7-5" class="bloc
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
      8. Сравнимость и Поля В
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q8-1" class="bloc
    <a href="#q8-2" class="bloc
    <a href="#q8-3" class="bloc
    <a href="#q8-4" class="bloc
    <a href="#q8-5" class="bloc
    <a href="#q8-6" class="bloc
    <a href="#q8-7" class="bloc
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
      9. Кольцо многочленов
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q9-1" class="bloc
    <a href="#q9-2" class="bloc
    <a href="#q9-3" class="bloc
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
        12. Взаимно простые многочлены)
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-blue-400">
      <a href="#q12-1" class="block">
      <a href="#q12-2" class="block">
      <a href="#q12-3" class="block">
    </div>
  </details>
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
        13. Корни многочленов.
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-emerald-400">
      <a href="#q13-1" class="block">
      <a href="#q13-2" class="block">
      <a href="#q13-3" class="block">
    </div>
  </details>
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
        14. Кратность корня. Коэффициент
      </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2 text-fuchsia-400">
```

```

<div class="mt-4 mb-2 text-xs font-l
  <a href="#question-17" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-tea
  </a>
  <a href="#question-18" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-blur
  </a>
  <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
    <span class="font-bold text-eme
  </a>
  <a href="#question-20" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-ros
  </a>
  <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
    <span class="font-bold text-ind
  </a>
  <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
    <span class="font-bold text-fuc
  </a>

  <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

    ▲ Ликбез: Алгоритмы выживания
  </a>

</nav>
<script>
  document.querySelectorAll('details.
    detail.addEventListener('toggle
      if (detail.open) {
        document.querySelectorA
          if (other !== detail

```

```

        const corresponding
        if (correspondingLi
            const details =
            if (details &&
                details.open
                document.qu
                if (othe
                othe
            }
        }));
    }
}
});
}, {
    root: null,
    rootMargin: '-10% 0px -70%
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

function fallbackModal(contentRaw,
    const modal = document.createEl
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';
    modal.style.height = '100vh';
    modal.style.backgroundColor = '
    modal.style.zIndex = '99999';
    modal.style.display = 'flex';
    modal.style.flexDirection = 'co
    modal.style.alignItems = 'cente
    modal.style.justifyContent = 'c

```

```

copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(textarea.value);
        copyBtn.innerText = 'Скопировано';
    } else {
        textarea.select();
        document.execCommand('copy');
        copyBtn.innerText = 'Скопировано';
    }
    setTimeout(() => copyBtn.innerText = 'Копировать', 1000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

```


ВЫСШАЯ АЛГЕБРА

</h1>

<p class="text-slate-400 italic text-lg">

<!-- Print Table of Contents -->

<div class="hidden print:block mt-8 text-center: always;">

<h2 class="font-bold text-xl mb-4">

<ul class="text-base space-y-2">

1. Ал

2. Ко

3. Тр

4. Ст

5. Гр

6. Де

7. Ал

8. Кл

9. Ко

10. ,

11. <

12. l

13. l

14. l

15. <

17. l

18. >

19. l

20. l

21. l

22. l

</div>

</header>

<!-- ===== WTF CONTAINER =====>

<div id="wtf-container" class="hidden">

<h2 class="text-3xl font-black text-cen

```

<a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-500"
    <div class="font-bold text-slate-700"
</a>

<!-- Row 3: НОД и НОК / Алгоритм Ев
<a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow-500"
    <div class="font-bold text-slate-700"
</a>
<a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500"
    <div class="font-bold text-slate-700"
</a>
<!-- Empty 3rd row cell -->
<div class="hidden md:block"></div>

<!-- Row 4: Факторизация / Структур
<a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow-500"
    <div class="font-bold text-slate-700"
</a>
<a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500"
    <div class="font-bold text-slate-700"
</a>
<a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-500"
    <div class="font-bold text-slate-700"
</a>

<!-- Row 5: Корни / Уравнения -->

```

```

        </a>
        <a href="#question-18" onclick="setf
-l-4 border-blue-500 hover:bg-slate-700 tra
        <div class="text-xs text-blue-50
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-19" onclick="setf
-l-4 border-emerald-500 hover:bg-slate-700
        <div class="text-xs text-emerald-500
        <div class="font-bold text-slate-700"
        </a>

        <a href="#question-20" onclick="setf
-l-4 border-rose-500 hover:bg-slate-700 tra
        <div class="text-xs text-rose-500
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
        <div class="text-xs text-indigo-500
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
        <div class="text-xs text-fuchsia-500
        <div class="font-bold text-slate-700"
        </a>

        <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
        <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

        <a href="#proof-algorithms" onclick="setf
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
        <div class="text-xs text-red-500
        <div class="font-bold text-slate-700"
        </a>
    </div>

```

```

<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-fuchsia" style="color: #f080f0;">
  <p class="text-sm text-slate" style="color: #a6b8c8;">
  <ul class="list-disc pl-5" style="list-style-type: none;">
    <li><span class="text-white" style="color: white;">
      i>
    <li><span class="text-white" style="color: white;">
      т.д.).</li>
    <li><span class="text-white" style="color: white;">
      что так и задумано: "Ой, противоречие, теор
    </ul>
  </div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-emerald" style="color: #2ecc71;">
  <p class="text-sm text-slate" style="color: #a6b8c8;">
  <ul class="list-disc pl-5" style="list-style-type: none;">
    <li><span class="text-white" style="color: white;">
      ,  $r_2$ ).</li>
    <li><span class="text-white" style="color: white;">
    <li><span class="text-white" style="color: white;">
      счет ограничений (размер остатка) докажи, ч
    </ul>
  </div>

```

```

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-yellow" style="color: #f1c40f;">
  <p class="text-sm text-slate" style="color: #a6b8c8;">
  <ul class="list-disc pl-5" style="list-style-type: none;">
    <li><span class="text-white" style="color: white;">
      валось.</li>
    <li><span class="text-white" style="color: white;">
      Просто перепиши формулу с буквой  $k$ .</li>
    <li><span class="text-white" style="color: white;">
      усок  $k$ , замени его на жареного карася из
    </ul>

```

```
        <p class="text-sm text-slate-400">
        <p class="text-xs font-medium text-slate-400">
ициентов (они делятся на $p$), кроме старшего,
зложить.</p>
```

```
</div>
```

```
    <div class="bg-slate-800 p-4">
      <h4 class="text-red-400 font-medium">
      <p class="text-sm text-slate-400">
      <p class="text-xs font-medium text-slate-400">
. Чаще всего после этого Фейс-контроль $p$ .
```

```
</div>
```

```
</div>
```

```
    <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
    <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
    <ul class="space-y-4 pb-10">
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
          <h4 class="text-cyan-400 font-medium">
          <p class="text-sm text-slate-400">
          <code class="text-x"
        </div>
      </li>
```

```
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
          <h4 class="text-cyan-400 font-medium">
          <p class="text-sm text-slate-400">
х двух, потом последних. <br>
          <code class="text-x
```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

ФАЙЛ 75/82: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а для элементов
 * которые в приложении всплывают по наведению, собираю их отдельно
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при отсутствии
 * async function collectCss(): Promise<string> {
 *   const parts: string[] = [];
 *   for (const sheet of Array.from(document.styleSheets)) {
 *     try {
 *       const rules = (sheet as CSSStyleSheet).cssRules;
 *       parts.push(Array.from(rules).map((r) => r.cssText));
 *     } catch {
 *       // ignore
 *     }
 *   }
 *   return parts.join("\n");
 * }
```

```

}
`;

const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string): { body: string; termIds: string[] } {
  const host = document.createElement("div");
  host.innerHTML = html;

  try {
    annotateTerms(host);
  } catch {
    /* если браузер не осилил регулярку – выгружаем текст как есть */
  }

  const ids: string[] = [];
  host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
    const id = el.dataset.termId;
    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
  });

  // <details> в файле лучше раскрыть: иначе код и разбегается
  host.querySelectorAll("details").forEach((d) => d.setOpen());

  return { body: host.innerHTML, termIds: ids };
}

```

```

<div style="font-size:.7rem;text-transform:uppercase"
<h1 style="color:#fff;font-size:1.6rem;font-weight:
<div style="font-size:.72rem;color:#475569;margin-t
</header>

<div class="export-note">
  Это автономная копия: стили внутри файла, интернет
  Интерактивные тренажёры и анимации в статичный HTML
  они живут в онлайн-версии: <a href="${escapeHtml(op
</div>

<article class="prose prose-invert max-w-none">
${opts.body}
</article>

${opts.gloss}

<footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
  const num = title.match(/^\\s*(\\d+)/)?.[1];

```



```
-----
const body = `
```

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 76/82: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
-----
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 *   node scripts/audit-coverage.mjs           # отчёт
 *   node scripts/audit-coverage.mjs --md      # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
```

```

});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && !
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
        ge)} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  * ${r.title} [$
    console.log();
  };
  dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

-----
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi)
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights).padStart(3),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
  );
}
console.log("\nГлав без единой подсказки:", bad.length,);
console.log("Терминов в глоссарии:", GLOSSARY.length,);
console.log("Не встретились:", GLOSSARY.filter((g) => !ids.has(g)).length,);

```

```
        CreationDate: new Date(),
      },
    ));

    const stream = fs.createWriteStream(PDF_PATH);
    doc.pipe(stream);

    for (const file of pages) {
      doc.addPage({ size: [A4.width, A4.height], margin: 0 });
      doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
    }

    doc.end();
    await new Promise((resolve, reject) => {
      stream.on("finish", resolve);
      stream.on("error", reject);
    });

    console.log("[3/3] png -> pdf");
    console.log(`      страниц: ${pages.length}`);
    console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
  }

  main().catch((err) => {
    console.error("Ошибка сборки PDF:", err.message);
    process.exit(1);
  });
```

ФАЙЛ 79/82: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.2 КБ

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
```

```

    "Конфигурация проекта",
    "Ядро приложения",
    "Контент и данные пособия",
    "Билеты (учебный контент)",
    "Интерактивные компоненты и симуляторы",
    "Визуализаторы (вложенные)",
    "HTML-разметка (layout)",
    "Утилиты",
    "Скрипты сборки и экспорта",
    "CI / автоматизация",
    "Прочее",
];

function listFiles() {
  let files;
  try {
    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
      .split("\n")
      .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))

```

```
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { time: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push("                УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push("                ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОММЕНТАРИИ)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalBytes)}`);
push(`Глав/билетов в пособии:         ${chapters.length}`);
push();

push(HR);
push("                                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

push(HR);
push("                                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")}. ${rel}`);
});
push();
```

```
main();
```

ФАЙЛ 80/82: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt      (collect-code.m
 * txt  -> page-XXXX.png              (render-pages.m
 * png  -> full_code_all_pages.pdf    (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLTo

/** Куда кладём финальные артефакты (эта папка отдаётся
export const OUT_DIR = path.join(ROOT, "public", "export
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжа
export const PAGES_DIR = path.join(ROOT, "tmp", "export

export const TXT_PATH = path.join(OUT_DIR, "full_code_a
export const PDF_PATH = path.join(OUT_DIR, "full_code_a

/** Параметры вёрстки страницы (подобраны под A4 @150dp
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
```

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize,
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(/<\/p>
  if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min((expanded.match(/<\/p>
  let rest = expanded;
  let first = true;
  while (rest.length > 0) {
    const width = first ? PAGE.cols : PAGE.cols - indent;
    parts.push((first ? " " : indent) + rest.slice(0, width));
    rest = rest.slice(width);
    first = false;
  }
  return parts;
}
```



```

    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
    "-pointsize", String(PAGE.pointsize),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(" ")}

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE.pngFile)}`);

```

```
workflow_dispatch:
  inputs:
    density:
      description: "DPI растеризации страниц (по умолчанию)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends
```

- name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v4
with:
 - name: full-code-export
 - path: |
 - public/export/full_code_all_pages.pdf
 - public/export/full_code_all_pages.txt
 - if-no-files-found: error
 - retention-days: 30
- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v4
with:
 - name: full-code-pages-png
 - path: tmp/export-pages/*.png
 - if-no-files-found: error
 - retention-days: 7
- name: Commit rebuilt export back to the branch
run: |
 - git config user.name "github-actions[bot]"
 - git config user.email "41898282+github-action"
 - git add -- public/export/full_code_all_pages.pdf
 - if git diff --cached --quiet; then
 - echo "Экспорт не изменился — коммит не нужен"
 - exit 0
 - fi
 - git commit -m "chore(export): пересборка full-code-export"
 - git push origin "HEAD:\${GITHUB_REF_NAME}"